Sig-Net™ Protocol Framework

V1.0

Copyright Singularity (UK) Ltd 2026

Table of Contents

Table of Figures 9

Preface 10

Acknowledgements 11

1 Introduction 12

1.1 Overview 12

1.2 Conditions of Use 12

1.3 Compliance Notice & Disclaimer 12

1.4 Vulnerability Disclosure 13

2 References to other Documents 13

2.1 Normative References 13

2.2 Informative References 14

3 Definitions 14

3.1 Document Conventions 14

3.2 Byte Order 15

3.3 Octet and Byte 15

4 Security Concepts 15

4.1 Introduction 15

4.2 Secure Design Rationale 15

4.3 Provenance and Attestation 15

4.4 Encryption 16

4.5 Authentication 16

4.6 Integrity 17

4.7 Authorisation 17

4.8 Freshness 17

4.9 Security Lifecycle & Extensibility (Informative) 17

4.9.1 Firmware Updates 17

4.9.2 Key Rotation 17

4.9.3 Future Encryption: 18

5 Threat Vectors 18

5.1 Introduction 18

5.2 Man-in-the-Middle 18

5.3 Masquerading and Spoofing 18

5.4 Replay Attack 18

5.5 Rogue device 18

5.6 Eavesdropping & Sniffing 18
5.7 Compromised Node or Sender 18
5.8 Compromised Manager 18
5.9 Offboarded Beacon Spoofing 19
6 System Architecture 19
6.1 Structure 19
6.2 Functional Roles 19
6.3 Trust 20
6.4 Air-Gap 20
6.5 SoemCode 20
6.6 Transport Unique Identifier (TUID) 21
6.7 TUID and E1.20 UID Relationship 21
6.8 Endpoints 21
6.8.1 Root Endpoint 21
6.8.2 Data Endpoints 22
6.8.3 Reserved Endpoints 22
6.8.4 Broadcast Endpoint 22
6.8.5 Endpoint Examples 22
7 Root Key and Key Derivation 22
7.1 Introduction 22
7.2 Root Key (K0) Specification 23
7.2.1 Entropy Validation 23
7.2.2 Offboarded Operation (Beacon Mode) 23
7.2.3 Input and Transfer Formats 24
7.2.4 First-Packet Bootstrapping 25
7.2.5 Open Mode (Unauthenticated Operation) 26
7.3 Key Derivation Function (KDF) 26
7.3.1 Key Lifecycle and Erasure 28
7.4 Human Readable Format (Display and Verification) 28
7.5 Machine Readable Format (Out-of-Band Transfer) 28
7.5.1 Restricted Key Export (Guest Access) 28
7.6 Storage and Security 29
7.7 Offboarding and Erasure 30
7.7.1 Authenticated Network Offboarding 30
7.7.2 Physical Out-of-Band Offboarding 30

8 Packet Structure 30
8.1 CoAP Summary (Informative) 30
8.2 URI Syntax Overview 31
8.3 Security Options 31
8.4 Packet Structure 33
8.5 HMAC Calculations 34
8.6 Responder Processing and Replay Attack Prevention 34
8.6.1 Manager Processing of Replies 37
9 Transport 37
9.1 Multicast Use 37
9.2 Multicast Address Mapping 37
9.2.1 URI to Multicast Determinism 37
9.2.2 Discovery and Management Subscriptions 37
9.2.3 Default Multicast Folding™ (Universe Mapping) 37
9.2.4 Multicast Folding™ Configuration and Overrides 38
9.2.5 Multicast Interoperability and Safety Reset 38
9.2.6 Sync and Timecode Address 39
9.3 Port Assignment 39
9.4 CoAP Message Type and Method 39
9.5 Network Infrastructure Requirements 39
9.6 Multicast TTL 39
10 Payload Syntax 39
10.1 Introduction 40
10.1.1 TID Allocation and Namespaces 40
10.1.2 Proprietary TID Resolution 40
10.2 Node Discovery and Polling 40
10.2.1 Offboarded Operation (Beacon Mode) 40
10.2.2 Manager Operation 42
10.2.3 Node Operation 42
10.2.4 Fragmentation 43
10.2.5 On-Boot Notification 43
10.2.6 Node Loss Detection (Lost Mode) 43
10.2.7 State Synchronisation and Consistency Tracking 44
10.3 Targeted Node Routing 45
10.3.1 Targeted URIs 45

10.3.2 Unicast vs. Multicast Command Routing: 45
10.4 Node Configuration and State Synchronisation 46
10.4.1 Directional URI Mapping 46
10.4.2 Parameter Updates (SET) 46
10.4.3 Parameter Queries (GET) 46
10.4.4 Batch State Retrieval (QUERY_LEVEL) 47
10.4.5 Network Interface Configuration (IP Settings) 47
10.5 RDM Payload 47
10.5.1 Transaction Responses 47
10.5.2 Proactive State Notification 48
10.5.3 RDM Boundary Restrictions (Normative) 48
10.5.4 Gateway Responsibilities 49
10.6 Universe Payload 49
10.6.1 Example Universe Payload (Informative) 50
10.6.2 Routing & Mixed Security Environments 51
10.6.3 Transmission Rates and Keep-Alive 51
10.6.4 Stream Loss Timeout: 52
10.7 Sync Payload 52
10.7.1 Transmission Rules 52
10.7.2 Processing Rules and the SYNC_ACTIVE State 53
10.7.3 Synchronisation Timeout 53
10.8 Timecode Payload 53
10.8.1 Transmission Rules 53
10.8.2 Timecode Stream Loss 54
10.9 Security Event Reporting 54
10.9.1 Event Counters 54
10.9.2 Offending Address Capture 54
10.9.3 Transmission and Rate Limiting 54
11 TID Definitions 55
11.1 Node-Discovery Type Identifiers 55
11.1.1 TID_POLL 55
11.1.2 TID_POLL_REPLY 55
11.2 Sender Type Identifiers 56
11.2.1 TID_LEVEL 56

11.2.2 TID_PRIORITY 56
11.2.3 TID_PREVIEW 56
11.2.4 TID_SYNC 56
11.2.5 TID_TIMECODE 56
11.2.6 TID_UNIVERSE 57
11.3 RDM Type Identifiers 59
11.3.1 TID_RDM_COMMAND 59
11.3.2 TID_RDM_RESPONSE 59
11.3.3 TID_RDM_TOD_CONTROL 59
11.3.4 TID_RDM_TOD_DATA 59
11.3.5 TID_RDM_TOD_BACKGROUND 60
11.3.6 TID_RDM_FLOW_CONTROL 60
11.4 Offboarding Type Identifiers 61
11.4.1 TID_RT_OFFBOARD 61
11.5 Network Configuration Type Identifiers 62
11.5.1 TID_NW_MAC_ADDRESS 62
11.5.2 TID_NW_IPV4_MODE 62
11.5.3 TID_NW_IPV4_ADDRESS 62
11.5.4 TID_NW_IPV4_NETMASK 62
11.5.5 TID_NW_IPV4_GATEWAY 63
11.5.6 TID_NW_IPV4_CURRENT 63
11.5.7 TID_NW_IPV6_MODE 63
11.5.8 TID_NW_IPV6_ADDRESS 63
11.5.9 TID_NW_IPV6_PREFIX 64
11.5.10 TID_NW_IPV6_GATEWAY 64
11.5.11 TID_NW_IPV6_CURRENT 64
11.6 Root Endpoint Type Identifiers 65
11.6.1 TID_RT_SUPPORTED_TIDS 65
11.6.2 TID_RT_ENDPOINT_COUNT 65
11.6.3 TID_RT_PROTOCOL_VERSION 65
11.6.4 TID_RT_FIRMWARE_VERSION 65
11.6.5 TID_RT_DEVICE_LABEL 66
11.6.6 TID_RT_MULT_OVERRIDE 66
11.6.7 TID_RT_IDENTIFY 66

11.6.8 TID_RT_STATUS 67
11.6.9 TID_RT_ROLE_CAPABILITY 67
11.6.10 TID_RT_REBOOT 67
11.6.11 TID_RT_MODEL_NAME 68
11.6.12 TID_RT_OTW_CAPABILITY 68
11.7 Data Endpoint Type Identifiers 69
11.7.1 TID_EP_UNIVERSE 69
11.7.2 TID_EP_LABEL 69
11.7.3 TID_EP_MULT_OVERRIDE 69
11.7.4 TID_EP_CAPABILITY 69
11.7.5 TID_EP_DIRECTION 70
11.7.6 TID_EP_INPUT_PRIORITY 70
11.7.7 TID_EP_STATUS 71
11.7.8 TID_EP_FAILOVER 71
11.7.9 TID_EP_DMX_TIMING 72
11.7.10 TID_EP_REFRESH_CAPABILITY 72
11.7.11 TID_EP_PROTOCOL 73
11.8 Diagnostic Endpoint Type Identifiers 74
11.8.1 TID_DG_SECURITY_EVENT 74
11.8.2 TID_DG_MESSAGE 74
11.8.3 TID_DG_LEVEL_FOLDBACK 74
11.9 TID Polling Cross Reference 75
11.10 TID Informative Summary 77
12 Protocol Versioning and Compatibility 79
12.1 Document and URI Relationship 79
12.2 Node Versioning Requirements 79
12.3 Manager Compatibility Requirements (Normative) 79
12.4 Sender Compatibility Requirements 79
13 Example Sig-Net Transaction (Informative) 80
13.1 Sig-Net Node Discovery 80
13.2 Example Node Endpoint Configuration 80
13.3 Example Multi-Manager Environments 81
13.3.1 Equal Managers 81
13.3.2 Guest Manager 81
14 Implementation Guidelines (Informative) 82

14.1 Hardware Considerations 82
14.1.1 Non-Volatile Memory Wear Endurance 82
14.1.2 Key Storage Hardware 82
14.1.3 Multicast Folding™ Load Scaling 82
14.2 Network Considerations 83
14.2.1 CoAP Port Coexistence 83
14.2.2 Multicast Address Exclusivity 83
14.3 Manufacturer Specific Extensions 83
15 Appendix A Multicast Addresses 84
16 Appendix B Definitions 85
17 Appendix C Machine-Readable TID Dictionary (Informative) 87
18 Appendix D Glossary 89
19 Appendix E CRA and ENISA Compliance Reference (Informative) 92
19.1 CRA Annex 1, Part 1 Mapping 92
19.2 ENISA Security by Design and Default Playbook Mapping 93
20 Appendix F Architectural Rationale (Informative) 94
20.1 Why CoAP? 94
20.2 Why Cleartext Routing 94
20.3 Why Out-of-Band Keying? 94
20.4 Why a Custom HMAC structure instead of OSCORE? 95
20.5 Why a Custom TLV structure instead of CBOR? 95
20.6 Why CoAP NON POST only? 95
20.7 Why multicast RDM responses? 95
20.8 Future Extensibility 95
21 Appendix G Document Revisions (Informative) 97

Table of Figures

Figure 1 Summary of Key Derivation 27
Figure 1 Summary of Derived Key Usage 27
Figure 2 Wireshark Example 51
Figure 2 Example TID Polling Cycle 80
Figure 3 Example Endpoint Configuration 80

Preface

Cybersecurity legislation, notably the EU Cyber Resilience Act (CRA), will imminently impact upon the entertainment technology industry.

Sig-Net aims to provide a CRA-defensible communication framework. It has five primary goals:

- Offer a level of security defensible under the CRA.
- Support DMX, RDM, timecode, and firmware upload.
- Re-use existing protocol structures and syntax wherever possible.
- Operate on a lightweight stack that can be implemented in a very short development cycle.
- Be scalable, supporting multiple Senders and very large universe counts.

What does "CRA Defensible" mean? This is a highly subjective question, best answered with an example. A simplistic reading of CRA Annex I, Part I, Point 2(e) suggests that encryption is mandated across the board. However, a more nuanced reading suggests that the risk assessment required by Article 13(2) defines the true applicability of the annex. Did the authors of the CRA genuinely intend for DMX lighting levels to be treated with the same cryptographic weight as a financial transaction? Recital (55) of Regulation (EU) 2024/2847 explicitly acknowledges the validity of this reading by stating that manufacturers may be required to "follow widely recognised interoperability standards even if its security features are no longer considered to be state of the art."

One of the core philosophies of Sig-Net is to reuse proven concepts rather than reinventing the wheel. The following list identifies the existing protocol elements that Sig-Net re-uses, either conceptually or physically:

- sACN: The 1:1 universe-to-multicast mapping creates scalability limits on standard network switches (IGMP exhaustion). However, the core concept of basing Sig-Net on multicast is correct and has been retained, albeit with an improved Multicast Folding strategy.
- Art-Net (DMX): There is almost no difference between how sACN and Art-Net handle DMX other than transport routing. Sig-Net uses multicast but fixes the sACN IGMP scalability issue.
- Art-Net (Discovery): The ArtPoll mechanism is universally understood, lightweight, and stateless. Sig-Net uses the concept of continuous polling but includes options for segmenting node-discovery to avoid network storms.
- Art-Net (RDM): The flat architecture and streaming approach to RDM in Art-Net have been highly successful. Sig-Net retains that concept but adds native multi-Sender synchronisation.
- Art-Net (Timecode): The simple multi-stream approach to timecode is well-liked and has been retained in Sig-Net.
- RDMnet (RPT): RDMnet has not seen widespread support, primarily because the requirement for a central broker has not been well received. Sig-Net retains a flat streaming approach and allows multi-Sender operation via passive snooping instead of a broker.
- RDMnet (LLRP): LLRP has been well received but, by its design intent, does not fit cleanly into a secured environment. That said, Sig-Net includes robust node recovery and configuration mechanisms.
- IoT: The IoT industry went through a lot of pain and embarrassment dealing with security. They have learnt a lot in recent years and Sig-Net builds on that, reusing protocols such as CoAP.

From these conclusions, the specification below has evolved.

Wayne Howell

February 2026

Acknowledgements

Singularity (UK) Ltd would like to thank for following individuals for their assistance in the evolution of Sig-Net:

Chris Kennedy

Thierry Dupont

Alan Giraudon

Karen Howell

Red Walter

Andrew Berry

Tracy Fitch

Simon Canins

Peter Newman

Ainars Pastars

Lars Wernlund

Shep Dick

1 Introduction

1.1 Overview

This protocol specification describes a method of transferring lighting control and timing data via a network. It introduces a level of network security that may allow users to comply with aspects of cybersecurity standards and legislation such as the EU Cyber Resilience Act (CRA), California SB-327, and Oregon HB-2395.

1.2 Conditions of Use

Sig-Net™ is a proprietary protocol designed and owned by Singularity (UK) Ltd.

Singularity (UK) Ltd hereby grants a perpetual, worldwide, non-exclusive, royalty-free, irrevocable license to any person or entity ("Licensee") to develop, manufacture, and distribute software or hardware that implements the Sig-Net protocol, subject to the following conditions:

1. Attribution: The Licensee shall include the following credit in the product's user manual, digital 'About' screen, or accompanying technical documentation: "Sig-Net™ Designed by and Copyright Singularity (UK) Ltd."
2. Product Identification: The Licensee shall assign a unique product identifier (SoemCode) for each product or product variant, and report it accurately during discovery polling, as defined in Section 6.5.
3. Trademarks: The Licensee acknowledges that Sig-Net™ and Multicast Folding™ are trademarks of Singularity (UK) Ltd. When using these trademarks in product marketing or user interfaces, the Licensee agrees to adhere to the branding guidelines available at www.Sig-Net.info.

By implementing Sig-Net in any product or system, the Licensee agrees to be bound by these conditions. This licence is governed by the laws of England and Wales. Any disputes arising under or in connection with this licence shall be subject to the exclusive jurisdiction of the courts of England and Wales.

1.3 Compliance Notice & Disclaimer

Compliance with this protocol specification is the sole and exclusive responsibility of the manufacturer or provider and is entirely within their control and discretion. Any markings, identification or other claims of compliance do not constitute certification or approval of any type or nature whatsoever by Singularity (UK) Ltd (SUL).

SUL neither guarantees nor warrants the accuracy or completeness of any information published herein and disclaims liability for any personal injury, property or other damage or injury of any nature whatsoever, whether special, indirect, consequential or compensatory, directly or indirectly resulting from the publication, use of, or reliance on this document.

Sig-Net is provided by SUL "AS IS", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability or fitness for a particular regulatory purpose.

Singularity (UK) Ltd does not warrant that implementations of Sig-Net™ will not infringe upon the intellectual property rights or patents of third parties.

While Sig-Net incorporates security concepts designed to assist manufacturers in securing their network interfaces, use of this protocol does not guarantee compliance with the EU Cyber Resilience Act (CRA), the UK Product Security and Telecommunications Infrastructure (PSTI) Act, California SB-327, Oregon HB-2395, or any other cybersecurity legislation.

Secure operation of a Sig-Net network is a shared responsibility, requiring proper product implementation by the manufacturer and diligent lifecycle management by the system integrator and end user.

Regulatory compliance is evaluated on the final manufactured product (including hardware design, key storage mechanisms, firmware implementation, and lifecycle management). Compliance is the sole and exclusive responsibility of the manufacturer implementing the protocol. SUL shall not be held liable for any regulatory fines, product recalls, compliance failures, or damages of any nature directly or indirectly resulting from the implementation of this document.

1.4 Vulnerability Disclosure

SUL maintains a vulnerability disclosure policy for this specification. Security vulnerabilities discovered in the Sig-Net protocol design should be reported to: office@singularity-uk.com.

Manufacturers implementing Sig-Net are independently responsible for establishing and publishing their own vulnerability handling processes as required by Article 13(6) of the EU Cyber Resilience Act.

2 References to other Documents

2.1 Normative References

This section details external documents and standards that are referenced by this specification.

- [CoAP] RFC 7252 The Constrained Application Protocol (CoAP)

  This standard is maintained by the IETF.

- [E1.31] ANSI E1.31 - 2025 Entertainment Technology - Lightweight streaming protocol for transport of DMX512 using ACN.

  This standard is maintained by ESTA.

- [E1.33] ANSI E1.33 Entertainment Technology – Message Transport and Device Management of ANSI E1.20 (RDM) over IP Networks (RDMnet).

  This standard is maintained by ESTA.

- [E1.37-2] ANSI E1.37-2 Entertainment Technology – Additional Message Sets for ANSI E1.20 (RDM) – Part 2, IPv4 & DNS Configuration Messages.

  This standard is maintained by ESTA.

- [E1.37-7] ANSI E1.37-7 Entertainment Technology – Additional Message Sets for ANSI E1.20 (RDM) – Gateway and Splitter Configuration.

  This standard is maintained by ESTA.

- [HMAC] RFC 2104 HMAC: Keyed-Hashing for Message Authentication

  This standard is maintained by the IETF.

- [HKDF] RFC 5869 HMAC-based Extract-and-Expand Key Derivation Function (HKDF).

  This standard is maintained by the IETF

- [SHA] FIPS PUB 180-4 Secure Hash Standard (SHS)

  This standard is maintained by the NIST.

- [Shannon] C. E. Shannon, "A Mathematical Theory of Communication," in The Bell System Technical Journal, vol. 27, no. 3, pp. 379-423, July 1948.

  This paper defines the foundational mathematics of information entropy.

- [CSPRNG] NIST Special Publication 800-90A Revision 1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators.

  This standard is maintained by the National Institute of Standards and Technology (NIST).

- [DMX] ANSI E1.11 - 2024 Entertainment Technology – USITT DMX512-A Asynchronous Serial Digital Data Transmission Standard for controlling lighting equipment and accessories.

  This standard is maintained by ESTA.

- [RDM] ANSI E1.20 - 2025 Entertainment Technology – Remote Device Management over DMX512 networks.

  This standard is maintained by ESTA.

- [Group OSCORE] No RFC yet, in development.

  This document is maintained by the IETF.

- [OSCORE] RFC 8613 Object Security for Constrained RESTful Environments (OSCORE).

  This standard is maintained by the IETF.

- [URI] RFC 3986 Uniform Resource Identifier (URI): Generic Syntax.

  This standard is maintained by the IETF.

- [ETC 0xDD] ETC, Inc 0xDD Alternate START Code Definition

  ETC, Inc (https://www.etcconnect.com)

2.2 Informative References

This section details external legal frameworks, guidelines, and regulatory texts that inform the secure design rationale of this specification but do not dictate protocol mechanics.

- [CRA] Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) No 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act). https://eur-lex.europa.eu/eli/reg/2024/2847/oj
- [ENISA] European Union Agency for Cybersecurity (ENISA), Security by Design and Default Playbook, 2026.
- [PSTI] United Kingdom Product Security and Telecommunications Infrastructure Act 2022.

3 Definitions

3.1 Document Conventions

A name enclosed in square brackets (e.g. [CoAP]) refers to an external document (Section 2.1) .

A name enclosed in angle brackets (e.g. <mult_poll>) is a symbolic name whose value is defined in Section 16.

3.2 Byte Order

All multi-byte data shall be transmitted in network byte order (Big-Endian).

3.3 Octet and Byte

Octet is an eight-bit byte. Octet and byte are used interchangeably in this document.

4 Security Concepts

4.1 Introduction

This section introduces key security concepts and discusses how Sig-Net interacts with them.

4.2 Secure Design Rationale

Security reviewers may note that Sig-Net uses custom CoAP Options to carry cryptographic data rather than adopting the [OSCORE] or [Group OSCORE] standards. This departure is a deliberate, necessary architectural decision driven by the extreme real-time constraints of live entertainment control:

- Compute Overhead and Multicast State: Standard secure CoAP frameworks rely on Authenticated Encryption with Associated Data (AEAD) ciphers, such as AES-CCM or ChaCha20-Poly1305. Applying these to a live lighting environment introduces two severe barriers for baseline embedded hardware. First, enforcing AES decryption in software on a mid-tier DMX Gateway receiving 8 multiplexed universes (processing 352 packets per second) exceeds the real-time frame budget, causing dropped frames. Second, while alternative software-optimised ciphers (like ChaCha20) reduce CPU load, all AEAD ciphers require the strict, synchronised management of unique Initialisation Vectors (Nonces). In a decentralized, multi-Sender UDP multicast environment, guaranteeing global Nonce uniqueness without a central broker introduces unacceptable architectural fragility. Sig-Net's use of symmetric HMACs solves both issues, providing stateless integrity and implicit authentication that remains viable on constrained silicon.
- Cleartext Routing: [OSCORE] encrypts the CoAP Uri-Path as an inner option to prevent traffic analysis. However, Sig-Net explicitly relies on the URI (e.g. /sig-net/v1/<scope>/level/11) as a hardware-layer routing filter. By keeping the URI in cleartext and appending the HMAC as a trailing CoAP Option, network switches and multi-port Gateways can parse routing targets at line-rate, dropping irrelevant multicast universes without wasting CPU cycles on cryptographic verification. See Section 20.2.
- Forward Compatibility: The Sig-Net-Security-Mode (Option 2076) explicitly provides a mechanism to support future payload encryption ciphers should legislative guidance eventually mandate the confidentiality of lighting levels, while currently prioritising the processing speed required by the industry.

4.3 Provenance and Attestation

Device provenance is the process of proving a device's origin and that it is genuine hardware. Sig-Net does not directly verify cryptographic device provenance over the network (Attestation). Instead, an operator must physically verify a device before manually providing it with the Root Key (aka K0).

The Manager, which permanently retains K0, represents the highest-value physical security target on a Sig-Net network. See Section 5.7 for a full threat analysis of a compromised Manager.

4.4 Encryption

Sig-Net does not encrypt payload data. Data is transmitted as plaintext and is therefore readable by any party with access to the network (see Section 5.6).

CRA Consideration (Informative): Annex 1, Part I, Point 2(e) of the EU Cyber Resilience Act references the encryption of transmitted data. Sig-Net's position with respect to this point is that compliance is subject to the manufacturer's risk assessment as required by Article 13(2).

Furthermore, Recital (55) of Regulation (EU) 2024/2847 explicitly permits manufacturers to deviate from state-of-the-art security requirements when complying with widely recognised interoperability standards, or when a requirement is fundamentally incompatible with the nature of the product. Enforcing payload encryption severely limits the density of universes that baseline, non-hardware-accelerated Gateways can process at 44Hz, rendering it incompatible with the real-time constraints of the entertainment industry.

In the context of live entertainment control, neither lighting levels nor device management parameters originate as confidential information. The operational state of the system can be derived by a competent observer simply by looking at the physical output (the lighting rig) or reading the local user interface screens on the devices. Therefore, the primary security risk is not whether this operational data can be observed in transit, but whether an unauthorised party can alter the state of the rig.

Sig-Net addresses this risk entirely through cryptographic authentication and integrity (HMAC) rather than payload confidentiality (Encryption). While commands and levels are transmitted as plaintext, they cannot be forged, spoofed, or replayed without possession of the appropriate role-specific derived key (e.g. Ks or Km_local), which is held in protected non-volatile storage.

Furthermore, enforcing payload encryption severely limits the density of universes that baseline, non-hardware-accelerated Gateways can process at 44Hz. Given this reality, the most productive security investment is ensuring that management commands cannot be forged, spoofed, or replayed—which Sig-Net achieves cryptographically via Km_local—rather than mandating payload encryption that traffic analysis would in any case contextualise.

Physical network access is a prerequisite for any passive observation. In deployments where topology confidentiality is a specific identified risk — shared infrastructure venues, multi-tenant environments, or rental contexts where network access changes between events — physical network isolation or VLAN segmentation shall be treated as a mandatory compensating control rather than an optional recommendation.

The Security-Mode option (Section 8.3, Option 2076) is explicitly designed to provide a forward-compatible migration path should a manufacturer's risk assessment, or future regulatory guidance, conclude that payload encryption is required.

Manufacturers should document their assessment of whether the nature of entertainment control data (lighting levels, timecode) and their specific deployment environment justify the conclusion that HMAC-based integrity and authentication, without payload encryption, is proportionate to the identified risk. This formal risk assessment should be included in the product's technical documentation, citing the use of role-specific HMACs and physical network isolation as the primary controls for ensuring data integrity and preventing unauthorized access.

4.5 Authentication

Authentication is the process of proving that a device is a legitimate member of a trusted network role. Possession of a key derived from K0 serves as cryptographic proof that the sender is an authentic participant in the system.

4.6 Integrity

Integrity is the process of ensuring that a packet has not been tampered with in transit. This is achieved by sending each packet with an [HMAC] (Hash-based Message Authentication Code).

4.7 Authorisation

Authorisation is the process of determining if a device has the permission to perform an action. Sig-Net uses an Implicit Authorisation model; possession of a role-specific key (e.g. the Manager Key) provides the mathematical proof required to authorize a command.

4.8 Freshness

Freshness checking is the ability to ensure that received packets are fresh and not old copies. Sig-Net achieves this with a sequence number and session ID inside the [HMAC].

4.9 Security Lifecycle & Extensibility (Informative)

4.9.1 Firmware Updates

Sig-Net provides a secure, authenticated transport layer that may be used to carry firmware update payloads. This may be achieved using the standard [TBA] RDM extension encapsulated within the TID_RDM_COMMAND structure, or natively via dedicated Sig-Net TIDs (such as manufacturer-proprietary TIDs).

Firmware Routing via Endpoints:

When using [TBA] over Sig-Net, the intended target of the firmware update is deterministically defined by the Endpoint specified in the Command URI:

- Root Endpoint (0): A TID_RDM_COMMAND containing an [TBA] payload directed to Endpoint 0 shall be processed by the Sig-Net Node to update its own internal firmware.
- Data Endpoints (1 to N): A TID_RDM_COMMAND containing an [TBA] payload directed to a Data Endpoint shall be routed downstream to update the physical RDM responder attached to that specific port.
- Security Mandate (Sig-Net Devices Only): Because Sig-Net transmits application payloads as plaintext, manufacturers choosing to implement native firmware updates for the Sig-Net Device itself (via the Root Endpoint) SHALL ensure that the firmware image is cryptographically signed by the manufacturer prior to transmission, and that the receiving Sig-Net device verifies the signature before applying the update. Encryption of the firmware image is additionally recommended to prevent reverse engineering of proprietary code.

  Informative Note: This mandate cannot apply to firmware updates routed to Data Endpoints for downstream legacy RDM responders, as those devices operate outside the Sig-Net protocol boundary and receive standard unauthenticated serial RDM. Manufacturers of downstream RDM responders are independently responsible for the security of their own firmware update mechanisms.

4.9.2 Key Rotation

Because standard Sig-Net control traffic prioritises execution speed via plaintext payloads, a new Root Key (K0) cannot safely be transmitted over the primary network interface. In the event of a compromised Sig-Net Device, K0 must be rotated out-of-band (e.g. via local UI or USB) or via a dedicated, cryptographically authenticated encrypted tunnel extension.

Operators should rotate the Root Key (K0) whenever a production is repurposed or if the physical security of the network has been compromised.

4.9.3 Future Encryption:

The Sig-Net-Security-Mode option is specifically reserved to allow future minor-version updates to introduce AES-GCM payload encryption for management commands and over-the-wire key rotation, providing an upgrade path as embedded silicon capabilities advance.

5 Threat Vectors

5.1 Introduction

This section identifies some key threat vectors and discusses how Sig-Net does or does not handle them.

5.2 Man-in-the-Middle

A Man-in-the-Middle (MITM) attack involves an attacker intercepting a valid packet, modifying it and retransmitting. For example: changing a channel level from zero to full. Sig-Net protects against this using integrity checking. Each packet is sent with an [HMAC] which allows the responder to confirm that the packet has not been tampered with in transit.

5.3 Masquerading and Spoofing

Masquerading attacks involve an attacker pretending to be a legitimate device. For example, an attacker pretending to be a lighting console and sending bad data. As with MITM attacks, the [HMAC] protects against this.

5.4 Replay Attack

A replay attack involves an attacker recording part of the show and replaying it at another time, perhaps to force a blackout during the show. Sig-Net uses a sequence number and session ID inside the [HMAC] to reject old or duplicate numbers.

5.5 Rogue device

The threat is someone plugging a rogue device into the network. Sig-Net protects against this using authentication. Without the Root Key that authenticates a device, it cannot generate a valid [HMAC] and therefore cannot participate in communication.

5.6 Eavesdropping & Sniffing

Data is transmitted as plaintext and can therefore be read by any party with access to the network segment. Physical or VLAN-level network isolation is the recommended mitigation where confidentiality is required.

5.7 Compromised Node or Sender

Because Sig-Net uses a unique per-device management key (Km_local), the blast radius of a physically compromised Node is contained. If an attacker successfully compromises a legitimate Node (e.g. physically extracting the memory from a moving light), they only expose that specific Node's Km_local, the global Citizen Key (Kc), and the Sender Key (Ks). Because the Node deleted the Root Key (K0) immediately after onboarding, the attacker cannot mathematically derive the management keys for any other Node on the network. They cannot spoof the Manager to reconfigure the rig or alter the state of other devices.

5.8 Compromised Manager

The threat profile of a compromised Manager device is categorically different to a Node or Sender (Section 5.7) and should be understood by any operator deploying Sig-Net.

A Manager permanently retains the Root Key (K0). If an attacker successfully extracts K0 from a Manager, the attacker can derive every key in the system (Ks, Kc, Km_global, and any Km_local for any Node whose TUID is known). They can forge authenticated packets for any role, spoof any Sender or Manager command, and reconfigure or disrupt the entire system.

To mitigate this risk:

- Manager role devices shall use the highest available hardware security for K0 storage (see Section 14.1.2). A dedicated secure element is strongly recommended for Manager implementations.
- Operators shall treat Manager role devices with the same physical security discipline as other high-value credentials. Manager devices should not be left unattended and accessible in shared or publicly accessible spaces.
- In the event of a suspected Manager compromise, the Root Key shall be rotated immediately via out-of-band onboarding off all devices on the network (Section 4.9.2).

5.9 Offboarded Beacon Spoofing

Because offboarded device beacons (Security-Mode 0xFF) carry no valid HMAC by design, they cannot be authenticated. An attacker with access to the network can forge a beacon carrying an arbitrary TUID, causing a Sender to display a false "offboarded device present" notification or to incorrectly mark an operational device as lost or offboarded.

This is an inherent limitation of the offboarded beacon mechanism. It does not compromise the integrity of authenticated Sig-Net traffic — a forged beacon cannot cause an onboarded Node to change its behaviour or accept commands — but it may cause operational confusion at the management console level. See Section 10.2.1 for mitigation.

Physical or VLAN-level network isolation (Section 5.6) is the primary mitigation. Operators should treat unexpected offboarded beacon notifications during live operation as a potential indicator of network intrusion or misconfiguration.

6 System Architecture

6.1 Structure

A flat, peer to peer network architecture is used by Sig-Net.

6.2 Functional Roles

Sig-Net operates on a flat, peer-to-peer multicast architecture. To manage network authorisation without requiring a central server or hierarchical broker, functionality is divided into four logical roles. A single physical Sig-Net Device may implement any combination of these roles depending on its operational purpose.

- Manager Role: An administrative entity possessing the capability to perform network configuration, node discovery, and RDM transactions. Depending on the cryptographic keys provided to it during onboarding, a Manager operates in one of two states:
    - Equal Manager: The device possesses the Root Key (K0) and can dynamically derive all targeted keys (Km_local) to securely configure other endpoints. Multiple Managers sharing the same K0 operate as Equal Managers with full, redundant network authority (e.g. a Main and Backup Console).
    - Guest Manager: A restricted operational state where the device is provisioned only with the global derived keys (Km_global, Ks, and Kc), but lacks K0 and Km_local. Guest Managers can discover the network topology, monitor device status, and transmit real-

time data, but are cryptographically prohibited from altering device configurations or executing RDM commands (e.g. a touring console controlling a house rig).

- Sender Role: A control entity authorised to transmit real-time lighting levels and timecode. (e.g. the playback engine within a lighting console or an architectural wall plate).
- Node Role: A consumer entity that receives lighting control data, executes commands, and multicasts its state and replies. Devices acting in a Node role hold a unique, cryptographically derived management key (Km_local) to verify inbound configuration commands without needing to trust a central server.
- Visualiser Role: A passive consumer entity (e.g. a 3D rendering suite) that subscribes to lighting control data, preview streams, and patch directories. Visualisers do not output physical data or generate configuration commands, but they do participate in network discovery. They do not have data endpoints.

Informative note: A physical hardware device will frequently implement multiple roles simultaneously. Examples:

- A DMX512 Gateway that outputs Sig-Net as DMX512, while also receiving DMX512 to transmit onto the network, operates concurrently as both a Node and a Sender.
- A lighting console that streams DMX levels while also allowing the user to patch the rig operates concurrently as both a Sender and a Manager.

Informative Note (Manager-less Operation): While a Manager is required to onboard devices, configure IP addresses, and assign routing patches, a Sig-Net system can actively operate a live show without a Manager present on the network. If a system consists only of onboarded Senders and Nodes, the Senders will stream level data and the Nodes will output it. Unmanaged Nodes will continuously transmit <mult_node_lost> messages (as defined in Section 10.2.6), but this does not impede the real-time processing of lighting data or timecode.

6.3 Trust

The Root Key is used to define the trust structure of a Sig-Net network. Sig-Net uses a Human-in-the-Loop trust model; the operator is responsible for verifying the provenance and physical integrity of a device before supplying it with the Root Key (K0) via a local interface.

6.4 Air-Gap

Sig-Net is self-contained and is natively capable of operating in air-gapped (non-Internet connected) environments. It does not require external Time Servers (NTP), DNS, or external certificate validation to maintain cryptographic integrity.

6.5 SoemCode

A SoemCode is a 32-bit unsigned integer. To ensure global uniqueness without requiring a central registry, the upper 16 bits shall contain the manufacturer's officially assigned ESTA Manufacturer ID, and the lower 16 bits shall contain a specific product variant ID managed independently by the manufacturer. SoemCode 0x00000000 is reserved and shall not be used.

Manufacturers that do not hold an ESTA Manufacturer ID shall obtain one prior to implementing Sig-Net. ESTA Manufacturer IDs are available to any organisation manufacturing relevant products and are assigned by the ESTA Technical Standards Manager. Implementing Sig-Net without a valid ESTA Manufacturer ID will result in SoemCode collisions with other manufacturers and is not permitted.

For devices that also support RDM (E1.20), the 16-bit Product Variant ID should match the device's RDM Device Model ID to ensure consistent identification across both protocol layers.

6.6 Transport Unique Identifier (TUID)

The TUID is a 48-bit value consisting of a 16-bit Manufacturer ID (assigned by ESTA) in the upper two bytes, and a 32-bit Device ID (managed independently by the manufacturer) in the lower four bytes.

To prevent identity collisions between physical hardware devices and software applications (e.g. PC-based consoles or conformance tools) operating on the same network, the 32-bit Device ID shall be partitioned using its Most Significant Bit (MSB):

- 0x00000000 to 0x7FFFFFFF (MSB = 0): Static Assignment. Used for physical hardware devices. This ID must be uniquely assigned by the manufacturer during production and stored in non-volatile memory.
- 0x80000000 to 0xFFFFFFEF (MSB = 1): Dynamic Assignment. Used for software applications that lack a fixed hardware identity. The application shall generate a random Device ID within this range using a [CSPRNG]. Permanent installations should save this dynamically generated TUID to non-volatile storage during initial installation and reuse it persistently. Truly transient software (such as diagnostic apps) may generate a new Dynamic TUID upon each execution.
- 0xFFFFFFF0 to 0xFFFFFFFF: Reserved for future administrative or diagnostic use.

Informative Note on Dynamic TUID Generation: To ensure statistical uniqueness when multiple transient software instances operate on the same host machine sharing a single MAC address, applications generating Dynamic TUIDs should seed their [CSPRNG] using a combination of the host MAC address, a sub-millisecond timestamp, and the application's unique Process ID. The application takes the bottom 31 bits of the CSPRNG output and sets the MSB to 1.

Informative Note on Dynamic TUID Collisions: In specialist applications where a single host dynamically generates large numbers of virtual nodes, the originator application is responsible for internal collision avoidance. Originators should monitor the scope's TID_POLL_REPLY communication to verify their randomly generated 31-bit Device ID is unique on the subnet prior to initiating authenticated network communication.

Informative Note on Human Readable Format: The recommended method for representing the TUID in text is a hexadecimal format with a colon separating the Manufacturer ID and the Device ID, identical to the convention defined in [RDM].

Example format: mmmm:dddddddd (where mmmm is the 16-bit ESTA Manufacturer ID and dddddddd is the 32-bit Device ID). Devices should use uppercase characters (A-F) for all TUID.

6.7 TUID and E1.20 UID Relationship

A Sig-Net Device that is simultaneously a native IP-connected RDM Responder possesses both a TUID (identifying it as a Sig-Net network entity) and an E1.20 UID (identifying it as an RDM device, as defined in [RDM]). These two identifiers exist in separate protocol namespaces and serve distinct purposes.

Such a device shall assign the same 48-bit value to both its TUID and its E1.20 UID.

6.8 Endpoints

The logical and physical interfaces of a Sig-Net Node are addressed using a 16-bit integer known as an Endpoint.

6.8.1 Root Endpoint

Endpoint 0 is known as the Root Endpoint and is reserved as the administrative interface for the entire physical product. It is used to report and set global device parameters (e.g. TID_RT_DEVICE_LABEL or IP network settings). A Sig-Net Node shall implement one Root Endpoint.

Administrative RDM: The Root Endpoint shall not consume or proxy standard DMX or fixture-level RDM configuration (such as DMX_START_ADDRESS). It shall only process encapsulated TID_RDM_COMMAND payloads related to the physical device's global administration (such as [TBA] Firmware Updates for the native Sig-Net network processor).

6.8.2 Data Endpoints

Endpoints in the range 0x0001 to 0xFF00 represent logical or physical ports that consume, route, or generate data. Data Endpoints shall be numbered consecutively. Sig-Net Nodes are not required to implement Data Endpoints.

- Physical Endpoints: A Data Endpoint that maps to a physical hardware port (e.g. a 5-pin DMX socket on a Gateway). It outputs TID_LEVEL data as physical DMX and acts as a transparent proxy for TID_RDM_COMMAND payloads, forwarding them to downstream responders.
- Virtual Endpoints: A Data Endpoint that maps to an internal logical block (e.g. the motor control engine of a native IP moving light). It consumes TID_LEVEL data internally. To allow Managers to configure standard fixture properties (such as DMX_START_ADDRESS or DEVICE_MODE), the Node shall expose an internal Virtual RDM Responder on this Endpoint. The Node terminates and processes TID_RDM_COMMAND payloads directed here as if it were a standalone physical fixture.

Linked Hardware Capabilities: If a Node's physical hardware design restricts a specific Data Endpoint parameter (such as TID_EP_FAILOVER or TID_EP_DMX_TIMING) to a global state across multiple ports, the Node may apply a Parameter Update intended for a single endpoint to all its linked endpoints simultaneously.

If a Node applies a configuration change to linked endpoints as a side-effect of a Parameter Update, the Node shall proactively multicast the updated parameter state for every affected endpoint to its Reply URI (as defined in Section 10.4.2). This guarantees that all active Managers are informed of the hardware's linked behaviour and can maintain accurate interface synchronisation.

6.8.3 Reserved Endpoints

Endpoints in the range 0xFF01 to 0xFFFE are reserved for future use and shall not be used.

6.8.4 Broadcast Endpoint

The Broadcast Endpoint 65535 is used exclusively in command URIs to target an instruction at all valid Data Endpoints (1 to N) on a Node simultaneously. Commands addressed to 65535 shall not affect the Root Endpoint.

6.8.5 Endpoint Examples

- A moving light that consumes a single DMX universe has an endpoint count of 1. Endpoint 0 sets the fixture name; Endpoint 1 sets the Universe the fixture listens to.
- A DMX Gateway with 10 physical outputs has an endpoint count of 10. Endpoint 0 sets the gateway name; Endpoints 1 through 10 route configuration and RDM to each respective physical serial port.

7 Root Key and Key Derivation

7.1 Introduction

Sig-Net uses a 256-bit Root Key (K0) to cryptographically derive four role-specific keys: the Global Manager Key (Km_global), the Node-Specific Manager Key (Km_local), the Sender Key (Ks), and the Citizen Key (Kc). Kc serves as the 'Citizen Key' used by all Sig-Net devices to announce their presence and identity.

7.2 Root Key (K0) Specification

The Root Key (K0) is the sole cryptographic root of trust. K0 shall be 32 bytes in length.

To comply with CRA secure-by-default requirements, devices shall not be shipped with a pre-configured, hardcoded, or default K0 (e.g. a string of zeros). A Sig-Net Device shall refuse to process or transmit secure network control traffic until a cryptographically strong K0 is established. See Section 7.2.1.

Establishing K0:

To ensure K0 is established securely and to prevent typographical errors that would compromise network interoperability, devices shall only permit K0 to be established via the following specific methods:

- On a Manager: A Manager shall establish K0 via one of three methods:
    - Internal Generation (Recommended): The Manager generates a high-entropy 256-bit K0 (or a compliant passphrase) using its internal [CSPRNG].
    - Passphrase Entry: The operator manually types a compliant passphrase via the device's local user interface.
    - Electronic Transfer: The 64-character K0 hexadecimal string is loaded out-of-band via a machine-readable interface (e.g. USB or NFC) as defined in Section 7.5.
- On a Node or Sender: A Node or Sender shall establish K0 via one of two methods:
    - Passphrase Entry: The operator manually types the passphrase via the device's local user interface.
    - Electronic Transfer: The 64-character K0 hexadecimal string is loaded out-of-band via a machine-readable interface (e.g. USB, NFC or SNOW) as defined in Section 7.5.

Manual Hexadecimal Entry Prohibition: Under no circumstances shall any Sig-Net Device (Manager, Sender, or Node) provide a user interface mechanism that allows an operator to manually type the 64-character hexadecimal K0 string. All manual human entry must utilize the Passphrase method.

Cleartext Network Onboarding Prohibition: To prevent key interception, K0 onboarding shall not be accepted over the standard Sig-Net UDP transport. An offboarded device shall silently discard any standard Sig-Net network-delivered onboarding attempt. Over-The-Wire (OTW) onboarding via an IP network is only permitted if the device implements a dedicated, cryptographically authenticated and encrypted onboarding extension (e.g. via a distinct TLS/DTLS tunnel) explicitly designed for secure key exchange.

7.2.1 Entropy Validation

Managers shall implement algorithmic entropy validation when automatically generating K0 using their internal [CSPRNG].

A Manager shall reject any internally generated 64-character hexadecimal K0 string that possesses a character-level [Shannon] entropy of less than 3.0 bits per character. This algorithmic check natively prevents the use of repeated characters, simple repeating patterns, and highly predictable sequences.

(Note: Entropy validation for human-entered passphrases is handled independently via the strict complexity rules defined in Section 7.2.3).

7.2.2 Offboarded Operation (Beacon Mode)

A device that lacks a Root Key (K0) is referred to as ‘offboarded’. An offboarded Sig-Net Device lacks the derived cryptographic key material (Km_local, Ks, Kc) required to generate or verify an HMAC. Therefore, it cannot participate in authenticated network communication and shall silently discard all incoming authenticated Sig-Net packets.

To alert Managers that it requires onboarding, the device shall operate exclusively in "Beacon Mode," transmitting a periodic, unauthenticated presence beacon (using Security-Mode 0xFF). The complete URI syntax, timing rules, and payload structure for Beacon Mode are defined in Section 10.2.1.

7.2.3 Input and Transfer Formats

A Sig-Net Device shall support the establishment of the Root Key (K0) through one of the following two formats, determined by the device's physical interface:

- Passphrase Mapping (Human Interface): To support manual entry via a local user interface (e.g. a keypad or 4-button display), a device shall accept a UTF-8 Passphrase, subject to the character limits defined below. The device shall derive K0 from this passphrase using the PBKDF2-HMAC-SHA256 algorithm with 100,000 iterations and the fixed salt 'Sig-Net-K0-Salt-v1' (18 bytes, ASCII).

  When accepting a passphrase for K0 derivation, a Sig-Net Device shall enforce the following minimum requirements:

  - Minimum Length: 10 characters .
  - Maximum Length: 64 characters .
  - Character Class Diversity: The passphrase shall contain characters from at least three of the following four classes:
  - Uppercase letters (A-Z)
  - Lowercase letters (a-z)
  - Digits (0-9)
  - Symbols (!@#$%^&*()-_=+[]{}|;:',.<>?/)

  The passphrase shall not contain:

  - More than 2 consecutive identical characters (e.g. 'aaa' rejected)
  - More than 3 consecutive sequential characters (e.g. 'abcd' or '1234' rejected)

  User Feedback: If a passphrase fails validation, the device shall display a specific error message indicating which requirement was not met, rather than a generic 'weak passphrase' message.

  Enforcement Roles: A Manager shall enforce all of the above complexity rules and length limits when a user inputs a passphrase or when the Manager generates one. A Manager shall refuse to initialise the network if the passphrase fails any rule.

  Nodes or Senders should also validate the passphrase locally at the user interface before deriving keys.

  Informative Note on Node UX: If a Node does not validate the input and a user makes a typographical error, the Node will derive an incorrect K0 and cease transmitting Offboarded Beacons. Because it possesses the wrong keys, it will silently drop all valid discovery polls from the Manager due to HMAC verification failures. While advanced Managers may detect and flag the Node's subsequent node_lost broadcasts as cryptographic anomalies, the Node will fundamentally fail to authenticate and participate in the network, requiring a physical out-of-band factory reset to correct. Local validation prevents this operational failure.

  Validation Consistency and Reference Implementation: Singularity (UK) Ltd publishes a normative Reference Implementation of the passphrase validation algorithm (written in C) within the official Sig-Net GitHub repository. Implementers should integrate this reference routine directly and if a manufacture chooses to implement their own they shall ensure it behaves identically to the reference implementation.

Manager Generated Passphrases: A Manager should provide a user interface facility to automatically generate a cryptographically secure, random 10-character passphrase that adheres to the complexity rules above, using a [CSPRNG].

Informative Note: These requirements are designed to ensure a minimum entropy of approximately 50 bits, providing adequate resistance to offline brute-force attacks even with the fixed PBKDF2 configuration.

- Direct Hexadecimal (Machine Interface): For electronic transfer via an out-of-band interface (e.g. USB, NFC, or OTW / SNOW), K0 shall be accepted as a 64-character hexadecimal string or a 32-byte binary block.

Once K0 is established (either via direct hexadecimal entry or passphrase mapping), the Device shall immediately proceed to the Key Derivation process (Section 7.3). Nodes, Senders and Visualisers shall then permanently discard the source entry material and K0 from volatile memory as mandated in Section 7.6).

Interoperability Mandate: To ensure a consistent Root Key across the network, all Sig-Net devices utilizing the Passphrase Mapping method shall employ this specific PBKDF2 configuration. A Device shall not use a simple SHA-256 hash for K0 derivation from a passphrase.

Informative Note on Fixed Salt and Domain Separation:

Cryptographic best practice typically requires a unique, per-installation salt to prevent pre-computation (Rainbow Table) attacks. However, requiring an operator to manually type a high-entropy, 16-byte random salt alongside a passphrase into constrained edge-device user interfaces (e.g. a 4-button LCD menu) is operationally unfeasible and actively hostile to the user experience in live entertainment environments.

To resolve this, Sig-Net uses a fixed global salt ("Sig-Net-K0-Salt-v1"). This acts as a cryptographic Domain Separator. It invalidates all existing, pre-computed rainbow tables derived from standard IT data breaches or common dictionaries, forcing an attacker to compute hashes specifically for the Sig-Net ecosystem. The defence against offline attacks therefore relies entirely on the high Work Factor (100,000 PBKDF2 iterations) combined with strict passphrase complexity and the strong recommendation for Manager-generated random passphrases.

7.2.4 First-Packet Bootstrapping

Upon booting and loading its derived keys into memory, Nodes track Sender Session IDs and Sequence Numbers in volatile memory to prevent non-volatile memory exhaustion (see Section 8.6, Step 9).

When a Node powers on, its internal Sender tracking table is empty. When a Node receives its first packet from a previously unknown Sender TUID (i.e. no Session ID record currently exists in RAM for that Sender), it shall treat the received Session ID and Sequence Number as the initial baseline for that Sender and accept the packet unconditionally, provided all cryptographic HMAC checks pass. Subsequent packets from that Sender shall be subject to strict forward-only enforcement from that new baseline.

Security Note: Because Nodes reset their tracking tables upon power loss, an attacker who recorded a previous session could theoretically replay it immediately after a Node reboots, before the legitimate originator transmits a new packet. However, because a legitimate Sender generates TID_LEVEL keep-alive packets at 1 Hz, and a legitimate Manager generates TID_POLL requests every <poll_time> seconds, the window for a successful replay attack is bounded to a few seconds. This represents a trivial increase in threat surface in exchange for guaranteed hardware non-volatile memory longevity

7.2.5 Open Mode (Unauthenticated Operation)

A Sig-Net Device may operate in an unauthenticated "Open Mode". This is designed to support temporary trade show environments and educational use where the risk assessment concludes that cryptographic security is unnecessary.

- Explicit Opt-In: To comply with secure-by-default requirements, devices shall not be shipped with Open Mode enabled. A device shall only enter Open Mode via a deliberate, physical interaction with the device's local user interface (e.g. a menu selection or physical button press) or via an authenticated out-of-band channel. Open Mode shall not be enabled via standard Sig-Net UDP network commands.
- Key Bypass: When placed in Open Mode, the device does not require a Root Key (K0). The device skips the Key Derivation Function (Section 7.3) and ceases transmitting Offboarded Beacons (Section 10.2.1).
- UI Indication: A device operating in Open Mode should clearly indicate this unsecure state on its local physical display (if equipped).
- Compliance Mandate: A Sig-Net Device shall not be manufactured to operate exclusively in Open Mode. To claim compliance with this specification, all devices shall fully implement the cryptographic framework (Secure Mode) and the K0 onboarding lifecycle. Open Mode shall solely be provided as a temporary bypass mechanism for low-risk environments, not as a substitute for the mandatory security stack.

7.3 Key Derivation Function (KDF)

Role-specific keys shall be derived from K0 using the HKDF-Expand function defined in [HKDF] RFC 5869 Section 2.3, using HMAC-SHA256 as the underlying hash function.

K0 serves directly as the Pseudorandom Key (PRK) input. The output length (L) for all derived keys shall be 32 bytes.

Devices shall independently generate their required keys by passing specific ASCII info strings into the HKDF-Expand function:

Global Keys:

- Sender Key (Ks): HKDF-Expand(K0, "Sig-Net-Sender-v1", 32)
- Citizen Key (Kc): HKDF-Expand(K0, "Sig-Net-Citizen-v1", 32)
- Global Manager Key (Km_global): HKDF-Expand(K0, "Sig-Net-Manager-v1", 32)

Km_global is used exclusively by Managers to authenticate network-wide discovery multicasts to the TID_POLL URI.

Per-Device Management Key (Km_local):

Targeted management is cryptographically bound to each individual Node's Transport Unique Identifier (TUID). This ensures that a physically compromised Node cannot be used to spoof configuration commands to other devices.

When a Node is onboarded with K0, it shall derive its own unique management key by appending its 12-character uppercase hexadecimal TUID to the derivation string:

- Node-Specific Manager Key (Km_local): HKDF-Expand(K0, "Sig-Net-Manager-v1-" || TUID, 32)

(Example info string for TUID 123456789ABC: "Sig-Net-Manager-v1-123456789ABC")

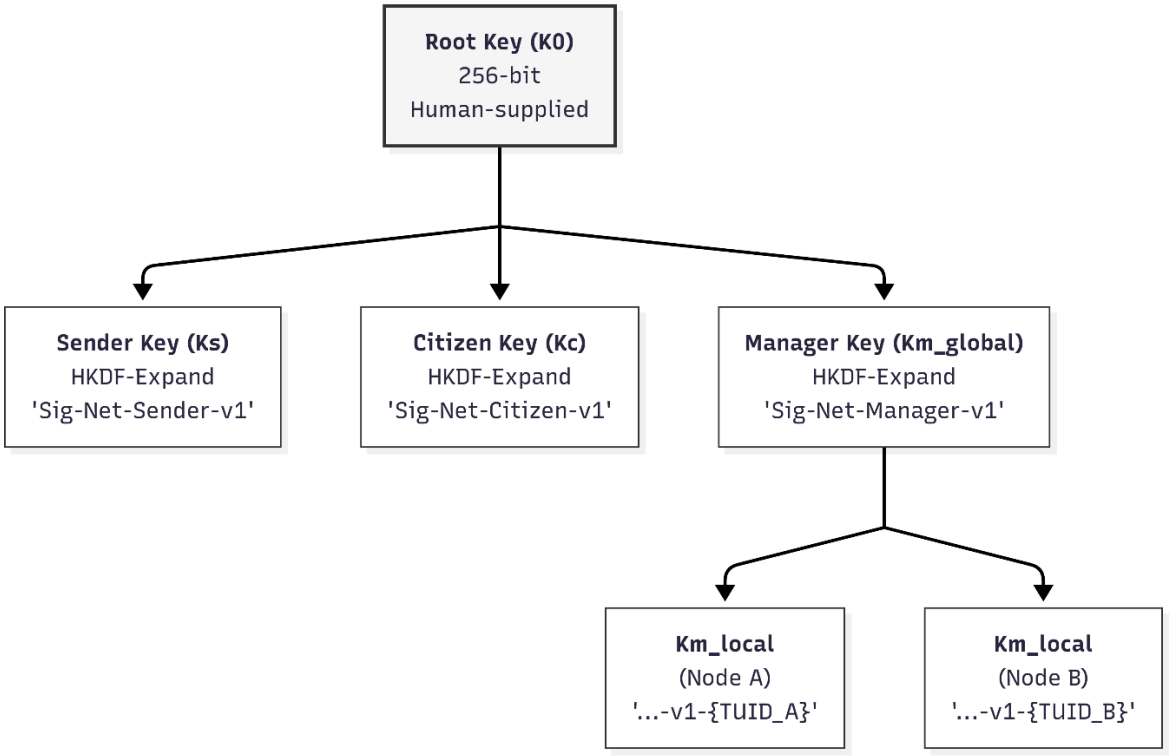

Figure 1 Summary of Key Derivation

Informative Note on Key Independence: Although the Km_global and Km_local derivation strings share a common prefix ("Sig-Net-Manager-v1"), the HKDF-Expand outputs are computationally independent. Knowledge of Km_global does not assist in computing any Km_local value, and vice versa, because HKDF-Expand is a Pseudorandom Function (PRF): without K0, the output for any info string is computationally indistinguishable from a random value. This independence holds provided K0 has sufficient entropy as required by Section 7.2.

Informative Note on Versioning: The -v1 version suffix ensures that derived keys from future protocol versions are cryptographically distinct, preventing cross-version key reuse. HKDF-Expand for a 32-byte output simplifies internally to a single HMAC calculation: HMAC-SHA256(K0, info || 0x01). Any implementation possessing HMAC-SHA256 can implement this without additional cryptographic libraries.

Figure 2 Summary of Derived Key Usage

```
+-----------+                                      +-----------+
|           |========= Km_global (Multicast) =======>           |
|           |  URI: /sig-net/<ver>/poll            |           |
|  Manager  |                                      |           |
|   Role    |---------- Km_local (Unicast) --------->           |
|           |  URI: /sig-net/<ver>/manager/{tuid}  |           |
|           |                                      |           |
|           |<========== Kc (Multicast) ============|   Node    |
|           |  URI: /sig-net/<ver>/node/{tuid}     |   Role    |
+-----------+       /sig-net/<ver>/node_lost/{tuid} |           |
                                                   |           |
+-----------+                                      |           |
|           |                                      |           |
|  Sender   |============= Ks (Multicast) ==========>           |
|   Role    |  URI: /sig-net/<ver>/level/{universe} |           |
|           |       /sig-net/<ver>/sync            |           |
```

```
+-----------+       /sig-net/<ver>/timecode/{stream}|           |
                                                   +-----------+
```

7.3.1 Key Lifecycle and Erasure

Nodes and Senders shall adhere to a strict "Calculate-then-Discard" lifecycle. This is critical to limiting the Blast Radius of a physically compromised device.

- Derivation: Upon onboarding, the device shall use K0 to derive the required role-specific keys (Ks, Kc, Km_global, and Km_local).
- Storage: The derived role-specific keys shall be committed to hardware-secure non-volatile storage.
- Erasure: Immediately following successful derivation, the device shall permanently erase the UTF-8 passphrase (if used) and the Root Key (K0) from its internal RAM and any temporary buffers.
- Operation: During live network operation, the device shall use only the derived role-specific keys for HMAC calculations. The device shall not retain K0 in memory after the initial onboarding sequence is complete.

7.4 Human Readable Format (Display and Verification)

While manual keyboard entry of a 64-character hexadecimal Root Key (K0) is prohibited to prevent typographical errors (see Section 7.2.3), devices acting in a Manager role may need to display the raw 256-bit K0 on a screen for visual verification, cryptographic auditing, or out-of-band export.

When visually presented to an operator, the 256-bit K0 shall be represented as a 64-character hexadecimal string. To ensure visual clarity and readability, devices shall standardise to lowercase characters (a-f) for all visual displays.

Furthermore, when a device parses a 64-character hexadecimal string supplied via a machine-readable out-of-band method (such as a JSON file on a USB drive), the parsing logic shall be case-insensitive.

7.5 Machine Readable Format (Out-of-Band Transfer)

When transferring K0 electronically between systems (e.g. via USB flash drive, NFC, or a mutually authenticated secure channel such as BLE or a dedicated TLS/DTLS onboarding extension), it shall be encoded as a simple JSON object:

```
{ "sig-net_k0": "E4D909C290D0FB1CA068FFADDF22CBD0..." }
```

Informative Note: The JSON key name "sig-net_k0" uses lowercase characters. Implementations shall use this key name. The value must be 64 uppercase hexadecimal characters representing the full 256-bit Root Key.

7.5.1 Restricted Key Export (Guest Access)

To facilitate the secure handover of a lighting network to a guest controller without compromising the administrative control of the rig, an Equal Manager may export a subset of the derived network keys to a JSON file. This file shall not contain K0 or any Km_local keys. It shall only contain the global keys required to operate as a Guest Manager.

It shall be encoded as a simple JSON object:

```
{
  "sig-net_guest_keys":
      {
      "scope": "local",
```

```
            "km_global": "A1B2C3D4...",
            "ks": "E5F6G7H8...",
            "kc": "I9J0K1L2..."
            }
        }
```

The derived key values shall be encoded as 64-character uppercase hexadecimal strings, matching the format required for out-of-band K0 transfers.

A Manager application loaded with these restricted keys will successfully authenticate TID_POLL and TID_LEVEL traffic on the designated scope, but will fail HMAC verification if it attempts to transmit configuration Parameter Updates, enforcing a Read-Only administrative state.

7.6 Storage and Security

Devices must limit which derived keys they retain after onboarding:

- Managers: A Manager requires permanent access to K0 to derive management keys for the network. Because the PBKDF2 derivation is mathematically irreversible, a Manager shall securely store the plaintext passphrase (or the direct 64-character hex K0 if onboarded out-of-band) in its hardware-secure enclave. This ensures an authorized operator can recall and view the passphrase via the Manager's UI when onboarding new Nodes added to the system.
    - Exception: A Manager may optionally offer a high-security "Stateless" mode where K0 is derived at runtime and held exclusively in volatile memory (RAM). In Stateless mode, the user accepts that they must re-enter the passphrase upon every power cycle and must manually manage their own secure record of the passphrase.
- Nodes: Shall derive and store Kn, Ks, Km_global, and their own unique Km_local. They shall immediately and permanently discard K0 and any source passphrase after successful derivation.
- Senders: Shall derive and store Ks and Kn. They shall immediately and permanently discard K0 and any source passphrase after successful derivation.
- Visualisers: Shall derive (or receive via SNOW) and store Ks and Kc. They shall immediately and permanently discard K0 and any source passphrase after successful derivation.

All cryptographic keys and passphrases retained by a device shall be protected by a secure storage mechanism that preserves their confidentiality and integrity at rest.

- Where the device hardware provides a protected cryptographic peripheral or secure element capable of performing the required operations, key material should be held within that subsystem and shall not be exposed to the general application processor.

- Where baseline hardware is used and keys must be loaded into application-accessible memory (RAM) for cryptographic computations, they shall be present in plaintext only for the minimum duration necessary to complete the operation and shall be securely zeroized immediately afterwards. Keys shall never be written to non-volatile storage (Flash/EEPROM) in plaintext, unless the manufacturer explicitly relies on the legacy hardware physical-security provisions detailed in Section 14.1.2.

Key Extraction Prohibition: Under no circumstances shall a Sig-Net Device provide a mechanism to export or extract derived cryptographic keys (Km_local, Km_global, Ks, Kc) over the network or via local interfaces. If a device provides a "Configuration Backup" or "Cloning" feature (e.g. via a web interface or USB file export), all derived cryptographic keys shall be excluded from the exported data. Keys must remain bound to the physical hardware that derived them.

To maintain cryptographic integrity during out-of-band transfers (e.g. loading K0 via a USB flash drive or NFC token), manufacturers shall mandate strict physical hygiene. Operators must ensure that physical access to the device UI is restricted and that transfer media (such as USB drives containing K0.json) are securely wiped immediately after onboarding is complete.

7.7 Offboarding and Erasure

To comply with security lifecycle requirements, support key rotation, and allow hardware to be safely repurposed, Sig-Net Devices must support a secure return to an offboarded state. Returning to an offboarded state mandates the permanent deletion of the Root Key (K0) and all derived role keys, and the reset of the Sig-Net-Session-ID to zero.

Informative Note: Because the derived keys are permanently destroyed upon offboarding, the device's cryptographic epoch is ended. Therefore, resetting the Session ID to zero is cryptographically safe; any previously recorded packets from prior sessions will automatically fail HMAC authentication against any future onboarding keys.

7.7.1 Authenticated Network Offboarding

A Manager in possession of the current valid cryptographic keys may offboard a Node over the network by unicasting the TID_RT_OFFBOARD command to the Node's Root Endpoint.

Operational Safety Lockout: To mitigate the risk of accidental erasure or malicious insider sabotage during live operation, Nodes shall only accept the TID_RT_OFFBOARD command within the first <offboard_lockout> seconds after the Node physically powers on.

If the authenticated TID_RT_OFFBOARD command is received after this lockout period has expired, the Node shall silently discard the command. Therefore, to execute a secure network wipe, an operator must intentionally power-cycle the target Node(s) immediately prior to transmitting the command.

Informative Note: To maintain strict physical security, a software-initiated warm reboot (e.g., via TID_RT_REBOOT) does not satisfy the requirement of a physical power cycle and shall not reset the <offboard_lockout> timer

7.7.2 Physical Out-of-Band Offboarding

To prevent a device from becoming permanently inoperable if the network key is lost, manufacturers shall implement a physical, out-of-band mechanism to force a return to the offboarded state.

This fallback mechanism shall not be accessible via the Sig-Net network port to prevent unauthenticated network attacks. It shall require either physical human proximity or an authenticated out-of-band channel that is independent of the Sig-Net network. Acceptable mechanisms include a local user interface menu, an NFC interaction, a recessed hardware button, a proprietary serial wipe command received via the physical DMX512/RDM connector (facilitating external hardware dongles), or a command received over a mutually authenticated out-of-band connection (such as a BLE channel established by a paired management application).

8 Packet Structure

8.1 CoAP Summary (Informative)

Sig-Net uses the Constrained Application Protocol [CoAP] operating over UDP. A standard CoAP message consists of a compact 4-byte header, followed by a token, zero or more Options (which function like HTTP headers), a payload marker (0xFF), and the payload itself.

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
```

```
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Ver| T |  TKL  |      Code     |          Message ID           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Token (if any, length is TKL) ...                           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Options (if any) ...                                        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|1 1 1 1 1 1 1 1|    Payload (if any) ...                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

8.2 URI Syntax Overview

The destination of the data is defined using the CoAP Uri-Path option. All Sig-Net URI-Paths (Option 11) shall begin with the following prefix:

/sig-net/<version>/<scope>

- Where <version> is the protocol version (e.g. v1).
- Where <scope> is a 1 to 32-character ASCII string containing only URL-safe unreserved characters (as defined in [URI] Section 2.3). The scope isolates discrete control networks sharing the same physical IT infrastructure (e.g. separating "Hall_A" from "Hall_B"). The factory configuration for all devices shall be “local”.

The remainder of the URI-Path defines the specific resource (e.g. universes, RDM, management) and is fully defined in Section 10.

Where a URI-Path component contains a {tuid}, it shall be represented as a 12-character uppercase hexadecimal string with no separators (e.g. 123456789ABC). This canonical form shall be used in all URI-Path options and in the HMAC calculation input defined in Section 8.5.

Where a URI-Path component contains a numerical index—specifically {universe}, {endpoint}, or {stream}—it shall be represented as a Base-10 decimal integer string. To guarantee deterministic HMAC calculations across all implementations, these decimal strings shall not contain leading zeros (e.g. Endpoint 5 shall be represented as /5, not /05 or /005).

Note that the hyphen in “sig-net” is character 0x2d.

The major version number of this specification document (e.g, 1.x) shall correspond directly to the <version> string used in the CoAP URI-Path (e.g. /v1/). Sub-point revisions (e.g. 1.1) represent non-breaking changes (such as new TIDs or clarified definitions) and do not trigger a change in the URI-Path.

8.3 Security Options

To maintain a clean and standardised payload, Sig-Net does not embed security metadata inside the lighting data payload. Instead, Sig-Net uses custom CoAP Option fields to carry the authentication and freshness data.

Implementations shall include the following five options in every Sig-Net packet. As mandated by [CoAP] Section 3.1, these options must appear in order of their Option Numbers on the wire (using standard CoAP extended delta encoding). Note: The Option numbers below are selected from the CoAP private use range to be Elective, Safe-to-Forward, and NoCacheKey, and to guarantee the correct sequential parsing order required for the HMAC calculation.

- Sig-Net-Security-Mode (Option Number: 2076): A 1-byte unsigned integer defining the cryptographic mode. The following values are defined:
    - 0x00: Signifying Plaintext Payload with HMAC-SHA256.

- 0x01: Signifying Open Mode (Unauthenticated). Packets carrying this value shall set the Sig-Net-HMAC option value to 32 zero bytes (0x00).
- 0xFF: Indicate an Offboarded Device Beacon. Packets carrying this value shall set the Sig-Net-HMAC option value to 32 zero bytes (0x00). Processing is as described in Section 7.2.

- Sig-Net-Sender-ID (Option Number: 2108): An 8-byte string uniquely identifying the specific origin of the packet. The first 6 bytes shall contain the 48-bit Transport Unique Identifier (TUID) of the device transmitting the packet. The final 2 bytes shall contain the 16-bit Endpoint number originating the payload (or 0x0000 for Root/Manager commands).
- Sig-Net-Mfg-Code (Option Number: 2140): A 2-byte unsigned integer representing the sender's ESTA Manufacturer ID. For all standard Sig-Net control messages, this value shall be 0x0000. A non-zero value explicitly flags the Application Payload as manufacturer-specific proprietary data.
- Sig-Net-Session-ID (Option Number: 2172): A 4-byte unsigned integer serving as an increasing session identifier to prevent whole-session replay attacks. The value is a global property of the Sig-Net Device and is bound to its TUID.

  During startup or initialisation, before transmitting its first authenticated Sig-Net packet, every device (Manager, Sender, Node and Visualiser) must increment this Session ID counter by 1 and successfully save the new value to non-volatile storage. This ensures the Session ID increases with each boot cycle or initialisation. Using a random value is not permitted, since random values cannot guarantee that the counter will always be higher than its previous value after a power cycle. The Session ID shall also be incremented by 1 during live operation if the Sig-Net-Seq-Num wraps (see below).

  Devices operating exclusively in Open Mode (0x01) or Beacon Mode (0xFF) are exempt from Session ID and Sequence Number tracking and shall transmit 0x00000000 for these option values.

  - Session ID Overflow: In the event that the Session ID reaches 0xFFFFFFFF, the device shall permanently cease transmitting authenticated Sig-Net packets and shall enter an error state. Normal operation can only be restored by executing a full offboarding and rekeying cycle (as defined in Section 7.7), which securely resets the Session ID to zero under a new cryptographic epoch.
  - Informative Note on Session Exhaustion: While 0xFFFFFFFF represents over 4 billion sessions (a number unlikely to be reached in the physical lifespan of most hardware), poorly written firmware could artificially accelerate this counter. Managers should monitor the reported Session IDs of discovered Nodes and proactively warn the operator if a device's Session ID approaches exhaustion (e.g., crossing 0xFFFF0000). This allows the operator to schedule a preventative rekeying cycle outside of live show conditions.

- Sig-Net-Seq-Num (Option Number: 2204): A 4-byte unsigned integer. An increasing counter that increments by 1 for every packet sent within a specific Sender-ID data stream. When the counter reaches 0xFFFFFFFF, the Sender shall increment its non-volatile Session ID by 1 and reset the Sequence Number to 1, effectively initiating a new session.

  Devices operating exclusively in Open Mode (0x01) or Beacon Mode (0xFF) are exempt from Session ID and Sequence Number tracking and shall transmit 0x00000000 for these option values.

    - Informative Note: Senders shall maintain independent sequence counters for each of their Endpoints. This ensures that network jitter or packet loss on one universe stream does not cause anti-replay lockouts on other streams originating from the same physical device.
- Sig-Net-Auth (Option Number: 2236): A variable-length byte string containing the cryptographic signature or authentication tag for the packet. The length and generation algorithm of this option are defined by the Sig-Net-Security-Mode:
    - Mode 0x00 this shall be a 32-byte HMAC-SHA256 signature.
    - Mode 0x01 this shall be length 0.
    - Mode 0xff this shall be length 0.

  This option shall be the highest-numbered option in the packet so that it appears last before the payload marker.

Informative Note: Whilst standard CoAP libraries may automatically insert Token bytes (TKL > 0), Sig-Net does not use them for state tracking. CoAP option encoding uses a delta scheme; the large numerical gap between the standard Uri-Path option (Option 11) and the first Sig-Net custom option (Option 2076) will result in the 2-byte extended delta encoding prefix being used, as defined in RFC 7252 Section 3.1. Implementations must handle this extended delta correctly.

8.4 Packet Structure

The resulting Sig-Net packet encapsulates the CoAP standard, integrating the target URI, the security option, and the TLV application payload.

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|Ver| T |  TKL  |      Code     |          Message ID           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Token (if any, length in TKL)                               |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Uri-Path Options (eg "sig-net", "v1", "local", "level", "1")|
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Sig-Net-Security-Mode Option (1 Byte)                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Sig-Net-Sender-ID Option (8 Bytes)                          |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Sig-Net-Mfg-Code Option (2 Bytes)                           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Sig-Net-Session-ID Option (4 Bytes)                         |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Sig-Net-Seq-Num Option (4 Bytes)                            |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Sig-Net-HMAC Option (32 Bytes)                              |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   0xFF (Payload Marker)                                       |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Application Payload (Defined in Section 10)                 |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

If the Application Payload is empty (zero bytes), the Payload Marker (0xFF) shall be omitted from the packet, as required by [CoAP] Section 3.

8.5 HMAC Calculations

The HMAC is calculated over a deterministic concatenation of the critical packet values. This guarantees Integrity and Authenticity without requiring complex CoAP byte-stream reconstruction.

The Sender shall calculate the [HMAC] using the HMAC-SHA256 algorithm as defined in [HMAC] and [SHA].

1. Message: The concatenation of the following byte sequences, in exact order:
    1. The full constructed URI string as ASCII: Derived from the CoAP Uri-Path and Uri-Query options as defined by [CoAP] Section 6.5 (steps 6-8). The URI string shall begin with a forward slash / character (0x2F), shall not include a null terminator, and shall not include the scheme or authority components (e.g. coap://host). If Uri-Query options are present in the packet, they shall be appended to the string using the ? and & delimiters as defined in the referenced RFC steps.

        Example byte sequence for /sig-net/v1/local/level/1: 0x2F 0x73 0x69 0x67 0x2D 0x6E 0x65 0x74 0x2F 0x76 0x31 0x2F 0x6C 0x6F 0x63 0x61 0x6C...

    2. The Sig-Net-Security-Mode option value: (1 byte).
    3. The Sig-Net-Sender-ID option value: (8 bytes).
    4. The Sig-Net-Mfg-Code option value: (2 bytes).
    5. The Sig-Net-Session-ID option value: (4 bytes).
    6. The Sig-Net-Seq-Num option value: (4 bytes).
    7. The Application Payload bytes.

As the Sig-Net-Seq-Num increments for every transmission, each individual CoAP packet produces a unique HMAC even if the application payload remains identical (e.g. streaming static DMX levels).

If a Sender needs to retransmit a lost configuration command (e.g. after detecting no change in a Node's CHANGE_COUNT), it must treat the retry as a new transmission. It shall increment the Sig-Net-Seq-Num and generate a fresh HMAC. Retransmitting a packet with a previously used Sequence Number will cause the responder's strict anti-replay sequence enforcement (Section 8.6) to drop the packet.

Informative Note on Message ID and Token: Hop-by-hop CoAP header fields (such as Message ID and Token) are explicitly excluded from the HMAC calculation. This ensures that standard CoAP proxies can dynamically alter these transport fields for routing purposes without invalidating the Sig-Net security signature.

Informative Note on CoAP Code: The CoAP Code byte (which is always 0x02 for POST in Sig-Net) is also excluded from the HMAC calculation, consistent with the general exclusion of CoAP header fields. Because Sig-Net exclusively uses CoAP NON POST messages, an attacker cannot meaningfully alter the Code byte without rendering the packet invalid to a standard CoAP parser. The URI, which is covered by the HMAC, definitively identifies the resource and operation, providing equivalent protection to explicit Code byte authentication within the Sig-Net operational context.

8.6 Responder Processing and Replay Attack Prevention

Upon receiving a Sig-Net packet, a responder shall process the security option in the following order:

1. Security-Mode Pre-check: The responder shall inspect the Sig-Net-Security-Mode (Option 2076).
    a. If the value is not explicitly supported by the responder's firmware (e.g. a future cryptographic mode), the responder shall silently discard the packet and perform no further processing or cryptographic verification.
    b. If the value is 0xFF, the packet is an Offboarded Device Beacon. The responder shall not attempt HMAC verification and shall not act on the payload beyond presenting the

device's presence to the operator UI (as defined in Section 10.2.1). All further security processing steps shall be skipped.

   c. If the value is 0x00, the packet is a standard authenticated control message. If the responder is currently configured for Open Mode, it lacks the keys to verify the packet and shall silently discard it. Otherwise, the responder shall proceed to Step 2.
   d. If the value is 0x01, the packet is an Open Mode control message. The responder shall evaluate its own operational state. If the responder is currently onboarded with derived cryptographic keys (Secure Mode), it shall silently discard the packet to definitively prevent cryptographic downgrade attacks. If the responder has been explicitly configured for Open Mode via local opt-in, it shall proceed to Step 2, but shall completely bypass Step 8 (Freshness), Step 9 (Integrity & Authentication), and Step 10 (State Commit).

2. CoAP Version Check: The responder shall verify that the Ver field in the CoAP header is set to 1 (binary 01), as mandated by [CoAP] Section 3. If any other value is present, the responder shall silently discard the packet and perform no further processing.
3. Token Handling: The responder checks the TKL (Token Length) field in the CoAP Base Header. Sig-Net does not use CoAP Tokens for state tracking. If TKL > 0, responders shall skip the Token bytes and proceed to parse the Options. The presence of a Token shall not cause packet rejection.
4. CoAP Code Check: The responder shall inspect the CoAP Code field in the base header. Sig-Net V1 exclusively utilises POST (0x02). If the Code is any other value, the responder shall silently discard the packet and perform no further processing.
5. URI Version Check: The responder shall verify the <version> string in the URI-Path as defined in Section 12.3. If the version is unsupported, the packet shall be discarded.
6. URI Routing Check: The responder parses the CoAP Uri-Path option.
   a. The responder shall verify the <scope> string in the URI-Path. If the scope does not match the responder's currently configured scope, the packet shall be silently discarded and perform no further processing or cryptographic verification.
   b. If the URI contains a targeted {tuid} (such as a /manager/ or /node/ command) and the TUID does not match the responder's own TUID, the responder is not the intended recipient. The responder shall silently discard the packet and perform no further processing or cryptographic verification.
7. Session & Sequence Evaluation: The responder extracts the Sig-Net-Session-ID, Sig-Net-Seq-Num, and Sig-Net-Sender-ID option.
   a. The Sender TUID (the first 6 bytes of the Sender-ID) is the absolute identifier for Session ID tracking.
   b. The full 8-byte Sender-ID is the absolute identifier for Sequence Number tracking, allowing multi-endpoint gateways to maintain independent sequence threads per physical port.
8. Freshness: The responder evaluates the freshness of the packet:
   a. If the received Session ID is less than the stored Session ID for this TUID, the packet is a replay; the responder shall silently drop the packet.
   b. If the received Session ID equals the stored Session ID, but the Sequence Number is less than or equal to the stored Sequence Number for this Sender-ID, the packet is a replay; the responder shall silently drop the packet.
   c. For the handling of packets from previously unknown Sender TUIDs where no Session ID record exists, see the First-Packet Bootstrapping rule in Section 7.2.1.

d. If the packet passes these freshness checks, the responder proceeds to Step 8. The responder shall NOT update its stored sequence state or write to non-volatile memory yet.

Security Note (Anti-Replay): Because Sig-Net-Session-ID uses an increasing integer, a received Session ID that is lower than the stored value is definitively a replay and is rejected. A received Session ID that is greater is definitively a legitimate reboot or sequence wrap. Therefore, artificial "hold-off" delays are not required and should not be implemented, as they would cause unacceptable operational delays. Nodes shall report detected replay anomalies to the network using the TID_DG_SECURITY_EVENT payload.

Security Note (Network Robustness): Because state tracking is cryptographically bound to the Sender TUID and not the IP address, network events such as DHCP IP renewals, or operators manually swapping a backup console into the IP address of a crashed primary console, will not falsely trigger anti-replay lockouts. The backup console will present a new Sender TUID and automatically trigger First-Packet Bootstrapping.

9. Integrity & Authentication: The responder calculates the expected HMAC using the derived role key designated by the URI as defined in Section 15 and the inputs defined in Section 8.5. It performs a constant-time comparison against the received Sig-Net-HMAC option. If the hashes do not match, the packet is forged or corrupted; the responder shall silently drop the packet. Nodes shall report sustained HMAC verification failures to the network using the TID_DG_SECURITY_EVENT payload.

   Security Note (DoS Mitigation): Whilst a failed HMAC drops an unauthorised packet, the packet consumed bandwidth and the cryptographic calculation consumes CPU resources. Nodes shall implement protection to ensure that excessive unwanted traffic does not significantly impact processing of desired network traffic and tasks unrelated to network processing.

   When this processing budget is exhausted by a flood of incoming packets, the Node shall simply drop unprocessed packets from its receive buffer. While this will result in the dropping of legitimate traffic, it allows the underlying hardware to remain stable. The Node will resume normal operation when network flood ceases.

   Nodes shall not implement automated blocking based on UDP Source IP addresses in an attempt to filter the flood, as attackers can trivially spoof the legitimate Sender's IP address to trigger a self-inflicted denial of service. True volumetric DoS mitigation must be handled by physical network isolation or switch-level rate limiting.

10. State Commit: Only after the HMAC is successfully verified shall the responder update its internal security state.
    a. RAM-Only Tracking: To prevent premature hardware failure through non-volatile memory exhaustion and to accommodate standard "power-cycle" troubleshooting, responders shall store all received Session ID and Sequence Number data in volatile memory (RAM) only.
    b. Tracking Update: The responder shall update its internal RAM table for the associated Sender TUID and Sender-ID with the newly received values.
    c. Storage Capacity and Anti-Exhaustion: Responders shall support tracking a minimum of 32 unique Sender TUID records. If a packet arrives from a new TUID and the internal table is full, the responder shall evaluate the Least Recently Used (LRU) record. To prevent an attacker from intentionally flushing the table with spoofed TUIDs to facilitate a replay attack, the responder shall only overwrite the LRU record if that Sender has not

transmitted a valid packet within the last 3600 seconds (1 hour). If the LRU record is newer than 1 hour, the responder shall assume the table is saturated with legitimate active Senders and shall silently drop the new, unauthenticated packet.

Informative Note: While received state is stored in RAM, devices acting in a Sender or Manager role must still commit their own transmitted Session ID to non-volatile memory during initialisation as required by Section 8.3.

11. Acceptance: The responder executes the Application Payload.

8.6.1 Manager Processing of Replies

Managers shall apply the same Session ID and Sequence Number validation rules defined above when processing reply packets received from Nodes on <mult_node_send>. Each responding Node's TUID serves as the sender identifier for tracking sequence freshness.

9 Transport

9.1 Multicast Use

Because Sig-Net relies on derived symmetric keys for authentication across the entire network, it natively supports efficient one-to-many communication via UDP Multicast. A Sender signs a packet once using its specific role key, and all authorised Sig-Net Devices within the multicast group can verify and process it simultaneously. This is highly advantageous for high-density universe streaming, discovery, and global sync commands.

9.2 Multicast Address Mapping

Sig-Net uses a deterministic mapping of CoAP URIs to specific UDP Multicast IP addresses to minimise configuration overhead, prevent network flooding, and ensure strict isolation from legacy unauthenticated protocols.

9.2.1 URI to Multicast Determinism

Sig-Net enforces a strict one-to-one relationship between a CoAP URI and its destination UDP Multicast IP address. Senders and Nodes are not required to maintain complex routing tables; the destination URI deterministically defines the Multicast IP. Where this specification instructs a device to "transmit to URI", the implementation shall automatically resolve that URI to the corresponding multicast address as defined in Section 15.

9.2.2 Discovery and Management Subscriptions

All onboarded Nodes shall subscribe to <mult_manager_poll> and <mult_manager_send>.

All Managers shall subscribe to <mult_node_send>, <mult_node_lost>, and <mult_node_beacon>.

Senders that only transmit level data, sync, or timecode do not require these subscriptions..

9.2.3 Default Multicast Folding™ (Universe Mapping)

Sig-Net uses a default pool of 109 Multicast IP addresses (<mult_u1> to <mult_u109>) to prevent IGMP snooping table exhaustion on network switches. The use of a prime number (109) prevents clustering if an operator patches universes in repeating patterns.

All Sig-Net devices shall, by default, map universes to a Multicast IP using a modulo-109 operation. This ensures that for installations of 109 universes or fewer, each universe is assigned a unique Multicast IP, resulting in zero Discard Load.

The last digit (Index) of the IPv4 multicast address is calculated as follows:

Index = ((Universe - 1) % 109) + 1

Example mapping:

- Universe 1, 110, 219... map to <mult_u1> (239.254.0.1)
- Universe 109, 218, 327... map to <mult_u109> (239.254.0.109)

Because multiple universes share a single multicast address once the 109-address pool is exceeded, the CoAP URI-Path (e.g. /sig-net/v1/<scope>/level/110) serves as the definitive hardware-layer routing filter. Responders shall parse the URI and silently discard packets addressed to universes to which they are not assigned.

Informative Note. The 109 multicast groups are just defined for default operation. Sig-Net may be configured to operate with any number of multicast groups. However, system operators should ensure that they do not allocate more multicast groups than can be processed by the least capable switch in the network.

9.2.4 Multicast Folding™ Configuration and Overrides

While the 100-address pool is the mandatory default to guarantee out-of-the-box interoperability, this mapping scheme is configurable via Sig-Net management commands. This provides an option to balance and optimise grouping for complex systems.

A Manager may assign a custom Multicast IP to any specific Data Endpoint using the TID_EP_MULT_OVERRIDE command. When a custom IP is applied, it shall be treated as a non-volatile setting. Support for custom multicast overrides is optional; Nodes that do not support this feature shall silently discard the TID_EP_MULT_OVERRIDE command.

Senders can detect if a Node is using custom routing by inspecting the TID_RT_MULT TLV, which is returned in the Node's basic poll reply.

If a Manager updates an endpoint's TID_EP_UNIVERSE, the Node shall revert to the default folded multicast address, unless a valid TID_EP_MULT_OVERRIDE TLV is included in the same CoAP packet.

Informative Note: The ability to remap multicast addressing with TID_EP_MULT_OVERRIDE is an advanced feature that is intended to be applied algorithmically rather than from a user configuration menu.

Split Routing Edge Case: TID_EP_MULT_OVERRIDE shall not be used to configure a network such that the same Universe number is expected on multiple different Multicast IP addresses simultaneously. If a Universe is moved to a custom IP address via TID_EP_MULT_OVERRIDE, all Nodes consuming that Universe on the network must be patched to that same custom IP. Senders are only required to transmit a Universe to a single Multicast IP address; split-routing a single universe across multiple IPs is undefined behaviour.

9.2.5 Multicast Interoperability and Safety Reset

To ensure reliable interoperability in environments where control consoles or Managers are frequently interchanged (e.g. touring or festival environments), a Manager may force the network to abandon any previously configured custom routing plans.

A Manager may transmit a TID_RT_MULT_OVERRIDE command with a value of 0x00 (Default) to target Nodes. This command forces the Node to discard any previous TID_EP_MULT_OVERRIDE settings and return to the standard predictable folding pool. To reset an entire network, the Manager shall transmit this command individually to the TUID of every discovered Node (via Unicast or <mult_manager_send>), authenticated with each Node's respective Km_local key.

Operational Safety: Because this command immediately alters the network architecture and triggers non-volatile memory writes across all Nodes, Managers shall not transmit this command automatically during their initialisation sequence. This safety reset must only be executed as a deliberate, user-initiated action.

9.2.6 Sync and Timecode Address

Both synchronisation commands and timecode streams share a single, fixed administrative multicast address (<mult_time>).

9.3 Port Assignment

All Sig-Net traffic shall be transmitted to the registered CoAP UDP port <coap_port>.

9.4 CoAP Message Type and Method

All Sig-Net packets shall be transmitted as CoAP NON POST messages.

Informative Note: Because Sig-Net uses CoAP NON exclusively, configuration command packets may be silently lost in transit. The CHANGE_COUNT mechanism (Section 10.2.3) provides eventual consistency but does not guarantee immediate delivery.

For critical configuration changes, Managers shall verify the Node's replied state by monitoring the CHANGE_COUNT in subsequent TID_POLL_REPLY packets. If the CHANGE_COUNT does not reflect the expected change within <node_lost_timeout> consecutive poll cycles, the Manager shall retransmit the configuration command. Each retransmission shall use a new incremented Sig-Net-Seq-Num and freshly calculated HMAC as required by Section 8.5. Managers shall not retransmit more than five times for a single configuration command without presenting an error condition to the operator. Managers should implement an exponential or fixed inter-retransmission delay of at minimum one poll cycle (<poll_time>) between retransmission attempts to avoid unnecessary network load

9.5 Network Infrastructure Requirements

Network switches shall support IGMPv2 as a minimum. IGMPv3 is recommended for large installations.

An IGMP querier shall be active on any network or VLAN where IGMP snooping is enabled.

Informative Note: Direct device-to-device connections, or networks utilising unmanaged switches without IGMP snooping, do not require an IGMP querier.

When a Data Endpoint's universe assignment or custom routing is altered, the Node shall dynamically manage its active network subscriptions. If a configuration change results in the Node no longer requiring an active subscription to a specific Multicast IP address across any of its Endpoints, the Node shall immediately transmit an IGMPv2 Leave Group message for that discarded IP address. Conversely, if a configuration change introduces a requirement for a new Multicast IP address, the Node shall join the new group before transmitting or receiving data on that Endpoint.

9.6 Multicast TTL

All Sig-Net multicast packets shall be transmitted with a default IP Time To Live (TTL) value of <mult_ttl>. This value is selected to provide sufficient headroom for routed single-site venue deployments while preventing Sig-Net traffic from propagating beyond the intended network boundary.

Informative Note: In air-gapped deployments, network administrators should additionally configure multicast boundary filters at any edge router to prevent Sig-Net traffic from leaking onto external networks regardless of TTL value. A TTL of <mult_ttl> is a defence-in-depth measure, not a substitute for boundary filtering.

10 Payload Syntax

10.1 Introduction

Sig-Net application payloads are structured using Type-Length-Value (TLV) encoding to maximise network efficiency. This allows multiple related data structures to be packed sequentially into a single CoAP payload and allows for variable-length arrays (such as sparse universe updates) to conserve bandwidth.

The basic structure of a Sig-Net TLV is:

- Type Identifier (TID) (2 Bytes): An identifier defining the nature of the data (Section 11.9).
- Length (2 Bytes): A 16-bit integer (Network Byte Order) defining the exact size of the Value field in bytes.
- Value (Variable): The actual data payload.

A single Sig-Net CoAP packet may contain one or more TLVs appended sequentially.

Forward Compatibility Rule: If a device receives a TLV containing a Type Identifier (TID) that it does not recognise or support, it shall use the Length field to silently skip over the Value payload and continue parsing the next TLV in the sequence. Unknown TIDs shall not cause the packet to be rejected.

10.1.1 TID Allocation and Namespaces

The 16-bit Type Identifier (TID) is divided into two namespaces:

- Standard TIDs (0x0000 to 0x7FFF): Reserved for identifiers defined within this specification and its official future extensions.
- Manufacturer TIDs (0x8000 to 0xFFFF): Reserved for manufacturer-proprietary data.

10.1.2 Proprietary TID Resolution

Multiple manufacturers may use the same TID in the 0x8000+ range. A Manufacturer TID shall only be interpreted in the context of the Sig-Net-Mfg-Code (Option 2140) provided in the CoAP header.

12. A Node receiving a TID in the proprietary range (>= 0x8000) shall first inspect the Sig-Net-Mfg-Code option.
13. If the Mfg-Code does not match the Node's own ESTA Manufacturer ID, the Node shall silently discard the TLV and use the Length field to skip to the next item in the payload.
14. If the Mfg-Code matches, the Node may process the TID according to the manufacturer’s internal specification.

10.2 Node Discovery and Polling

Sig-Net uses a polling mechanism to handle node discovery and network-level configuration without requiring a central broker.

10.2.1 Offboarded Operation (Beacon Mode)

An offboarded Sig-Net Device is one that has not yet been physically supplied with a Root Key (K0), and therefore lacks the derived cryptographic keys (Km_local, Kc, Ks) required to generate or verify an HMAC.

An offboarded device cannot participate in authenticated network communication. It shall silently discard all incoming Sig-Net packets.

To ensure unauthenticated traffic does not congest the primary discovery network, offboarded devices in Beacon Mode shall periodically announce their presence exclusively to the <mult_node_beacon> multicast group using the URI /sig-net/<version>/local/node_beacon/{tuid}.

Beacons shall always be transmitted on the local scope, ensuring they remain discoverable to newly connected Managers regardless of the Manager's operational scope configuration.

This presence beacon shall be transmitted periodically at the <beacon_min_interval> rate. To facilitate identification by a Manager, the CoAP Options and payload for a beacon shall be populated with the following static values:

CoAP Security Options:

- Sig-Net-Security-Mode (2076): 0xFF (Offboarded).
- Sig-Net-Sender-ID (2108): The TUID of the offboarded device, with the Endpoint bytes set to 0x0000.
- Sig-Net-Mfg-Code (2140): 0x0000.
- Sig-Net-Session-ID (2172): 0x00000000.
- Sig-Net-Seq-Num (2204): 0x00000000.
- Sig-Net-HMAC (2236): Length 0.
- Application Payload shall consist of (in this order)
    - TID_POLL_REPLY (with CHANGE_COUNT set to 0x0000)
    - TID_RT_DEVICE_LABEL
    - TID_RT_ENDPOINT_COUNT
    - TID_RT_OTW_CAPABILITY (If the device supports OTW onboarding)

To minimise the discovery-layer attack surface and prevent offboarded devices from congesting the network, devices operating in Beacon Mode shall adhere to the following:

- Transmission Rate: A device shall not transmit beacons more frequently than once every <beacon_min_interval> seconds.
- Internal Failure Handling: If a firmware or hardware error causes a device to lose its timing baseline, it should cease beacon transmissions until the next physical power cycle rather than risk flooding the discovery network.
- Logging: Managers should provide a mechanism for local or network-based logging of beacon activity if supported by the hardware.
- Multicast Isolation: All unauthenticated Security-Mode 0xFF beacons shall be transmitted exclusively to the <mult_node_beacon> address.

When displaying offboarded device beacons to an operator, A Manager shall implement the following safeguards:

- Source Display: Display the source IP address of each beacon alongside the reported TUID and Device Label.
- Anomaly Detection: If multiple beacons are received claiming the same TUID from different source IP addresses within a <beacon_timeout> window, the Manager SHALL flag this entry as a potential spoofing anomaly and alert the operator.
- Stale Entry Removal: Automatically remove beacon entries that have not been refreshed within <beacon_timeout> seconds.
- Onboarded Device Exclusion: The Manager should not display beacon entries for TUIDs that are already present in the authenticated device table.

Informative Note: The isolation of beacons to a dedicated multicast address is the primary defence against unauthenticated Denial of Service. This allows network administrators to configure switches to heavily rate-limit or entirely drop traffic destined for <mult_node_beacon> during live show operation, ensuring that discovery-layer traffic cannot impact show-critical authenticated control data.

10.2.2 Manager Operation

A Manager initiates discovery by sending a Sig-Net packet with a payload containing a TID_POLL TLV to the URI /sig-net/<version>/<scope>/poll at an interval <poll_time>. This packet shall be authenticated using Km_global.

Managers shall implement poll snooping to prevent multicast storms in multi-Manager environments. A Manager shall monitor the <mult_manager_poll> multicast group. To prevent simultaneous poll transmissions from multiple Managers (collisions), a Manager shall calculate a randomised delay window for each polling cycle by adding a jitter value, uniformly distributed between 0 and <manager_poll_jitter> milliseconds, to the <poll_time> interval. A Manager shall only transmit a TID_POLL if no other valid TID_POLL has been received from another Manager within the combined <poll_time> plus jitter window. If a valid TID_POLL is received from another Manager at any point prior to transmission, the Manager shall immediately cancel its pending poll, suspend its own polling cycle, and reset its internal <poll_time> second timer.

Requests for configuration or diagnostic state (QUERY_LEVEL>0) must be controlled to prevent severe burst-traffic packet loss on the management subnet. A Manager shall not transmit global QUERY_LEVEL>QUERY_HEARTBEAT polls as part of its continuous, routine background discovery process. During routine operation, a Manager shall only transmit a targeted QUERY_LEVEL>0 poll (matching TUID_LO and TUID_HI to a specific device) in response to detecting a CHANGE_COUNT increment from that Node.

Initialisation Exception: A Manager is permitted to transmit a network-wide global QUERY_LEVEL=QUERY_FULL (TUID_LO=0, TUID_HI=MAX) exclusively upon its initial boot sequence, connection to a new network, or operator-initiated manual 'Full Refresh' command, in order to populate its initial internal state tables.

Open Mode Discovery: To discover devices operating in Open Mode (Section 7.2.5), a Manager may optionally interleave unauthenticated discovery requests by multicasting a TID_POLL with the Sig-Net-Security-Mode set to 0x01.

10.2.3 Node Operation

Upon receipt of a TID_POLL, a node shall check that its TUID is in range. If in range, the Node evaluates the QUERY_LEVEL byte (Value [14]):

Query Enumerations:

- 0x00 (QUERY_HEARTBEAT): The Node returns the TID_POLL_REPLY and all TIDs marked as HEARTBEAT in Section 11.9. This is used for discovery and presence detection.
- 0x01 (QUERY_CONFIG): The Node returns the Heartbeat set plus those marked as CONFIG (e.g. Labels, Universes, Directions).
- 0x02 (QUERY_FULL): The Node returns the Config set plus those marked as FULL (e.g. Firmware, IP Settings, MAC, Supported TIDs).
- 0x03 (QUERY_EXTENDED): The Node returns the Full set plus those marked as EXTENDED (e.g. Security Events, Diagnostic Messages).

The Node shall generate a reply for the requested endpoint(s). Every reply packet payload shall begin with a TID_POLL_REPLY. This shall be followed by the supported TIDs relevant to the requested endpoint(s) whose Poll Category (Section 11.9) is less than or equal to the requested QUERY_LEVEL.

If the resulting collection of TLVs exceeds the 1,400-byte UDP payload limit, the Node shall split the data into multiple sequential packets as defined in Section 10.2.4.

Before transmitting any reply, a Node shall evaluate the poll scope.

- If TUID_HI == TUID_LO (indicating a poll targeted at this specific Node only), the Node shall reply within <node_processing_max> milliseconds.
- If TUID_HI != TUID_LO (indicating a range or broadcast poll), the Node shall delay for a random period uniformly distributed between 0 and <poll_backoff_max> milliseconds. The Node shall then transmit its reply within <node_processing_max> milliseconds of the conclusion of this random delay.
- All-Endpoints: If the TID_POLL requests data for All Endpoints (END_POINT = 0xFFFF), a Node with multiple Data Endpoints shall not transmit all of its endpoint replies as a simultaneous, back-to-back burst. The Node shall introduce a minimum pacing delay of <endpoint_spacing_delay> milliseconds between transmitting the UDP packet for Endpoint 1, the UDP packet for Endpoint 2, and so forth. This prevents high-density Nodes from overflowing the receive buffers of the Manager.

Informative Note on Backoff Duration: In range and broadcast polls, the random delay serves to prevent simultaneous reply storms from large numbers of Nodes. The value of <poll_backoff_max> defines the upper bound of this delay window and is specified in Section 16. The actual delay should be uniformly distributed between 0 and <poll_backoff_max>.

10.2.4 Fragmentation

A Node shall ensure that no single UDP payload exceeds a maximum safe length of 1,400 bytes. To prevent complex transport-layer fragmentation, no individual TLV shall exceed this limit.

If the total collection of TLVs required to satisfy a QUERY_FULL poll exceeds this limit, the Node shall sequentially transmit multiple independent CoAP packets to the same Reply URI. Each packet shall be a mathematically valid Sig-Net payload in its own right, allowing the Manager to process them as independent state updates regardless of UDP delivery order.

10.2.5 On-Boot Notification

Every Sig-Net Device (Manager, Sender, and Node) shall proactively announce its presence upon completing its boot sequence and establishing a network connection.

After a random delay (uniformly distributed between 0 and <poll_backoff_max> milliseconds), the device shall transmit a single CoAP packet to the Reply URI (/sig-net/<version>/<scope>/node/{tuid}/0) signed with the Citizen Key (Kc).

To ensure traceability for regulatory compliance and automated conformance testing (SNACtest), the application payload shall comprise the following TLVs appended in exact order:

- TID_POLL_REPLY
- TID_RT_PROTOCOL_VERSION
- TID_RT_ROLE_CAPABILITY
- TID_RT_ENDPOINT_COUNT
- TID_RT_MULT_OVERRIDE
- TID_RT_OTW_CAPABILITY (If OTW onboarding supported).

The transmission of this packet shall be completed within <node_processing_max> milliseconds of the conclusion of the random delay.

10.2.6 Node Loss Detection (Lost Mode)

A Manager shall consider a Node lost if no TID_POLL_REPLY has been received from that Node within <node_lost_timeout> consecutive poll cycles.

Nodes are Manager-agnostic and do not track individual Manager identities for discovery purposes. If an onboarded Node has not received a valid TID_POLL request from any Manager within <node_lost_timeout> consecutive expected poll intervals, the Node shall assume the entire management network has gone offline and shall enter Lost Mode. In Lost Mode, the Node shall:

- Continue to process and output data from active Senders normally (Stream Loss is evaluated independently as defined in Section 10.6.2).
- Transmit a periodic packet to the URI /sig-net/<version>/<scope>/node_lost/{tuid} at the standard poll interval, signed with its global Citizen Key (Kc), to signal its continued presence to any Manager that comes online.

The payload of this periodic node_lost transmission shall comprise the following TLVs appended in exact order:

- TID_POLL_REPLY
- TID_RT_PROTOCOL_VERSION
- TID_RT_ROLE_CAPABILITY
- TID_RT_ENDPOINT_COUNT
- TID_RT_MULT_OVERRIDE
- TID_RT_OTW_CAPABILITY (If OTW onboarding supported).

This allows any newly connected Manager to identify the orphaned Node and its routing state.

Informative Note for Managers: If an active Manager receives a packet on the /node_lost/ URI from a Node it is already tracking as "Online," the Manager should assume its own outbound TID_POLL multicasts are failing to reach that specific Node, indicating a one-way network routing failure.

10.2.7 State Synchronisation and Consistency Tracking

TID_POLL_REPLY includes a 16-bit wrap-around CHANGE_COUNT variable that is designed to guarantee eventual consistency across multiple Managers without requiring reliable transport.

The CHANGE_COUNT shall increment by 1 whenever any Native Status or Configuration state (TIDs) on a Node changes, whether triggered by a network command, a local user interface interaction , or as a side-effect of an encapsulated RDM transaction.

Note: The CHANGE_COUNT shall not increment when encapsulated RDM parameter states change, as RDM state changes are handled independently via Section 10.5.2.

Proactive Notification:

When a Node's configuration or internal status changes, it shall immediately transmit a proactive state update to the URI /sig-net/<version>/<scope>/node/{tuid}/{endpoint}, where {endpoint} represents the specific port that was altered (or 0 for a global change).

To conserve bandwidth, the payload of this proactive notification shall consist of a TID_POLL_REPLY (containing the new global CHANGE_COUNT), followed immediately by only the specific configuration TLV(s) that were altered on that endpoint.

Rate Limit: To prevent high-frequency ephemeral status changes (e.g. DMX data activity bits flipping) from generating continuous multicast storms, Nodes shall rate-limit proactive state notifications for Status TIDs (such as TID_RT_STATUS and TID_EP_STATUS). A Node shall not proactively multicast status updates for the same endpoint more frequently than once every <status_publish_rate> seconds.

Consistency Tracking:

A Manager shall actively track the CHANGE_COUNT of every discovered Node. If a Manager receives a routine TID_POLL_REPLY (QUERY_LEVEL=QUERY_HEARTBEAT) during background discovery and detects that the CHANGE_COUNT is greater than its internally cached value for that Node, the Manager knows it missed a proactive configuration update.

Upon detecting a CHANGE_COUNT discrepancy, the Manager shall immediately transmit a targeted TID_POLL (matching TUID_LO and TUID_HI to the specific Node) with QUERY_LEVEL>QUERY_HEARTBEAT to fetch the updated configuration state.

IP Address Resolution: Because a CHANGE_COUNT increment may indicate that the Node's physical IP address was altered by another Manager, the polling Manager SHALL update its internal routing table using the Source IP address of the received UDP TID_POLL_REPLY packet before transmitting the targeted Unicast TID_POLL. This ensures the follow-up query is correctly routed to the Node's current network location.

10.3 Targeted Node Routing

To send data directly to a specific Node or read its state (such as Configuration changes or RDM transactions), Sig-Net uses a targeted routing model. Data is routed using the target Node's TUID and Endpoint.

10.3.1 Targeted URIs

Two specific URIs are defined to separate incoming commands from outgoing state reports and replies:

- Discovery URI (/sig-net/<version>/<scope>/poll): Used exclusively by Managers to multicast discovery requests (TID_POLL) to all Nodes on the network. Packets sent to this URI shall be sent via Multicast UDP to <mult_manager_poll> and shall be authenticated using the Global Manager Key (Km_global).
- Command URI (/sig-net/<version>/<scope>/manager/{tuid}/{endpoint}): Used by Managers to send Native Configuration changes (TIDs) or RDM commands to a specific Node. Packets addressed to this URI may be sent via Unicast UDP or Multicast UDP (<mult_manager_send>), and shall be authenticated using the target Node's unique Km_local key.
- Reply URI (/sig-net/<version>/<scope>/node/{tuid}/{endpoint}): Used by Nodes to multicast their current configuration state or RDM Push Notifications. Packets sent to this URI shall be signed using the global Citizen Key (Kc).

10.3.2 Unicast vs. Multicast Command Routing:

To minimise unnecessary network traffic on Nodes, Managers should default to transmitting all Parameter Updates (SET), Parameter Queries (GET), and TID_RDM_COMMAND via Unicast.

However, to allow management in environments where a newly connected Node resides on an unreachable or misconfigured IP subnet, Managers may transmit targeted commands via Multicast UDP to the <mult_manager_send> group.

- **Security and Processing:** Regardless of whether the command is Unicast or Multicast, the packet shall target a specific {tuid} in the URI and shall be authenticated using that Node's unique Km_local key. Nodes receiving traffic on <mult_manager_send> shall parse the cleartext URI and silently discard any packets that do not match their own TUID, as required by Section 8.6 (Step 5). This guarantees that multicast configuration routing cannot be used to execute network-wide configuration changes or elevate privileges.

10.4 Node Configuration and State Synchronisation

Sig-Net uses a Directional Data Flow model to manage device configuration. The nature of a transaction (whether it is an update, a query, or a report) is determined by the Destination URI and the TLV Length field, rather than explicit command bytes within the payload.

10.4.1 Directional URI Mapping

The Destination URI defines the role and intent of every configuration packet:

- Command URI (/manager/{tuid}/{endpoint}): Used by a Manager to send instructions to a Node. The Node interprets packets on this URI as either a Parameter Update or a Parameter Query.
- Reply URI (/node/{tuid}/{endpoint}): Used by a Node to multicast its current state. The Node sends packets to this URI in response to a Parameter Query, as a confirmation of an update, or as a Proactive Change Notification.

10.4.2 Parameter Updates (SET)

To change a configuration parameter, a Manager transmits the corresponding TID with a Length > 0 directly to the Node's Command URI.

- Batching: A Manager may pack multiple Parameter Updates (SET) into a single CoAP packet to improve network efficiency, provided the total payload does not exceed the 1,400-byte UDP limit defined in Section 10.2.4.
- The Node verifies the HMAC and parses the TLV.
- The Node updates its internal state and commits the change to non-volatile memory.
- The Node increments its CHANGE_COUNT (Section 10.2.7) and within <node_processing_max> ms, multicasts the updated state to its Reply URI for all listening Managers to observe.

10.4.3 Parameter Queries (GET)

To retrieve the current value of one or more parameters, a Manager transmits the corresponding TID with a Length = 0x0000 to the Node’s Command URI.

- Batching: A Manager may pack multiple Query TIDs into a single CoAP packet.
- Queryable TIDs: Any TID marked as "Queryable: Yes" in Section 11 shall be interpreted as a request for the Node's current state.
- Response Packing: To minimise network overhead, the Node should pack all requested responses into as few CoAP packets as possible, subject to the 1,400-byte UDP limit defined in Section 10.2.4.
- Error Handling: If a requested TID is not supported or not Queryable on that endpoint, the Node shall silently ignore the request and continue parsing any subsequent TLVs in the packet.

Informative Note on Guest Manager Query Mechanics: Because the Command URI /manager/{tuid}/{endpoint} requires the Node-Specific Manager Key (Km_local) for authentication, a device operating in the Guest Manager state (which only possesses Km_global) is blocked from issuing targeted Parameter Queries (GETs). To retrieve specific parameter values or RDM Tables of Devices, a Guest Manager instead uses the TID_POLL mechanism (Section 11.2.2). The Guest Manager sends a targeted TID_POLL (matching TUID_LO and TUID_HI to the specific fixture) to the Discovery URI, authenticated with Km_global, and requests the appropriate QUERY_LEVEL. The Node will return the requested data blocks to the <mult_node_send> reply group. This allows Read-Only network discovery without requiring Km_local.

10.4.4 Batch State Retrieval (QUERY_LEVEL)

To retrieve a Node's configuration in bulk (e.g. during discovery), a Manager uses the TID_POLL mechanism (Section 10.2). The QUERY_LEVEL byte within the TID_POLL determines the density of information returned by the Node, as defined by the Poll Categories in Section 11.9.

10.4.5 Network Interface Configuration (IP Settings)

A Manager may configure a Node's IP address settings by transmitting Network Configuration TIDs (Section 10.4.5) to the Node's Root Endpoint.

In order to reduce the risk of accidental configuration change severing communication, atomic execution and rollback rules apply:

Atomicity: A Manager shall pack all required IPv4 or IPv6 parameters for the intended change into a single CoAP POST payload. The Node shall evaluate all network configuration TIDs within the packet simultaneously.

Because applying new IP settings inherently disrupts active network connectivity, the Node SHALL successfully transmit the confirmation response (containing the updated Parameter TIDs, as required by Section 10.4.2) using its existing, working IP configuration prior to applying the new settings to the hardware network stack. This guarantees the Manager receives cryptographic confirmation that the command was accepted before the Node temporarily drops offline to execute the change.

Rollback Verification: Upon successfully applying a new IP network configuration, the Node shall start a rollback timer of <ip_rollback_timer> seconds. To prove that the newly assigned IP address is fully routable and valid on the network, the configuring Manager shall immediately transmit a targeted, Unicast TID_POLL (matching TUID_LO and TUID_HI to the specific Node) directly to the Node's new IP address.

If the Node does not successfully receive and cryptographically verify a targeted Unicast TID_POLL directed to its specific TUID within the rollback window, it shall ignore any background Multicast TID_POLL traffic, assume the new network configuration is unreachable, and immediately discard the new settings. It shall revert to its previously working IP configuration and multicast its restored state to the network.

10.5 RDM Payload

Sig-Net encapsulates standard [E1.20] RDM packets inside TLVs. RDM transactions use the Targeted Node Routing architecture defined in Section 10.3.

The following Type Identifiers (TID) are defined:

- TID_RDM_COMMAND (Manager Command): Contains the raw E1.20 request packet generated by a Manager.
- TID_RDM_RESPONSE (Responder Response): Contains the raw E1.20 response packet generated by a responder.

The payload shall be a fully formatted RDM packet starting with the Start_Code and ending with the RDM_Checksum.

10.5.1 Transaction Responses

Regardless of whether a TID_RDM_COMMAND was received via Unicast or Multicast, the Node shall multicast its corresponding TID_RDM_RESPONSE reply within <node_processing_max> milliseconds of completing the physical RDM transaction.

10.5.2 Proactive State Notification

Sig-Net supports multi-Manager synchronisation without requiring a central broker. This is achieved by mandating proactive multicast notifications whenever a Node's parameter state changes.

> When a Node's RDM parameter state is altered, the Node shall proactively transmit a multicast TID_RDM_RESPONSE containing an RDM GET_COMMAND_RESPONSE reflecting the new state. This rule applies under the following conditions:

- Unicast Routing: The state change was triggered by a TID_RDM_COMMAND received via Unicast UDP.
- Multicast Routing: The state change was triggered by a TID_RDM_COMMAND received via Multicast UDP (<mult_manager_send>). To prevent network storms, the Node shall delay for a random period (between 0 and <poll_backoff_max>) before transmitting the proactive response. The response shall then be transmitted within <node_processing_max> milliseconds of the conclusion of this delay window.
- Local UI Change: The state change was triggered by a physical interaction with the device's local user interface.

Node behaviour on receipt of a GET command is defined in Section 10.5.1. Where the altered RDM parameter does not have a corresponding GET command defined in [RDM], the Node is not required to transmit a proactive TID_RDM_RESPONSE for that parameter change.

Sig-Net relies on UDP multicast to transport proactive RDM state notifications. Because UDP is a non-reliable transport, Sig-Net accepts the possibility that a proactive TID_RDM_RESPONSE may occasionally be dropped. Manager applications may wish to implement a low-frequency, background polling loop (issuing targeted TID_RDM_COMMAND GET requests for critical PIDs) to ensure their internal fixture state caches remain accurate over time.

To prevent devices from being overrun with redundant background GET requests in multi-Manager environments, only the Manager that is currently actively transmitting discovery requests (i.e. the Manager that has not suspended its polling cycle under the poll snooping rules defined in Section 10.2.2) shall execute this background RDM polling loop. All other Managers shall passively synchronise their state caches by snooping the resulting multicast TID_RDM_RESPONSE packets.

Informative Note for Gateways: The proactive notification requirement does not mandate that a Gateway cache the entire RDM parameter state of every downstream physical Responder in its internal memory. Instead, Gateways should rely on the native "Queued Message" mechanism defined in the [RDM] standard. When a downstream physical Responder locally alters a parameter or executes a Broadcast SET command, it queues a message. The Gateway simply retrieves the queued GET_COMMAND_RESPONSE during its routine background RDM polling, encapsulates the raw RDM packet within a TID_RDM_RESPONSE, and multicasts it to the Sig-Net network.

10.5.3 RDM Boundary Restrictions (Normative)

Sig-Net Native Configuration TIDs (Section 11.5) are the absolute authority for the device's physical network interface and routing topology.

To prevent state-machine conflicts and guarantee the enforcement of the Sig-Net IP Rollback Verification mechanism (Section 10.4.5), Virtual Endpoints shall reject any encapsulated TID_RDM_COMMAND SET requests that attempt to alter the device's fundamental IP network settings or alternate network routing topologies.

Specifically, Virtual Endpoints shall return an RDM NR_UNSUPPORTED_COMMAND_CLASS or NR_WRITE_PROTECT response to any SET command targeting PIDs defined within:

- [E1.33], [E1.37-2], and any possible future replacement for [E1.37-2].

Managers requiring to configure the IP address, Subnet Mask, Gateway, or MAC address of a Sig-Net Device shall utilise the authenticated Sig-Net Network Configuration TIDs directed to the Root Endpoint.

Informative Note: Virtual Endpoints may legitimately process read-only RDM queries (GET) for these network parameters to populate legacy console patching screens, provided the returned values accurately reflect the current Sig-Net native state.

10.5.4 Gateway Responsibilities

A Node that implements a Data Endpoint as a physical DMX/RDM port (a Gateway) shall adhere to the following rules:

- Background Queue Polling: The Gateway shall act as the sole E1.20 master on its downstream RS485 link for the purpose of retrieving queued messages. The Gateway shall run a continuous, autonomous background polling loop, transmitting GET QUEUED_MESSAGE to all discovered downstream E1.20 responders. When the Gateway retrieves a queued GET_COMMAND_RESPONSE, it shall encapsulate it within a TID_RDM_RESPONSE and proactively multicast it to the Sig-Net network.
- ACK_OVERFLOW Handling: The Gateway shall autonomously handle E1.20 ACK_OVERFLOW responses occurring on the downstream RS485 link. The Gateway shall internally generate the subsequent GET commands required to pump the remaining data. The Gateway shall not wait for the entire overflow sequence to complete before transmitting to the network; it shall encapsulate and multicast the TID_RDM_RESPONSE blocks as they arrive. The Gateway may perform short-term buffering to efficiently pack multiple RDM responses into a single CoAP TLV payload, provided it does not significantly delay transmission. Manager applications on the IP network shall not attempt to manually pump an ACK_OVERFLOW sequence through a Gateway Endpoint.

  Informative Note: [RDM] requires that a responder abort its ACK_OVERFLOW sequence if another PID is interleaved. Designers should ensure that other network traffic does not accidentally trigger this condition.

- ACK_TIMER Handling: If a downstream responder replies with an ACK_TIMER, the Gateway shall encapsulate and multicast this response to the Sig-Net network. Manager applications shall treat this network ACK_TIMER purely as an informative "transaction pending" status; they should use it to extend their internal UDP retry timeouts, but shall not transmit follow-up IP requests to resolve the timer. The Gateway shall autonomously handle the physical RS485 timer delay and the subsequent data retrieval, multicasting the final data response to the network once the transaction completes on the serial link.

10.6 Universe Payload

The {universe} variable in the URI shall be a decimal integer in the range of 1 to 63999.

The following Type Identifiers (TID) are reserved for Universe data sent to the URI: /sig-net/<version>/<scope>/level/{universe}

- TID_LEVEL (Zero Start Code DMX): The Value field contains up to 512 bytes of slot data.
- TID_PRIORITY (Priority Data): The Value field contains 1 to 512 bytes of priority values (0-200).
  - Universe Priority: If the payload length is 1, the single byte value shall be applied uniformly to all 512 slots.

- Per-Slot Priority: If the length is greater than 1, the bytes map sequentially to the priority slots starting at Slot 1. Any remaining unaddressed priority slots (up to 512) shall be set to 0.
- Constrained Node Fallback: Nodes that support Data Endpoints should support the full 1 to 512-byte range for TID_PRIORITY. If a Node is memory-constrained and cannot store a 512-byte per-slot priority buffer, it shall use the first byte of any received TID_PRIORITY payload as a global Universe Priority, applying that value to all slots in its merge calculation. It shall then silently discard the remaining bytes.

A Sender shall send priority data subject to the rules defined in [ETC 0xDD] . A Sender may pack a TID_PRIORITY and a TID_LEVEL into the same packet.

Sig-Net uses the 8-byte Sig-Net-Sender-ID CoAP option to identify sources. When a Node applies the per-slot merge rules defined in [ETC 0xDD], it shall use the full 8-byte Sig-Net-Sender-ID to uniquely identify the source of the TID_LEVEL stream. This guarantees that multi-port gateways routing separate physical DMX inputs to the same Sig-Net universe are correctly processed as distinct, mergeable sources.

If a Node receives a TID_LEVEL packet on a universe with no accompanying TID_PRIORITY data (either in the same packet or from any prior packet in the current session), it shall treat all slots in that packet as having a priority value of 100, consistent with the default priority defined in [ETC 0xDD].

Once TID_PRIORITY data has been received for a universe, the Node shall use the received priority values for merge calculations until priority data is updated or the Sender's session expires.

When a node receives TID_LEVEL data from multiple Senders on the same universe, it shall apply the per-slot merge rules defined in [ETC 0xDD], using the priority values conveyed in TID_PRIORITY.

When a Sender packs both TID_PRIORITY and TID_LEVEL into a single CoAP payload, TID_PRIORITY shall be placed before TID_LEVEL in the TLV sequence. This allows Nodes to apply the priority context before processing level data, avoiding the need to buffer and reprocess TLVs.

10.6.1 Example Universe Payload (Informative)

The example below shows a universe payload with a single TID_LEVEL TLV.

Figure 3 Wireshark Example

```
Data (516 bytes)
[Community ID: 1:nKqNAN8icIrgOritQGJnUO5NcYc=]
Sig-Net
    01.. .... = CoAP Version: 1
    ..01 .... = CoAP Type: Non-Confirmable (1)
    .... 0000 = Token Length: 0
    CoAP Code: 0x02
    CoAP Message ID: 0x0063
    Sig-Net URI: /sig-net/v1/local/level/1
    Sig-Net Version: v1
    Sig-Net Resource: local/level/1
    Security Mode: Plaintext payload with HMAC-SHA256 (0x00)
    Sender ID: 534ccd14afe70001
    Sender TUID: 534CCD14AFE7
    Sender Endpoint: 1
    Manufacturer Code: 0x0000
    Session ID: 0x00000001
    Sequence Number: 99
    HMAC: 111c13d37bd9b2f1c1c1b98f60fa51fac78276c82e3c9b105d548a21b31e4992
    CoAP Header: ver=1, type=Non-Confirmable, code=0x02, message_id=0x0063
    URI Path: /sig-net/v1/local/level/1
    Sig-Net CoAP Options (11)
    Sender ID Summary: 534CCD14AFE7/1
    Security Mode Summary: Plaintext payload with HMAC-SHA256 (0x0)
    Payload […]: 010102009c63009c63009c63009c63009c63009c63009c63009c63009c6
    TLVs (516 bytes)
        1: TID_LEVEL (0x0101), Length 512
            TID: TID_LEVEL (0x0101)
            Length: 512
            Value […]: 9c63009c63009c63009c63009c63009c63009c63009c63009c
            Level slots: 512
            Preview: 1:156, 2:99, 3:0, 4:156, 5:99, 6:0, 7:156, 8:99, 9:0, 10:1
            Preview truncated at 32 of 512 bytes
```

10.6.2 Routing & Mixed Security Environments

Senders are permitted to transmit TID_LEVEL and TID_PRIORITY payloads via Unicast UDP directly to the IP address of a specific Node. Nodes shall accept properly authenticated universe payloads regardless of whether they arrive via Unicast or Multicast.

Operating in Unicast is discouraged. Unicast universe delivery prevents passive snooping by other Devices, creates asymmetric behaviour in multi-Sender merge scenarios (other Senders cannot detect that a universe is actively being driven), and eliminates the bandwidth efficiency advantage of multicast for multi-Node deployments sharing a universe.

If a network contains a mix of Secure Nodes and Open Mode Nodes patched to the same universe, a Secure Sender transmitting a Security-Mode 0x00 payload will not be decoded by the Open Nodes (as required by Section 8.6). To support mixed environments, Senders may optionally dual-cast their TID_LEVEL payloads, transmitting one packet with Security-Mode 0x00 and a duplicate packet with Security-Mode 0x01.

10.6.3 Transmission Rates and Keep-Alive

Senders shall adhere to the following transmission rate rules for TID_LEVEL & TID_PRIORITY payloads:

- Maximum Rate: A Sender shall default to transmitting TID_LEVEL & TID_PRIORITY payloads at a maximum rate of 44 frames per second. Senders may be configured by an operator to transmit at higher framerates (e.g. 60Hz or 120Hz for high-speed SPI pixel mapping).
    - Informative Note on Speed Governance: Because pure Senders do not perform node discovery, they cannot natively query the TID_EP_REFRESH_CAPABILITY of the receiving Nodes. If a Sender transmits a universe faster than a subscribed Node can physically process it, the Node will silently drop the excess frames. Management software should ideally warn operators if a Sender's output rate exceeds the capabilities of the slowest Node patched to that universe.
- Dynamic Data: When slot values are actively changing, the Sender shall transmit packets as the changes occur, subject to the maximum rate.
- Transition to Idle: When data stops changing, the Sender shall transmit three identical packets containing the final state at the active frame rate before reducing its transmission speed. (Note: Per Section 8.5, the CoAP Sig-Net-Seq-Num and resulting HMAC will still increment uniquely for each of these three packets).
- Idle Data: While data remains unchanged, the Sender shall transmit a keep-alive packet at a minimum refresh rate of 1 Hz. This keep-alive shall be a standard TID_LEVEL packet containing the full current slot data for the universe. This ensures that Nodes joining the network or recovering from a reboot can synchronise to the current lighting state during idle periods.

Informative note: Sig-Net is intended to be easily integrated with sACN and Art-Net code bases. In the absence of normative instruction, Sig-Net equivalent packets can be sent and received with the same timings and expiration logic as sACN and Art-Net.

10.6.4 Stream Loss Timeout:

If a Node does not receive a valid TID_LEVEL packet for a patched universe from an active Sender within <universe_lost_timeout> consecutive seconds, the Node shall assume the stream is lost. The Node shall then execute its configured data-loss behaviour.

If the endpoint is configured via TID_EP_PROTOCOL to consume level data from an alternative protocol (e.g. Art-Net), the Node shall evaluate Stream Loss based on the expiration logic defined by that specific protocol.

10.7 Sync Payload

Sig-Net implements a global synchronisation mechanism that allows multiple universe to be output in synchronism and so eliminate ‘tearing’ problems in visual effects.

The sync command uses the TID_SYNC Type Identifier which contains no payload bytes.

10.7.1 Transmission Rules

A Sender generating synchronised data shall transmit all necessary TID_LEVEL and TID_PRIORITY packets to their respective universe URIs.

The Sender shall introduce a minimum delay of 5 milliseconds after transmitting the final TID_LEVEL packet of the frame to allow receiving Nodes sufficient time to process network buffers. After this delay, the Sender shall transmit a single TID_SYNC packet to the global sync URI (/sig-net/<version>/<scope>/sync).

This packet shall be authenticated using the Sender Key (Ks).

10.7.2 Processing Rules and the SYNC_ACTIVE State

A Node shall maintain a SYNC_ACTIVE boolean state flag for each of its data endpoints. Upon boot, the SYNC_ACTIVE flag for all endpoints shall default to False.

When SYNC_ACTIVE is False (Asynchronous Mode):

1. When a Node receives a valid TID_LEVEL packet, it shall immediately apply the new data to the active output of the corresponding endpoint.
2. If the Node receives a TID_SYNC packet, it shall extract the 8-byte Sig-Net-Sender-ID from the CoAP header. The Node shall evaluate all active data endpoints. If an endpoint is currently receiving its active TID_LEVEL data from a Sender matching that Sig-Net-Sender-ID (as determined by the priority merge rules in Section 10.6), the Node shall transition the SYNC_ACTIVE flag for that specific endpoint to True, and reset its internal Sync Timeout timer. Endpoints receiving active data from other Senders shall ignore the TID_SYNC packet.

When SYNC_ACTIVE is True (Synchronised Mode):

1. When a Node receives a valid TID_LEVEL packet from the active Sender, it shall store the level data in an internal holding buffer for that endpoint. It shall not immediately apply this new data to the active output.
2. When the Node receives a TID_SYNC packet matching the Sig-Net-Sender-ID of the active Sender currently driving the endpoint, it shall immediately transfer the contents of the holding buffer to the active output, executing the new levels simultaneously. The Node shall then reset its internal Sync Timeout timer for that endpoint.

10.7.3 Synchronisation Timeout

When an endpoint is in the SYNC_ACTIVE state, the Node shall continuously monitor the time elapsed since the last valid TID_SYNC packet was received.

If a TID_SYNC packet is not received within <sync_lost_timeout> milliseconds of the previous TID_SYNC packet, the Node shall assume the synchronised Sender has gone offline or dropped the stream. The Node shall:

1. Immediately transition the SYNC_ACTIVE flag for that endpoint back to False.
2. Automatically flush any data currently held in the holding buffer to the active output.

10.8 Timecode Payload

Sig-Net provides a lightweight payload for transmitting timecode to support show synchronisation. The data structure is based on the MIDI Timecode (MTC) format.

Timecode data shall be transmitted to the following URI:

/sig-net/<version>/<scope>/timecode/{stream}

The {stream} variable allows multiple independent timecode tracks to coexist on the network and shall be an integer in the range of 1 to 255.

The Type Identifier (TID) TID_TIMECODE shall be used for timecode transmission:

10.8.1 Transmission Rules

A Sender generating timecode shall transmit a TID_TIMECODE block once per frame to maintain synchronisation across the network. Because timecode is a continuous, high-frequency stream, responders shall use the Sig-Net-Seq-Num option (Section 8.3) to ignore any out-of-order or delayed packets

Idle/Paused Transmission: If the timecode source is paused or stopped, the Sender shall continue to transmit the frozen, non-advancing timecode value as a keep-alive at a rate of 1 Hz. This prevents receiving devices from prematurely declaring a stream loss.

Duplicate Frame Handling: Implementations consuming TID_TIMECODE shall be robust against receiving identical timecode values in rapid succession (e.g. due to Sender software stutter or idle keep-alive) and shall process them gracefully.

10.8.2 Timecode Stream Loss

A device consuming timecode shall consider a stream lost if no TID_TIMECODE packet has been received for the relevant {stream} within <timecode_lost_timeout> seconds. Upon stream loss, the consuming device shall:

- Hold the last received timecode position and cease advancing.
- Resume normal synchronisation immediately upon receipt of the next valid TID_TIMECODE packet for that stream, without requiring a restart or re-synchronisation procedure.

Implementations shall not assume that timecode resumes from the position at which the stream was lost. The frame value in the next received packet shall be treated as the authoritative current position regardless of the elapsed gap.

10.9 Security Event Reporting

This section defines the behaviour of TID_DG_SECURITY_EVENT for diagnostic security monitoring. Sig-Net Nodes shall implement security event tracking using TID_DG_SECURITY_EVENT.

10.9.1 Event Counters

Nodes shall maintain a 32-bit unsigned integer counter for each security Event Code defined in Section 11.8.1. These counters should be stored in volatile memory (RAM) only.

10.9.2 Offending Address Capture

For each event code, the Node shall capture the IP address of the most recent packet that triggered the event. Nodes are not required to maintain a history or list of offending addresses; capturing the single most recent address is sufficient to minimise memory overhead.

10.9.3 Transmission and Rate Limiting

A Node shall unilaterally multicast a TID_DG_SECURITY_EVENT to the Reply URI upon detecting a qualifying security event.

Rate Limit: To prevent network flooding during a sustained attack, transmission of this TID shall be rate-limited to a maximum of one packet per second per Event Code.

Accuracy: The "Event Counter" in the payload shall reflect the actual total number of events detected, allowing a Manager to observe the full rate of an attack even when the network reports are being suppressed by the rate-limiter.

11 TID Definitions

The following section defines the Type Identifiers and parameters used in all Sig-Net TLVs.

- Length Column: Defines the length of the Value field. The total TLV size on the wire is Length + 4 bytes (2-byte TID + 2-byte Length field).
- Mandated Column: Specifies whether a Node is legally required to support the TID. If a TID is marked "Mandated: Yes," the Node shall accurately report its state when queried. However, if the Node's physical hardware does not support altering that specific parameter (e.g. a hardwired DMX Output port receiving a TID_EP_DIRECTION SET command), the Node shall silently ignore the invalid parameter update while continuing to accurately report its fixed state.
- Proprietary Range: TIDs 0x8000 to 0xFFFF are reserved for Manufacturer-specific use (See Section 10.1.2).
- URI Routing: All URIs listed in the 'Sent by | to URI' columns are relative to the /sig-net/<version>/<scope> prefix defined in Section 8.2.

11.1 Node-Discovery Type Identifiers

11.1.1 TID_POLL

Description: Requests node-discovery replies or specific configuration state updates from nodes on the network.			
Sent by | to URI	Enum	Length	Value
Manager /poll	0x0001	25	[0-5] MANAGER_TUID: The 48-bit TUID of the Manager originating the poll. [6-9] MANAGER_SOEM_CODE: The 32-bit SoemCode of the Manager originating the poll. [10-15] TUID_LO [16-21] TUID_HI [22-23] END_POINT. 0xFFFF = All endpoints [24] QUERY_LEVEL. See Section 11.9
Target Endpoint: Root & Data			Queryable: No
Notes			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.1.2 TID_POLL_REPLY

Description: Reports a node's presence, TUID, product code, and current configuration change count.			
Sent by | to URI	Enum	Length	Value
Node /node/{tuid}/{endpoint}	0x0002	12	[0-5] TUID [6-9] SOEM_CODE [10-11] CHANGE_COUNT. A 16-bit wrap around counter that shall increment by 1 when any Native Configuration state (TIDs) on a node changes. It shall not increment when encapsulated RDM parameter states change.
Target Endpoint: Root & Data			Queryable: No
Note.			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.2 Sender Type Identifiers

11.2.1 TID_LEVEL

Description: Transmits up to 512 bytes of standard DMX slot level data.			
Sent by | to URI	Enum	Length	Value
Sender /level/{universe}	0x0101	1-512	Up to 512 bytes of level data.
Target Endpoint: Data			Queryable: No
Notes: This carries zero start-code only.			
Manager Mandated: No.			Node Mandated: Yes if Data Endpoints supported.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.2.2 TID_PRIORITY

Description: Transmits per-slot priority values to govern network merging behaviour.			
Sent by | to URI	Enum	Length	Value
Sender /level/{universe}	0x0102	1-512	Up to 512 bytes of priority data
Target Endpoint: Data			Queryable: No
Notes: If the payload length is 1, the single byte value shall be applied uniformly to all 512 data slots, functioning as a Universe Priority. If the length is > 1, the bytes map sequentially to the slots starting at data slot 1. Any remaining unaddressed slots (up to 512) shall be set to the default value of 100.			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: Yes.

11.2.3 TID_PREVIEW

Description: Transmits up to 512 bytes of standard DMX slot level data intended for offline visualisation.			
Sent by | to URI	Enum	Length	Value
Sender /preview/{universe}	0x0103	1-512	Up to 512 bytes of level data.
Target Endpoint: Data			Queryable: No
Notes: This carries zero start-code only. Senders transmitting Preview data shall direct it exclusively to the <mult_preview> multicast address to prevent unnecessary discard load on physical Nodes.			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: Yes.

11.2.4 TID_SYNC

Description: Triggers all receiving nodes to output their buffers.			
Sent by | to URI	Enum	Length	Value
Sender /sync	0x0201	0	
Target Endpoint: Data			Queryable: No
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.2.5 TID_TIMECODE

Description: Transmits a single frame of show synchronisation timecode.			
Sent by | to URI	Enum	Length	Value
Sender /timecode/{stream}	0x0202	5	[0]: Hours (Valid range: 0–23) [1]: Minutes (Valid range: 0–59)

			[2]: Seconds (Valid range: 0–59) [3]: Frames (Valid range: 0–29, depending on the Type byte) [4]: Type (Defines the frame rate) • 0x00 = 24 fps (Film) • 0x01 = 25 fps (EBU) • 0x02 = 29.97 fps (Drop Frame) • 0x03 = 30 fps (SMPTE Non-Drop) • 0x04 = 48 fps • 0x05 = 50 fps • 0x06 = 59.94 fps (Drop Frame) • 0x07 = 60 fps (Non-Drop) • 0x08 = 100 fps • 0x09 = 119.88 fps (Drop Frame) • 0x0A = 120 fps (Non-Drop) • 0x0B - 0xFF = Reserved
Target Endpoint: Data			Queryable: No
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.2.6 TID_UNIVERSE

Description: Proactively reports the active universes currently being transmitted by this Sender, allowing passive listeners (such as Visualisers) to dynamically manage their IGMP multicast subscriptions.			
Sent by | to URI	Enum	Length	Value
Sender /sync	0x0203	7	[0-1] Universe: A 16-bit unsigned integer defining the Sig-Net universe (Valid range 1 to 63,999). [2] Command: • 0x01 (Join): The Sender is actively transmitting TID_LEVEL data for this universe. • 0x02 (Leave): The Sender has ceased transmitting data for this universe. • 0x00, 0x03-0xFF: Reserved. [3-6] Multicast IPv4 Address: The specific IP address the Sender is transmitting this universe to. If the Sender is using the default Multicast Folding algorithm (Section 9.2.3), this value shall be 0.0.0.0, instructing the receiver to calculate the default IP address mathematically.
Target Endpoint: Data			Queryable: No
Notes: Senders shall proactively multicast this TID to the global sync URI when transmission of a universe begins (Join) or gracefully terminates (Leave). To ensure newly connected listeners discover active streams, Senders shall periodically re-transmit the Join payloads for all active universes at a background rate of <universe_announce_interval> seconds. Multiple TID_UNIVERSE payloads should be packed efficiently into a single CoAP packet.			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.3 RDM Type Identifiers

11.3.1 TID_RDM_COMMAND

Encapsulates a standard E1.20 RDM request packet originating from a Manager.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint}	0x0301	26-257	RDM command
Target Endpoint: Data			Queryable: No
Notes:			
Manager Mandated: No.			Node Mandated: Yes if Endpoint supports RDM.
Sender Mandated: No.			Visualiser Mandated: No.

11.3.2 TID_RDM_RESPONSE

Encapsulates an E1.20 RDM response or a proactive state-change notification originating from a Node.			
Sent by | to URI	Enum	Length	Value
Node /node/{tuid}/{endpoint}	0x0302	26-257	RDM Response.
Target Endpoint: Data			Queryable: No
Notes:			
Manager Mandated: No.			Node Mandated: Yes if Endpoint supports RDM.
Sender Mandated: No.			Visualiser Mandated: No.

11.3.3 TID_RDM_TOD_CONTROL

Commands a Node to force an immediate RDM discovery cycle or flush its current Table of Devices.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint}	0x0303	1	[0] Command Enum: 0x00: Force Node to Send TID_RDM_TOD_DATA 0x01: Flush ToD and Force Full Discovery
Target Endpoint: Data			Queryable: No
Notes:			
Manager Mandated: No.			Node Mandated: Yes if Endpoint supports RDM.
Sender Mandated: No.			Visualiser Mandated: No.

11.3.4 TID_RDM_TOD_DATA

Description: Reports the array of RDM responder UIDs currently discovered on the specified endpoint.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0304	0 /Variable, 2+ multiples of 6	[0] Packet_Index: An 8-bit unsigned integer defining the sequence number of this specific payload (ranging from 1 to Total_Packets). [1] Total_Packets: An 8-bit unsigned integer defining the total number of TID_RDM_TOD_DATA payloads required to transmit the complete Table of Devices for this endpoint. [2-Length] UID_Array: An array of 48-bit (6-byte) RDM UIDs representing all currently discovered RDM Responders on this endpoint included in this packet block. (If no RDM responders are discovered on the endpoint, Total_Packets shall be 1, Packet_Index shall be 1, and the UID_Array shall be empty).
Target Endpoint: Data			Queryable: Yes

Notes: To prevent UDP fragmentation, Nodes SHALL ensure the UID_Array within a single payload does not cause the CoAP packet to exceed the 1,400-byte limit (Section 10.2.4). For large discovery tables, the Node SHALL sequentially transmit multiple TID_RDM_TOD_DATA payloads, incrementing the Packet_Index for each block.	
Manager Mandated: No.	Node Mandated: Yes if Endpoint supports RDM.
Sender Mandated: No.	Visualiser Mandated: No.

11.3.5 TID_RDM_TOD_BACKGROUND

Description: Enables or disables the continuous background RDM discovery process on the specified endpoint.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0305	0/1	[0] Command Enum: 0x00: Disable Background RDM Discovery 0x01: Enable Background RDM Discovery
Target Endpoint: Data			Queryable: Yes
Notes:			
Mandated: Support for this TID is mandated if the Endpoint supports RDM.			
Manager Mandated: No.			Node Mandated: Yes if Endpoint supports RDM.
Sender Mandated: No.			Visualiser Mandated: No.

11.3.6 TID_RDM_FLOW_CONTROL

Description: Reports the real-time capacity of the endpoint's internal RDM command queue (FiFo buffer), allowing Managers to dynamically burst RDM traffic without overflowing the DMX512 UART.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0306	0/2	[0] Total RDM FiFo Buffers: An 8-bit unsigned integer defining the maximum number of RDM commands this Endpoint can queue simultaneously. [1] Available RDM FiFo Buffers: An 8-bit unsigned integer defining how many slots in the queue are currently empty and ready to accept new commands.
Target Endpoint: Data			Queryable: Yes
Notes: A Node shall proactively multicast this TID to its Reply URI whenever its Available FiFo Buffer count changes. Nodes should append this TLV to any outbound TID_RDM_RESPONSE when possible. Managers shall respect the Available buffer count to prevent command loss.			
Manager Mandated: No.			Node Mandated: Yes if Endpoint supports RDM.
Sender Mandated: No.			Visualiser Mandated: No.

11.4 Offboarding Type Identifiers

The Type Identifiers in this section shall only be used with the root endpoint (0).

11.4.1 TID_RT_OFFBOARD

Commands a Node to permanently delete its cryptographic keys and return to an offboarded state.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0	0x0401	4	[0-3] Magic Word 0x57495045 (ASCII "WIPE").
Target Endpoint: Root			Queryable: No
Notes: Instructs the Node to permanently delete all cryptographic keys, reset session counters, and return to the Offboarded state (Beacon Mode). Payload values other than 0x57495045 shall be ignored. Nodes only accept the TID_RT_OFFBOARD command within the first <offboard_lockout> seconds after the Node physically powers on.			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5 Network Configuration Type Identifiers

The Type Identifiers in this section are used to configure the fundamental IP network stack of the device. They shall only be used with the root endpoint (0). Support for these parameters is optional, but strongly recommended.

11.5.1 TID_NW_MAC_ADDRESS

Description: Reports the physical MAC address of the network interface.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0501	0/6	[0-5] The 48-bit MAC address of the device.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.2 TID_NW_IPV4_MODE

Description: Sets or reports the IPv4 address allocation mode.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0502	0/1	[0] Mode: 0x00 = Static. 0x01 = DHCP.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.3 TID_NW_IPV4_ADDRESS

Description: Sets or reports the configured Static IPv4 Address.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/{endpoint}	0x0503	0/4	[0-3] The 32-bit static IPv4 Address.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.4 TID_NW_IPV4_NETMASK

Description: Sets or reports the configured Static IPv4 Subnet Mask.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0504	0/4	[0-3] The 32-bit static IPv4 Subnet Mask.
Target Endpoint: Root			Queryable: Yes

Notes:	
Manager Mandated: No.	Node Mandated: No.
Sender Mandated: No.	Visualiser Mandated: No.

11.5.5 TID_NW_IPV4_GATEWAY

Description: Sets or reports the configured Static IPv4 Default Gateway.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0505	0/4	[0-3] The 32-bit static IPv4 Default Gateway.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.6 TID_NW_IPV4_CURRENT

Description: Reports the currently active IPv4 network settings, regardless of whether they were set statically or obtained via DHCP.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0506	0/12	[0-3] Currently active IPv4 Address. [4-7] Currently active IPv4 Subnet Mask. [8-11] Currently active IPv4 Default Gateway. All values are 32-bit IPv4 addresses in network byte order. These reflect the operational values regardless of whether they were set statically or assigned by DHCP.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.7 TID_NW_IPV6_MODE

Description: Sets or reports the IPv6 address allocation mode.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0581	0/1	[0] Mode: 0x00 = Static. 0x01 = SLAAC. 0x02 = DHCPv6.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.8 TID_NW_IPV6_ADDRESS

Description: Sets or reports the configured Static IPv6 Address.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node	0x0582	0/16	[0-15] The 128-bit static IPv6 Address.

/node/{tuid}/0			
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.9 TID_NW_IPV6_PREFIX

Description: Sets or reports the configured Static IPv6 Routing Prefix Length.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0583	0/1	[0] The prefix length (Valid range 0-128).
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.10 TID_NW_IPV6_GATEWAY

Description: Sets or reports the configured Static IPv6 Default Gateway.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0584	0/16	[0-15] The 128-bit static IPv6 Default Gateway.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.5.11 TID_NW_IPV6_CURRENT

Description: Reports the currently active IPv6 network settings, regardless of how they were assigned.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0585	0/33	[0-15] Active IPv6 Address. [16] Active Prefix Length. [17-32] Active Default Gateway.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.6 Root Endpoint Type Identifiers

The Type Identifiers in this section shall only be used with the root endpoint (0).

11.6.1 TID_RT_SUPPORTED_TIDS

Description: Reports the complete array of Type Identifiers (TIDs) supported by the Sig-Net device.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0601	0/Variable, multiples of 2	An array of 16-bit (2-byte) Type Identifiers (TID) supported by the Node. Reports the complete array of Type Identifiers (TIDs) supported by this device. The list shall include all supported TIDs regardless of whether they are mandated, optional or proprietary.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.2 TID_RT_ENDPOINT_COUNT

Description: Reports the total number of physical or logical data endpoints available on the device.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0602	0/2	[0-1] Total number of data endpoints (Endpoints 1 to N) on this device. Does not include Endpoint 0. A value of 0x0000 indicates a device with no data endpoints.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.3 TID_RT_PROTOCOL_VERSION

Description: Reports the Sig-Net protocol version supported on this device.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0603	0/1	[0] Protocol Version. An unsigned integer representing the major architectural version of the Sig-Net specification supported by the Node. This integer corresponds directly to the <version> string required in the CoAP URI-Path (e.g. a value of 0x01 corresponds to the URI string /v1/).
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.4 TID_RT_FIRMWARE_VERSION

Description: Reports the firmware version of this device.			
Sent by | to URI	Enum	Length	Value

Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0604	0/4-68	[0-3] Machine Version ID: A 32-bit unsigned integer representing the manufacturer's firmware version. This value shall follow the convention defined in [RDM]. [4-67] A UTF-8 encoded human-readable firmware version string (e.g. "v1.2.4b"). Maximum 64 bytes. Not null-terminated.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.5 TID_RT_DEVICE_LABEL

Description: Sets or reports the human-readable text label representing the entire physical device.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0605	0/1-64	UTF-8 encoded human-readable device label. Maximum 64 bytes. Not null-terminated. A Node shall store this label in non-volatile memory. To clear the label, a Manager shall transmit a Length of 1 containing a single Null character (0x00).
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.6 TID_RT_MULT_OVERRIDE

Description: Resets or reports whether the device's endpoints are utilising custom multicast routing overrides.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0606	0/1	[0] Global Multicast Routing State: 0x00: Default. All Data Endpoints on this Node are exclusively using the default folded mapping (<mult_u1> to <mult_u109>). 0x01: Custom. One or more Data Endpoints on this Node are patched to a custom Multicast IP.
Target Endpoint: Root			Queryable: Yes
Notes: When sent by manager, only the payload value of 0x00 is valid and shall return all multicast addresses to their default value.			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.7 TID_RT_IDENTIFY

Description: Commands the device to visually or audibly identify itself to an operator.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0607	0/1	[0] Identify State: 0x00: Identify off. Normal operation. 0x01: Identify Subtle. Causes the device to identify itself in a subtle way that could be used in show conditions. 0x02: Identify Full. Causes the device to identify itself in an obvious way.

			0x03: Mutes all indicators and backlights for dark-sky operation.
Target Endpoint: Root			Queryable: Yes
Notes: The Identify state shall be stored in volatile memory (RAM) only and shall default to 0x00 (Off) upon device initialisation or power cycle. It shall not persist across reboots.			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.8 TID_RT_STATUS

Description: Reports a bitfield detailing the global hardware health and configuration lock status of the device.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0608	0/4	[0-3] A 32-bit bitfield representing global device health. Bit 0: Hardware Fault. Bit 1: Booted from factory defaults. Bit 2: Configuration locked via local UI. Bit 3: Operating in Open Mode (Unauthenticated). Bits 4-31: Reserved.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.9 TID_RT_ROLE_CAPABILITY

Description: Reports the functional Sig-Net roles embedded within the device.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x0609	0/1	[0] An 8-bit bitfield supported roles. Bit 0: Node Role Supported. Bit 1: Sender Role Supported. Bit 2: Manager Role Supported. Bit 3: Visualiser Role Supported. Bits 4-6: Reserved. Bit 7: Open Mode supported.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.10 TID_RT_REBOOT

Description: Commands the device to perform a reboot.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0	0x060A	5	[0] An 8-bit command representing reboot type: 0xFF: Hardware Reset. 0xFE: Warm Reboot. 0x00-0xFD: Not used. [1-4] Magic Word “BOOT”
Target Endpoint: Root			Queryable: No

Notes: Instructs the device to immediately cease network operations, gracefully close any open hardware states if possible, and perform the requested reboot. The command shall be ignored if the Magic Word does not match.	
Manager Mandated: No.	Node Mandated: No.
Sender Mandated: No.	Visualiser Mandated: No.

11.6.11 TID_RT_MODEL_NAME

Description: Sets or reports the human-readable text label representing the entire physical device.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0x060B	0/1-64	[0-63] A UTF-8 encoded human-readable product model string. Maximum 64 bytes. Not null-terminated.
Target Endpoint: Root			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.6.12 TID_RT_OTW_CAPABILITY

Description: Reports the network port and cryptographic protocols supported by the device for Over-The-Wire (OTW) onboarding and key rotation.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0	0x060D	0/3	[0-1] OTW Listener Port: A 16-bit unsigned integer defining the TCP or UDP port on which the device is actively listening for a secure onboarding handshake. [2] Supported Protocols Bitfield: • Bit 0 (0x01): DTLS 1.2 Supported • Bit 1 (0x02): DTLS 1.3 Supported • Bit 2 (0x04): TLS 1.2 Supported (TCP) • Bit 3 (0x08): TLS 1.3 Supported (TCP) • Bits 4-7: Reserved.
Target Endpoint: Root			Queryable: Yes
Notes: Devices that do not support OTW onboarding shall not support this TID.			
Manager Mandated: Yes if OTW supported.			Node Mandated: Yes if OTW supported.
Sender Mandated: Yes if OTW supported.			Visualiser Mandated: Yes if OTW supported.

11.7 Data Endpoint Type Identifiers

The Type Identifiers in this section shall only be used with the data endpoint (1-n).

11.7.1 TID_EP_UNIVERSE

Description: Sets or reports the operational Sig-Net universe assigned to a specific endpoint.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0901	0/2	[0-1] UNIVERSE_VALUE. Valid range 1 to 63,999 and 0 if not set.
Target Endpoint: Data			Queryable: Yes
Notes:			
Mandated: Support for this TID is mandated for Nodes that support Data Endpoints.			
Manager Mandated: Yes.			Node Mandated: Yes if Data Endpoints supported.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.2 TID_EP_LABEL

Description: Sets or reports the human-readable text label assigned to a specific endpoint.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0902	0/1-64	UTF-8 encoded human-readable endpoint label. Maximum 64 bytes. Not null-terminated. A Node shall store this label in non-volatile memory. To clear the label, a Manager shall transmit a Length of 1 containing a single Null character (0x00).
Target Endpoint: Data			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes if Data Endpoints supported.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.3 TID_EP_MULT_OVERRIDE

Description: Applies or clears a custom, non-standard multicast IP address for a specific endpoint.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0903	0/4	[0-3] Multicast IPv4 Address: The specific custom multicast IP this endpoint shall use. A value of 0.0.0.0 (all zero bytes) acts as a clear command, instructing the Node to disable the override and use the default folded pool based on its TID_EP_UNIVERSE setting.
Target Endpoint: Data			Queryable: Yes
Notes:			
Mandated: Support for this TID is mandated for Nodes that support Multicast IP Override.			
Manager Mandated: Yes.			Node Mandated: Yes if Multicast IP override supported.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.4 TID_EP_CAPABILITY

Description: Reports the fixed hardware and logical capabilities of the Data Endpoint.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node	0x0904	0/1	[0] Port Capability bit field: • Bit 0 (0x01): Can Consume TID_LEVEL (Outputs DMX or consumes internally).

/node/{tuid}/{endpoint}			• Bit 1 (0x02): Can Supply TID_LEVEL (Receives DMX, transmits Sig-Net). • Bit 2 (0x04): Can Consume RDM (Proxies or terminates TID_RDM_COMMAND). • Bit 3 (0x08): Can Supply RDM (Receives RDM, transmits Sig-Net). • Bit 4 (0x10): Virtual Endpoint (This Endpoint terminates inside the device and hosts an internal RDM Responder; it is not a physical DMX port). • Bits 5-7: Reserved.
Target Endpoint: Data			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes if Data Endpoints supported.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.5 TID_EP_DIRECTION

Sets or reports whether the endpoint port is actively operating as an input, an output, or is disabled.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0905	0/1	[0] Port Direction and Feature Bitfield: Bits 0-1 (Direction): • 00 (0): Disabled (Port is inactive). • 01 (1): Consumer (Receives Sig-Net, outputs physical DMX). • 10 (2): Supplier (Receives physical DMX, transmits Sig-Net). • 11 (3): Fallback (Port actively monitors the DMX line as an input; if no valid DMX signal is detected, the port automatically transitions to a Consumer output to drive a severed DMX loop topology). Bit 2 (RDM Enable): • 0: RDM processing disabled on this port. • 1: RDM processing enabled on this port (subject to TID_EP_CAPABILITY). Bits 3-7: Reserved.
Target Endpoint: Data			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes if Data Endpoints supported.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.6 TID_EP_INPUT_PRIORITY

Description: Sets or reports the per-slot priority values that an input port will assign to incoming DMX data.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0906	0/1-512	[0-511] Defines the [ETC 0xDD] Priority value (0-200) this endpoint will assign to its network transmissions when TID_EP_DIRECTION is set to Input

Target Endpoint: Data	Queryable: Yes
Notes: If payload length is 1, the single byte value shall be applied to all 512 priority slots. If length > 1, the bytes map sequentially to the priority slots starting at slot 1. Any remaining unaddressed priority slots (up to 512) shall be set to 0.	
Manager Mandated: No.	Node Mandated: No.
Sender Mandated: No.	Visualiser Mandated: No.

11.7.7 TID_EP_STATUS

Reports a live status bitfield detailing the current data activity and fault state of the endpoint port.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0907	0/4	[0-5] A 32-bit bitfield representing endpoint health and status. Bit 0: This endpoint is actively transmitting or receiving. Bit 1: Hardware fault. Bit 2: Configuration locked via local UI. Bit 3: This endpoint is receiving TID_LEVEL. Bit 4: This endpoint is receiving more than one stream of TID_LEVEL. Bit 5: This endpoint is in Fallback mode. Bit 6: This endpoint is in Failover mode. Bits 7-31: Reserved. [0-3] Bit field:
Target Endpoint: Data			Queryable: Yes
Notes:			
Manager Mandated: Yes.			Node Mandated: Yes if Data Endpoints supported.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.8 TID_EP_FAILOVER

Description: Sets or reports the data-loss failover behaviour for a specific data endpoint. Defines the physical output state if the network stream is lost (as defined by <universe_lost_timeout>).			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0908	0/3	[0] Failover Mode: • 0x00: Hold Last State. • 0x01: Blackout (All slots to 0). • 0x02: Full (All slots to 255). • 0x03: Play Scene (Execute internal preset/scene). • 0x04: Stop generating DMX. • 0x05-0xFF: Reserved. [1-2] Scene Number: A 16-bit unsigned integer defining the internal preset or scene to execute when Mode is 0x03. Valid range is 1 to 65,535. (If Mode is not 0x03, these bytes shall be ignored by the Node but must still be transmitted to maintain the 3-byte payload length).
Target Endpoint: Data			Queryable: Yes
Notes: Nodes that do not support internal scenes shall silently reject SET commands specifying Mode 0x03.			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.9 TID_EP_DMX_TIMING

Description: Sets or reports the physical DMX512 transmission timing and update mode for a specific output port.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x0909	0/2	[0] Transmission Mode: • 0x00: Continuous (The port continuously transmits DMX frames at the configured framerate, regardless of whether slot data has changed). • 0x01: Delta / Change-Only (The port only transmits DMX frames when the underlying Sig-Net slot data changes, or to satisfy the 1 Hz keep-alive rule). • 0x02-0xFF: Reserved. [1] Output Timing: • 0x00: Maximum DMX512 timing and refresh rate. • 0x01: Medium DMX512 timing and refresh rate. • 0x02: Minimum DMX512 timing and refresh rate.
Target Endpoint: Data			Queryable: Yes
Notes: Nodes that only support fixed continuous output shall silently ignore SET commands attempting to change the Transmission Mode or Framerate, but must accurately report their fixed state if queried.			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.7.10 TID_EP_REFRESH_CAPABILITY

Description: Reports the maximum physical refresh rate (in frames per second) that the endpoint's hardware can output.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x090A	0/1	[0] An 8-bit unsigned integer representing the maximum supported framerate. • 0 - 44: The endpoint supports the standard DMX maximum of 44Hz. (Nodes SHALL NOT declare a capability lower than 44Hz). • 45 - 250: The endpoint supports high-speed output (e.g. for SPI LED tape) up to the specified framerate. • 251 - 255: Reserved.
Target Endpoint: Data			Queryable: Yes
Notes: This is a read-only hardware capability report. It does not configure the network stream rate. Managers may use this value to warn operators if a Sender is configured to transmit a universe faster than the consuming Node can physically output.			
Manager Mandated: No.			Node Mandated: No. (Defaults to 44Hz if unsupported).
Sender Mandated: No.			Visualiser Mandated: No.

11.7.11 TID_EP_PROTOCOL

Description: Sets or reports the active network protocol from which this endpoint consumes its lighting level data.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0x090B	0/1	[0] Enumerated value: 0x00: Sig-Net (Default). 0x01: Art-Net 4. 0x02: sACN. 0x03-0xff: Reserved.
Target Endpoint: Data			Queryable: Yes
Notes: This TID determines which protocol stream the endpoint actively forwards to its physical/virtual DMX output. Regardless of the active protocol selected for level data, the Node shall maintain its active Sig-Net Control Plane and process authenticated Sig-Net Management commands. Switching an endpoint to a legacy protocol removes cryptographic authentication from the control stream, Managers shall warn operators and require confirmation before applying such a configuration change to a Secure Node.			
Manager Mandated: Yes.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.8 Diagnostic Endpoint Type Identifiers

The Type Identifiers in this section are used for diagnostics and testing.

11.8.1 TID_DG_SECURITY_EVENT

Description: Reports security events.			
Sent by | to URI	Enum	Length	Value
Manager /manager/{tuid}/0 Node /node/{tuid}/0	0xFF01	0/11-23	[0-1] Event Code: 0x0001: HMAC Verification Failure. 0x0002: Replay Attack Detected (Sequence/Session ID anomaly). 0x0003: DoS Rate-Limiting Activated. 0x0004: Unauthorised Onboarding Attempt. [2-5] Event Counter: A 32-bit unsigned integer representing the total number of times this event has occurred since the device initialized (RAM-based). [6] Address Type: 0x00=None, 0x01=IPv4, 0x02=IPv6. [7-n] Source IPv6 Address: The IP address of the most recent packet that triggered this event. (4 bytes if IPv4, 16 bytes if IPv6).
Target Endpoint: Both			Queryable: Yes
Note: A Node shall unilaterally multicast this TID upon detecting a qualifying security event. To prevent network flooding during a sustained attack, Nodes shall aggressively rate-limit the transmission of this TID (recommended maximum: 1 packet per second per Event Code). When a Manager actively queries this TID (Length = 0), the Node shall respond by sequentially packing multiple TID_DG_SECURITY_EVENT TLVs into the reply packet, providing one TLV for each Event Code it actively tracks. (Informative Note for Managers: Because UDP source IP addresses are trivially spoofed, the IP address reported in this payload should be treated as a diagnostic indicator rather than absolute proof of an attacker's location).			
Manager Mandated: Yes.			Node Mandated: Yes.
Sender Mandated: Yes.			Visualiser Mandated: Yes.

11.8.2 TID_DG_MESSAGE

Description: Reports the human-readable diagnostic message.			
Sent by | to URI	Enum	Length	Value
Node /node/{tuid}/0	0xFF02	0-64	UTF-8 encoded human-readable diagnostic message. Strings shall NOT be null-terminated.
Target Endpoint: Root			Queryable: Yes
Notes: A Node proactively multicasts a diagnostic message to <mult_node_send> to alert all listening Managers of a significant internal event (e.g. hardware faults, thermal warnings, or port errors). Rate Limit: To prevent a recurring hardware or software fault from generating a multicast storm that congests the management network, Nodes SHALL aggressively rate-limit the autonomous transmission of TID_DG_MESSAGE payloads (e.g. to a maximum of one message every 5 seconds per unique fault condition).			
Manager Mandated: No.			Node Mandated: No.
Sender Mandated: No.			Visualiser Mandated: No.

11.8.3 TID_DG_LEVEL_FOLDBACK

Description: Reports the levels for a specific universe.			
Sent by | to URI	Enum	Length	Value

Manager /manager/{tuid}/{endpoint} Node /node/{tuid}/{endpoint}	0xFF03	0/1-512	[0-511] Copy of the current 512-byte DMX level buffer actively being processed or generated by this Data Endpoint.
Target Endpoint: Data			Queryable: Yes
Notes:			
Mandated: Support for this TID is mandated for Nodes that support Data Endpoints.			
Manager Mandated: No.			Node Mandated: Yes if Data Endpoints supported.
Sender Mandated: No.			Visualiser No.

11.9 TID Polling Cross Reference

This table provides a master summary of all Type Identifiers (TIDs) defined in this specification.

The QUERY_LEVEL column defines how a Node handles the TID when it receives a TID_POLL command (Section 10.4.4):

- Categories (0x00 to 0x03): The TID shall be packed into the Node's bulk poll response if the Manager requests that level or higher.
- None (Query Only): The TID is Queryable, but shall not be included in a bulk TID_POLL response. A Manager must explicitly request it by transmitting a Length = 0 header specifically for that TID.
- N/A (Not Queryable): The TID represents a write-only command, a proactive state push, or a unidirectional data stream. It cannot be queried.

TID Name	**Enum**	**Target Endpoint**	**QUERY_LEVEL**
TID_POLL_REPLY	0x0002	Root	QUERY_HEARTBEAT (0x00)
TID_RT_ENDPOINT_COUNT	0x0602	Root	QUERY_HEARTBEAT (0x00)
TID_RT_MULT_OVERRIDE	0x0606	Root	QUERY_HEARTBEAT (0x00)
TID_RDM_TOD_BACKGROUND	0x0305	Data	QUERY_CONFIG (0x01)
TID_RDM_FLOW_CONTROL	0x0306	Data	QUERY_CONFIG (0x01)
TID_RT_DEVICE_LABEL	0x0605	Root	QUERY_CONFIG (0x01)
TID_RT_IDENTIFY	0x0607	Root	QUERY_CONFIG (0x01)
TID_RT_STATUS	0x0608	Root	QUERY_CONFIG (0x01)
TID_RT_SCOPE	0x060C	Root	QUERY_CONFIG (0x01)
TID_EP_UNIVERSE	0x0901	Data	QUERY_CONFIG (0x01)
TID_EP_LABEL	0x0902	Data	QUERY_CONFIG (0x01)
TID_EP_MULT_OVERRIDE	0x0903	Data	QUERY_CONFIG (0x01)
TID_EP_CAPABILITY	0x0904	Data	QUERY_CONFIG (0x01)
TID_EP_REFRESH_CAPABILITY	0x090A	Data	QUERY_CONFIG (0x01)
TID_EP_PROTOCOL	0x090B	Data	QUERY_CONFIG (0x01)
TID_EP_DIRECTION	0x0905	Data	QUERY_CONFIG (0x01)
TID_EP_INPUT_PRIORITY	0x0906	Data	QUERY_CONFIG (0x01)
TID_EP_STATUS	0x0907	Data	QUERY_CONFIG (0x01)
TID_EP_FAILOVER	0x0908	Data	QUERY_CONFIG (0x01)
TID_EP_DMX_TIMING	0x0909	Data	QUERY_CONFIG (0x01)
TID_RT_SUPPORTED_TIDS	0x0601	Root	QUERY_FULL (0x02)
TID_RT_PROTOCOL_VERSION	0x0603	Root	QUERY_FULL (0x02)

TID Name	Enum	Target Endpoint	QUERY_LEVEL
TID_RT_FIRMWARE_VERSION	0x0604	Root	QUERY_FULL (0x02)
TID_RT_ROLE_CAPABILITY	0x0609	Root	QUERY_FULL (0x02)
TID_RT_MODEL_NAME	0x060B	Root	QUERY_FULL (0x02)
TID_RT_OTW_CABABILITY	0x060D	Root	QUERY_FULL (0x02)
TID_NW_MAC_ADDRESS	0x0501	Root	QUERY_FULL (0x02)
TID_NW_IPV4_MODE	0x0502	Root	QUERY_FULL (0x02)
TID_NW_IPV4_ADDRESS	0x0503	Root	QUERY_FULL (0x02)
TID_NW_IPV4_NETMASK	0x0504	Root	QUERY_FULL (0x02)
TID_NW_IPV4_GATEWAY	0x0505	Root	QUERY_FULL (0x02)
TID_NW_IPV4_CURRENT	0x0506	Root	QUERY_FULL (0x02)
TID_NW_IPV6_MODE	0x0581	Root	QUERY_FULL (0x02)
TID_NW_IPV6_ADDRESS	0x0582	Root	QUERY_FULL (0x02)
TID_NW_IPV6_PREFIX	0x0583	Root	QUERY_FULL (0x02)
TID_NW_IPV6_GATEWAY	0x0584	Root	QUERY_FULL (0x02)
TID_NW_IPV6_CURRENT	0x0585	Root	QUERY_FULL (0x02)
TID_DG_SECURITY_EVENT	0xFF01	Both	QUERY_EXTENDED (0x03)
TID_DG_MESSAGE	0xFF02	Both	QUERY_EXTENDED (0x03)
TID_RDM_TOD_DATA	0x0304	Root & Data	None (Query Only)
TID_DG_LEVEL_FOLDBACK	0xFF03	Data	None (Query Only)
TID_POLL	0x0001	Root & Data	N/A (Not Queryable)
TID_LEVEL	0x0101	Data	N/A (Not Queryable)
TID_PRIORITY	0x0102	Data	N/A (Not Queryable)
TID_PREVIEW	0x0103	Data	N/A (Not Queryable)
TID_SYNC	0x0201	Data	N/A (Not Queryable)
TID_TIMECODE	0x0202	Data	N/A (Not Queryable)
TID_UNIVERSE	0x0203	Data	N/A (Not Queryable)
TID_RDM_COMMAND	0x0301	Root & Data	N/A (Not Queryable)
TID_RDM_RESPONSE	0x0302	Root & Data	N/A (Not Queryable)
TID_RDM_TOD_CONTROL	0x0303	Data	N/A (Not Queryable)
TID_RT_OFFBOARD	0x0401	Root	N/A (Not Queryable)
TID_RT_REBOOT	0x060A	Root	N/A (Not Queryable)

11.10 TID Informative Summary

			Mandated			
TID Name	**Enum**	**Target Endpoint**	**Manager**	**Node**	**Sender**	**Visualiser**
TID_POLL	0x0001	Root & Data	Yes	Yes	Yes	Yes
TID_POLL_REPLY	0x0002	Root & Data	Yes	Yes	Yes	Yes
TID_LEVEL	0x0101	Data	No	Yes*	Yes	Yes
TID_PRIORITY	0x0102	Data	No	No	No	Yes
TID_PREVIEW	0x0103	Data	No	No	No	Yes
TID_SYNC	0x0201	Data	No	No	No	No
TID_TIMECODE	0x0202	Data	No	No	No	No
TID_UNIVERSE	0x0203	Data	No	No	Yes	Yes
TID_RDM_COMMAND	0x0301	Data	No	Yes*	No	No
TID_RDM_RESPONSE	0x0302	Data	No	Yes*	No	No
TID_RDM_TOD_CONTROL	0x0303	Data	No	Yes*	No	No
TID_RDM_TOD_DATA	0x0304	Data	No	Yes*	No	No
TID_RDM_TOD_BACKGROUND	0x0305	Data	No	Yes*	No	No
TID_RDM_FLOW_CONTROL	0x0306	Data	No	Yes*	No	No
TID_RT_OFFBOARD	0x0401	Root	No	No	No	No
TID_NW_MAC_ADDRESS	0x0501	Root	No	No	No	No
TID_NW_IPV4_MODE	0x0502	Root	No	No	No	No
TID_NW_IPV4_ADDRESS	0x0503	Root	No	No	No	No
TID_NW_IPV4_NETMASK	0x0504	Root	No	No	No	No
TID_NW_IPV4_GATEWAY	0x0505	Root	No	No	No	No
TID_NW_IPV4_CURRENT	0x0506	Root	No	No	No	No
TID_NW_IPV6_MODE	0x0581	Root	No	No	No	No
TID_NW_IPV6_ADDRESS	0x0582	Root	No	No	No	No
TID_NW_IPV6_PREFIX	0x0583	Root	No	No	No	No
TID_NW_IPV6_GATEWAY	0x0584	Root	No	No	No	No
TID_NW_IPV6_CURRENT	0x0585	Root	No	No	No	No
TID_RT_SUPPORTED_TIDS	0x0601	Root	Yes	Yes	Yes	Yes
TID_RT_ENDPOINT_COUNT	0x0602	Root	Yes	Yes	Yes	Yes
TID_RT_PROTOCOL_VERSION	0x0603	Root	Yes	Yes	Yes	Yes
TID_RT_FIRMWARE_VERSION	0x0604	Root	Yes	Yes	Yes	Yes
TID_RT_DEVICE_LABEL	0x0605	Root	Yes	Yes	Yes	Yes
TID_RT_MULT_OVERRIDE	0x0606	Root	Yes	Yes	Yes	Yes
TID_RT_IDENTIFY	0x0607	Root	Yes	Yes	Yes	Yes
TID_RT_STATUS	0x0608	Root	Yes	Yes	Yes	Yes
TID_RT_ROLE_CAPABILITY	0x0609	Root	Yes	Yes	Yes	Yes
TID_RT_REBOOT	0x060A	Root	No	No	No	No
TID_RT_MODEL_NAME	0x060B	Root	Yes	Yes	Yes	Yes
TID_RT_OTW_CAPABILITY	0x060D	Root	Yes*	Yes*	Yes*	Yes*
TID_EP_UNIVERSE	0x0901	Data	Yes	Yes*	No	No
TID_EP_LABEL	0x0902	Data	Yes	Yes*	No	No
TID_EP_MULT_OVERRIDE	0x0903	Data	Yes	Yes*	No	No
TID_EP_CAPABILITY	0x0904	Data	Yes	Yes*	No	No

			Mandated			
TID Name	**Enum**	**Target Endpoint**	**Manager**	**Node**	**Sender**	**Visualiser**
TID_EP_DIRECTION	0x0905	Data	Yes	Yes*	No	No
TID_EP_INPUT_PRIORITY	0x0906	Data	No	No	No	No
TID_EP_STATUS	0x0907	Data	Yes	Yes*	No	No
TID_EP_FAILOVER	0x0908	Data	No	No	No	No
TID_EP_DMX_TIMING	0x0909	Data	No	No	No	No
TID_EP_REFRESH_CAPABILITY	0x090A	Data	No	No	No	No
TID_EP_PROTOCOL	0x090B	Data	Yes	No	No	No
TID_DG_SECURITY_EVENT	0xFF01	Both	Yes	Yes	Yes	Yes
TID_DG_MESSAGE	0xFF02	Both	No	No	No	No
TID_DG_LEVEL_FOLDBACK	0xFF03	Data	No	Yes*	No	No

*Conditional – refer to detailed TID definition above.

12 Protocol Versioning and Compatibility

The Sig-Net protocol uses a Major.Minor versioning scheme.

- Major Version: Defines the core architectural framework, including the URI prefix (e.g. /v1/), Key Derivation Function (KDF) info strings, and HMAC calculation methodology. A change in the Major version represents a breaking change.
- Minor Version: Defines non-breaking extensions, such as the addition of new Type Identifiers (TIDs). A change in the Minor version does not alter the URI prefix or cryptographic foundations.

12.1 Document and URI Relationship

The Major number of this specification (e.g. V1.x) shall correspond to the version component of the CoAP URI-Path (e.g. /sig-net/v1/<scope>/...).

12.2 Node Versioning Requirements

A Node shall report its supported Major version in the TID_RT_PROTOCOL_VERSION TLV.

A Node shall inspect the version component of the URI-Path for every incoming packet. If the URI version does not match the Node's supported Major version, the Node shall silently discard the packet without attempting further parsing or cryptographic verification.

A Node shall not attempt to interpret or process packets belonging to a higher Major version than it supports.

12.3 Manager Compatibility Requirements (Normative)

To ensure system-wide interoperability in mixed-version environments, Managers shall adhere to the following:

- Version Discovery: A Manager shall be capable of discovering Nodes of its own Major version and at least one previous Major version (if applicable).
- Discovery Broadcasts: To fulfil the discovery requirement, a Manager shall independently multicast network-wide discovery requests (TID_POLL) to the Discovery URI of its native Major version (e.g. /sig-net/v2/poll) and to the Discovery URI of the supported legacy version (e.g. /sig-net/v1/poll).
- Multi-Stack Communication: When communicating with a discovered legacy Node, the Manager shall use the specific URI prefix, KDF strings, and HMAC calculation rules natively associated with that Node’s Major version for all targeted communication.

12.4 Sender Compatibility Requirements

Sender traffic (e.g. TID_LEVEL, TID_SYNC, TID_TIMECODE) is transmitted globally via multicast, Senders are not required to individually track the Major versions of the Nodes actively consuming their streams.

A Sender shall transmit its payload using its native Major protocol version and the corresponding URI prefix (e.g. /sig-net/v2/level/).

Mixed-Version Environments: In environments containing legacy Nodes, a Sender transmitting a higher Major version will not be decoded by lower-version Nodes (as required by Section 12.2). To ensure interoperability in mixed-version deployments, Senders should provide a user-configurable option to support legacy Major versions. Senders may support legacy networks by either globally downgrading their transmission version, or by dual-casting payloads to both the native and legacy URIs simultaneously (e.g. transmitting a TID_LEVEL to /v2/level/1 and /v1/level/1). When generating legacy traffic, the Sender shall use the exact URI prefix, KDF strings, and HMAC methodology defined by that legacy Major version.

13 Example Sig-Net Transaction (Informative)

This section provides guidance and flowcharts to describe how to implement Sig-Net functionality.

13.1 Sig-Net Node Discovery

A Sig-Net Device is any network device that communicates natively with Sig-Net (e.g. a moving light, dimmer rack, or DMX gateway). Node-discovery is the continuous background process by which a Manager finds all active Sig-Net Nodes on the network and tracks their configuration state.

The flowchart below demonstrates a routine polling cycle using the dedicated administrative multicast addresses.

Figure 4 Example TID Polling Cycle

```
Manager                                     Sig-Net Device
(<mult_manager_poll>)                       (Listening on <mult_manager_poll>)
            |            PHASE 1: POLL                        |
            |===============================================|
            |                                                 |
            |--- POST /poll (Multicast) ------------------->| [1]
            |    (TID_POLL: TUID_LO=0, TUID_HI=MAX,           |
            |     EP = 0xFFFF, QUERY_LEVEL=QUERY_HEARTBEAT) |
            |                                                 |
            |                                  (Node verifies its TUID is in range)
            |                                   (Node delays reply randomly)
            |                                                 |
            |            PHASE 2: POLL REPLY                  |
            |===============================================|
            |                                                 |
            |<-- POST /node/{tuid}/0 (Multicast)------------| [2]
            |    (To <mult_node_send>)                        |
            |    (TID_POLL_REPLY: TUID, SoemCode, CHANGE_COUNT)
            |                                                 |
  (Manager updates Table-of-Devices ToD)                    | [3]
            |                                                 |
```

Message Flow

1) The Poll: The Manager initiates node-discovery by sending a CoAP NON POST to /sig-net/<version>/<scope>/poll. It includes a TID_POLL block. Because QUERY_LEVEL=QUERY_HEARTBEAT, it is only asking for TID_POLL_REPLY.
2) The Reply: Every Node whose TUID falls within the requested range waits a random back-off period (to prevent a network storm) and then sends a CoAP NON POST to /sig-net/<version>/<scope>/node/{tuid}/0. The payload is a TID_POLL_REPLY block containing its TUID, its SoemCode (product identifier), and its current CHANGE_COUNT.
3) Consistency Check: The Manager logs the Node as "Online." It compares the received CHANGE_COUNT to its internal cache. If the count has increased, the Manager knows the Node's configuration has changed and will follow up with a targeted TID_POLL (QUERY_LEVEL=QUERY_CONFIG) to download the new settings.

13.2 Example Node Endpoint Configuration

The following example demonstrates a Manager configuring Endpoint 1 of a specific Sig-Net Device (TUID: 123456789ABC) to listen to Universe 5. This uses the TID_EP_UNIVERSE Type Identifier (defined in Section 11.7.1).

Figure 5 Example Endpoint Configuration

```
Manager                                  Sig-Net Device (TUID: 123456789ABC)
(Listening on <mult_node_send>)             (Listening on <mult_manager_send>)
            |                                                |
            |====== PHASE 1: CONFIGURATION COMMAND (SET) ======|
            |                                                |
            |--- POST /manager/123456789ABC/1 ---------------->| [1]
            |    To IP: <mult_manager_send>                   |
            |    Payload:[TID_EP_UNIVERSE] [Length: 2] [Val: 5] |
            |                                                |
            |                          (Node verifies HMAC)    | [2]
            |                          (Node updates memory)   |
            |                          (Node increments        |
            |                           CHANGE_COUNT)          |
            |                                                |
            |====== PHASE 2: STATE CONFIRMATION (RESPONSE) ====|
            |                                                |
            |<-- POST /node/123456789ABC/1 --------------------| [3]
            |    To IP: <mult_node_send>                      |
            |    Payload: [TID_EP_UNIVERSE] [Length: 2] [Val: 5]|
            |                                                |
  (Manager verifies HMAC)                                    | [4]
  (Manager updates UI to Universe 5)                         |
            |                                                |
```

Message Flow

1) The Command: The Manager constructs a Sig-Net TLV containing the TID_EP_UNIVERSE Type Identifier with a value of 5. It encapsulates this in a CoAP NON POST message and sends to URI /sig-net/<version>/<scope>/manager/123456789ABC/1.
2) Node Processing: The Node receives the packet, validates the HMAC, and parses the URI to determine that the command is intended for its own TUID and specifically for Endpoint 1. The Node applies Universe 5 to Endpoint 1's configuration and increments its global CHANGE_COUNT by 1.
3) The Confirmation: To transmit its new state, the Node constructs an identical TLV (TID_EP_UNIVERSE = 5). It transmits this via CoAP NON POST to the URI /sig-net/<version>/<scope>/node/123456789ABC/1.
4) Stateless Sync: The originating Manager (and any other Managers passively listening to <mult_node_send>) receives the reply, extracts the target TUID and Endpoint from the URI, and updates its internal patch sheet to reflect that the Node is now operating on Universe 5.

13.3 Example Multi-Manager Environments

Sig-Net relies entirely on cryptographic key possession to determine network authority. The following examples illustrate how multiple Managers interact.

13.3.1 Equal Managers

A Main Console and a Backup Console operate on the same network scope. The Main Console provisions the Backup Console with the Master Root Key (K0). Both consoles now operate as Equal Managers. If the Main Console fails mid-show, the Backup Console possesses K0 and can immediately derive the Km_local keys required to onboard a replacement moving light or re-patch the rig.

13.3.2 Guest Manager

A Touring Console arrives at a House Venue. The House Technician wishes to grant the tour temporary control of the rig, but prevent the tour from accidentally altering fixture IP addresses or configurations.

The House Technician provisions the Touring Console with the derived Guest Keys (Km_global, Ks, Kc) but explicitly withholds the Root Key (K0) and all Node-Specific Manager Keys (Km_local).

The Touring Console operates as a Guest Manager. It can discover the rig via Km_global and transmit DMX via Ks, but any attempt to transmit a configuration SET command will fail HMAC verification at the Node, providing a cryptography-enforced Read-Only state.

14 Implementation Guidelines (Informative)

This section provides guidance on miscellaneous concepts that an implementor should consider.

14.1 Hardware Considerations

14.1.1 Non-Volatile Memory Wear Endurance

The Session ID presents a significant memory endurance consideration. It must be committed to non-volatile memory before the first packet of every new session (see Section 8.3).
Manufacturers are therefore strongly recommended to store the Session ID in high endurance non-volatile memory or use a wear levelling algorithm.
Devices acting as Nodes (Receivers) are not required to use NVM for security tracking, as received state is stored in RAM only (see Section 8.6).

14.1.2 Key Storage Hardware

All derived keys retained by a Sig-Net Device shall be stored in hardware that is resistant to external, physical extraction. Storing derived keys in unprotected, easily accessible external flash memory is not recommended for new product designs.

Legacy Hardware (Informative): It is recognised that legacy entertainment devices use standard, unprotected non-volatile memory (e.g. SPI flash) and lack dedicated cryptographic secure elements. To support the secure adoption of Sig-Net on legacy hardware platforms, manufacturers may store derived keys in standard non-volatile memory, provided they explicitly document this hardware limitation in their cybersecurity risk assessment. Manufacturers declaring this limitation must mandate compensating controls in their user documentation, explicitly requiring that the physical security of the device (e.g. restricting access to the lighting rig or equipment racks) is maintained to prevent key extraction.

Furthermore, it is recommended that devices lacking dedicated secure elements should encrypt derived keys before committing them to standard non-volatile memory.

Manager Devices: Managers additionally retain the Root Key (K0)—or the plaintext passphrase used to derive it—permanently. Given the extreme sensitivity of K0 and its source material as the sole root of trust for the entire network, manufacturers are strongly encouraged to utilise dedicated Secure Elements, Cryptographic Co-Processors, or hardware-secured enclaves within the primary MCU to isolate these secrets from general application memory.

14.1.3 Multicast Folding™ Load Scaling

In the default 109-address pool configuration, Sig-Net uses Multicast Folding. For systems using 109 universes or fewer, the mapping is 1:1, resulting in a Discard Load of zero.

For large installations, the universes mathematically fold back over the pool. Because multiple universes share a single multicast group, a Node will naturally receive packets for universes it is not actively consuming. The processing overhead required to parse the CoAP URI and drop these unneeded packets is known as the Discard Load.

At the standard 44Hz transmission rate, a Node assigned to a single universe in a fully loaded 500-universe installation will receive packets for 5 universes (its own, plus 4 folded ones), resulting in a Discard Load of only 4 × 44 = 176 packets per second. Even on a 1,000-universe system, the modulo 109

allocation equation (see Section 9.2.3) caps this Discard Load at 10 × 44 = 440 packets per second. This load is trivial for modern embedded microcontrollers to parse and drop based on the cleartext CoAP URI.

If a specific hardware implementation requires further reduction of the Discard Load, the Manager can use the TID_EP_MULT_OVERRIDE command to re-allocate the Node to an isolated custom multicast group.

14.2 Network Considerations

14.2.1 CoAP Port Coexistence

Sig-Net uses the standard registered CoAP port <coap_port>. In environments where Sig-Net coexists with other CoAP-based systems on the same physical device, the /sig-net/ URI prefix is used as the application-layer discriminator. Network-level VLAN separation is recommended where Sig-Net traffic coexists with other CoAP protocols on shared infrastructure.

14.2.2 Multicast Address Exclusivity

The 239.254.0.0/24 block is drawn from the Administratively Scoped range (RFC 2365) and is not globally reserved exclusively for Sig-Net. In deployments where other multicast applications share the same physical network, operators should verify that no address conflicts exist between Sig-Net's allocated addresses and those used by other protocols. VLAN segmentation is recommended in shared infrastructure environments to provide additional isolation.

14.3 Manufacturer Specific Extensions

The Sig-Net cryptographic framework is designed to be highly extensible. Manufacturers may use the Sig-Net transport layer to secure their own proprietary protocols.

When transmitting proprietary payloads over the Sig-Net framework, manufacturers shall adhere to the following guidelines to prevent interference with standard Sig-Net devices:

1) Mfg-Code Flagging: Packets containing proprietary application payloads must include the Sig-Net-Mfg-Code option (Section 8.3) set to the manufacturer's specific ESTA ID.
2) Custom URIs: Proprietary traffic should utilise custom CoAP URI-Paths (e.g. /mfg/1234/sync) rather than the standard /sig-net/<version>/<scope>/ namespace. This ensures standard nodes drop the packets immediately at the URI Routing Check phase (Section 8.6, Step 4).
3) Multicast Isolation: To prevent unnecessary Discard Load, high-bandwidth proprietary traffic (such as continuous console tracking data) should be transmitted to dedicated, manufacturer-selected multicast IP addresses outside of the standard Sig-Net default pools defined in Section 15.
4) Key Use: Manufacturers should map their proprietary network actors to the standard Sig-Net cryptographic roles. For example, proprietary configuration protocols should be authenticated using Km_local, while proprietary playback or trigger protocols should be authenticated using Ks.

15 Appendix A Multicast Addresses

The following section defines the multicast address allocation based on URI.

Name	URI	Universe	Key	Multicast	Notes
<mult_node_beacon>[1]	/sig-net/<version>/<scope>/node_beacon/{tuid}/0	n/a		239.254.255.255[2]	Fixed
<mult_node_lost>	/sig-net/<version>/<scope>/node_lost/{tuid}/0	n/a	Kc	239.254.255.254	Fixed
<mult_node_send>	/sig-net/<version>/<scope>/node/{tuid}/{endpoint}	n/a	Kc	239.254.255.253	Fixed
<mult_manager_poll>	/sig-net/<version>/<scope>/poll	n/a	Km_global	239.254.255.252	Fixed
<mult_manager_send>	/sig-net/<version>/<scope>/manager/{tuid}/{endpoint}	n/a	Km_local	239.254.255.251	Fixed
<mult_time>	/sig-net/<version>/<scope>/sync /sig-net/<version>/<scope>/timecode/{stream}	n/a	Ks	239.254.255.250	Fixed
<mult_preview>	/sig-net/<version>/<scope>/preview/{stream}	n/a	Ks	239.254.255.249	Fixed
<mult_u1>	/sig-net/<version>/<scope>/level/{universe}	1, 101, 201 …	Ks	239.254.0.1	Programmable
<mult_u2>	/sig-net/<version>/<scope>/level/{universe}	2, 102, 302 …	Ks	239.254.0.2	Programmable
<mult_u3>	/sig-net/<version>/<scope>/level/{universe}	3, 103, 303 …	Ks	239.254.0.3	Programmable
…	…	…	…	…	…
<mult_u108>	/sig-net/<version>/<scope>/level/{universe}	108, 217, 326 ...	Ks	239.254.0.99	Programmable
<mult_u109>	/sig-net/<version>/<scope>/level/{universe}	109, 218, 327 …	Ks	239.254.0.100	Programmable

Note 1: Offboarded devices operating in Beacon Mode transmit to this URI using Security-Mode 0xFF. These packets carry no valid key and the HMAC field is set to 32 zero bytes. Receivers shall not attempt HMAC verification for packets carrying Security-Mode 0xFF.

Informative Note 2: All Sig-Net multicast addresses are allocated from the 239.254.0.0/24 block within the Administratively Scoped range defined by RFC 2365. This block is intentionally distinct from the 239.255.0.0/16 range used by [E1.31] (sACN) to prevent cross-protocol interference.

16 Appendix B Definitions

The following section defines fixed operational parameters referenced symbolically in this specification.

Name	Value	Notes
<poll_backoff_max>	1000	Maximum random delay (in ms) before a Node transmits a poll reply.
<mult_ttl>	32	Time to Live (TTL) for all Sig-Net traffic
<poll_time>	3	Time in seconds between manager polls.
<node_lost_timeout>	3	Number of missed polls before a Node is considered lost.
<coap_port>	5683	
<universe_lost_timeout>	3	Time in seconds before a Node considers a DMX universe stream lost.
<offboard_lockout>	300	Time in seconds after physical power-up during which a Node will accept an authenticated network wipe command.
<sync_lost_timeout>	250	Time in milliseconds a Node will wait for a sync packet before reverting to unsynchronised output mode.
<ip_rollback_timer>	60	Time in seconds a Node waits to detect TID_POLL after an IP change, prior to rollback.
<timecode_lost_timeout>	1	Time in seconds before a Node considers a Timecode stream lost.
<manager_poll_jitter>	500	Maximum random delay (in ms) added to the Manager's poll interval to prevent collisions.
<beacon_min_interval>	5	Minimum time in seconds between offboarded beacons.
<beacon_timeout>	30	Time in seconds before stale beacon entry removal.
<node_processing_max>	500	Maximum time (in milliseconds) for a Node to process a command (including internal RDM transactions) and multicast the reply.
<endpoint_spacing_delay>	1	Minimum delay (in milliseconds) a Node must insert between transmitting sequential reply packets when responding to an All Endpoints (0xFFFF) query.
<universe_announce_interval>	5	Time in seconds between periodic TID_UNIVERSE announcements sent by an active Sender.
<status_publish_rate>	1	Minimum time (in seconds) a Node must wait before proactively multicasting subsequent changes to highly dynamic Status TIDs (e.g.

		TID_EP_STATUS) on the same endpoint.

17 Appendix C Machine-Readable TID Dictionary (Informative)

Below is a JSON dictionary of all Type Identifiers (TIDs) defined in this specification.

The JSON structure defines the Enum, Target Endpoint (Root, Data, or Both), Queryable status, Polling QUERY_LEVEL, and the minimum/maximum payload byte lengths (excluding the 4-byte TLV header).

The definitive, maintainable version of this JSON file is hosted in the official Sig-Net GitHub repository.


```
{
  "protocol": "Sig-Net",
  "version": "0.17",
  "tids": [
    { "name": "TID_POLL", "enum": "0x0001", "target": "Root & Data", "queryable": false, "query_level": null, "length_min": 25, "length_max": 25 },
    { "name": "TID_POLL_REPLY", "enum": "0x0002", "target": "Root", "queryable": false, "query_level": "0x00", "length_min": 12, "length_max": 12 },
    { "name": "TID_LEVEL", "enum": "0x0101", "target": "Data", "queryable": false, "query_level": null, "length_min": 1, "length_max": 512 },
    { "name": "TID_PRIORITY", "enum": "0x0102", "target": "Data", "queryable": false, "query_level": null, "length_min": 1, "length_max": 512 },
    { "name": "TID_PREVIEW", "enum": "0x0103", "target": "Data", "queryable": false, "query_level": null, "length_min": 1, "length_max": 512 },
    { "name": "TID_SYNC", "enum": "0x0201", "target": "Data", "queryable": false, "query_level": null, "length_min": 0, "length_max": 0 },
    { "name": "TID_TIMECODE", "enum": "0x0202", "target": "Data", "queryable": false, "query_level": null, "length_min": 5, "length_max": 5 },
    { "name": "TID_UNIVERSE", "enum": "0x0203", "target": "Data", "queryable": false, "query_level": null, "length_min": 7, "length_max": 7 },
    { "name": "TID_RDM_COMMAND", "enum": "0x0301", "target": "Data", "queryable": false, "query_level": null, "length_min": 26, "length_max": 257 },
    { "name": "TID_RDM_RESPONSE", "enum": "0x0302", "target": "Data", "queryable": false, "query_level": null, "length_min": 26, "length_max": 257 },
    { "name": "TID_RDM_TOD_CONTROL", "enum": "0x0303", "target": "Data", "queryable": false, "query_level": null, "length_min": 1, "length_max": 1 },
    { "name": "TID_RDM_TOD_DATA", "enum": "0x0304", "target": "Data", "queryable": true, "query_level": null, "length_min": 2, "length_max": 1356 },
    { "name": "TID_RDM_TOD_BACKGROUND", "enum": "0x0305", "target": "Data", "queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 1 },
    { "name": "TID_RDM_FLOW_CONTROL", "enum": "0x0306", "target": "Data", "queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 2 },
    { "name": "TID_RT_OFFBOARD", "enum": "0x0401", "target": "Root", "queryable": false, "query_level": null, "length_min": 4, "length_max": 4 },
    { "name": "TID_NW_MAC_ADDRESS", "enum": "0x0501", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 6 },
    { "name": "TID_NW_IPV4_MODE", "enum": "0x0502", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 1 },
```

{ "name": "TID_NW_IPV4_ADDRESS", "enum": "0x0503", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 4 },

{ "name": "TID_NW_IPV4_NETMASK", "enum": "0x0504", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 4 },

{ "name": "TID_NW_IPV4_GATEWAY", "enum": "0x0505", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 4 },

{ "name": "TID_NW_IPV4_CURRENT", "enum": "0x0506", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 12 },

{ "name": "TID_NW_IPV6_MODE", "enum": "0x0581", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 1 },

{ "name": "TID_NW_IPV6_ADDRESS", "enum": "0x0582", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 16 },

{ "name": "TID_NW_IPV6_PREFIX", "enum": "0x0583", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 1 },

{ "name": "TID_NW_IPV6_GATEWAY", "enum": "0x0584", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 16 },

{ "name": "TID_NW_IPV6_CURRENT", "enum": "0x0585", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 33 },

{ "name": "TID_RT_SUPPORTED_TIDS", "enum": "0x0601", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 1356 },

{ "name": "TID_RT_ENDPOINT_COUNT", "enum": "0x0602", "target": "Root", "queryable": true, "query_level": "0x00", "length_min": 0, "length_max": 2 },

{ "name": "TID_RT_PROTOCOL_VERSION", "enum": "0x0603", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 1 },

{ "name": "TID_RT_FIRMWARE_VERSION", "enum": "0x0604", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 68 },

{ "name": "TID_RT_DEVICE_LABEL", "enum": "0x0605", "target": "Root", "queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 64 },

{ "name": "TID_RT_MULT", "enum": "0x0606", "target": "Root", "queryable": true, "query_level": "0x00", "length_min": 0, "length_max": 1 },

{ "name": "TID_RT_IDENTIFY", "enum": "0x0607", "target": "Root", "queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 1 },

{ "name": "TID_RT_STATUS", "enum": "0x0608", "target": "Root", "queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 4 },

{ "name": "TID_RT_ROLE_CAPABILITY", "enum": "0x0609", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 1 },

{ "name": "TID_RT_REBOOT", "enum": "0x060A", "target": "Root", "queryable": false, "query_level": null, "length_min": 5, "length_max": 5 },

{ "name": "TID_RT_MODEL_NAME", "enum": "0x060B", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 64 },

{ "name": "TID_RT_SCOPE", "enum": "0x060C", "target": "Root", "queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 32 },

{ "name": "TID_RT_OTW_CAPABILITY", "enum": "0x060D", "target": "Root", "queryable": true, "query_level": "0x02", "length_min": 0, "length_max": 3 },

{ "name": "TID_EP_UNIVERSE", "enum": "0x0901", "target": "Data", "queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 2 },

```
    { "name": "TID_EP_LABEL", "enum": "0x0902", "target": "Data", "queryable": true,
"query_level": "0x01", "length_min": 0, "length_max": 64 },

    { "name": "TID_EP_MULT_OVERRIDE", "enum": "0x0903", "target": "Data",
"queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 4 },

    { "name": "TID_EP_CAPABILITY", "enum": "0x0904", "target": "Data", "queryable":
true, "query_level": "0x01", "length_min": 0, "length_max": 1 },

    { "name": "TID_EP_DIRECTION", "enum": "0x0905", "target": "Data", "queryable":
true, "query_level": "0x01", "length_min": 0, "length_max": 1 },

    { "name": "TID_EP_INPUT_PRIORITY", "enum": "0x0906", "target": "Data",
"queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 512 },

    { "name": "TID_EP_STATUS", "enum": "0x0907", "target": "Data", "queryable": true,
"query_level": "0x01", "length_min": 0, "length_max": 4 },

    { "name": "TID_EP_FAILOVER", "enum": "0x0908", "target": "Data", "queryable":
true, "query_level": "0x01", "length_min": 0, "length_max": 3 },

    { "name": "TID_EP_DMX_TIMING", "enum": "0x0909", "target": "Data", "queryable":
true, "query_level": "0x01", "length_min": 0, "length_max": 2 },

    { "name": "TID_EP_REFRESH_CAPABILITY", "enum": "0x090A", "target": "Data",
"queryable": true, "query_level": "0x01", "length_min": 0, "length_max": 1 },

    { "name": "TID_EP_PROTOCOL", "enum": "0x090B", "target": "Data", "queryable":
true, "query_level": "0x01", "length_min": 0, "length_max": 1 },

    { "name": "TID_DG_SECURITY_EVENT", "enum": "0xFF01", "target": "Root",
"queryable": true, "query_level": "0x03", "length_min": 0, "length_max": 23 },

    { "name": "TID_DG_MESSAGE", "enum": "0xFF02", "target": "Root", "queryable":
true, "query_level": "0x03", "length_min": 0, "length_max": 64 },

    { "name": "TID_DG_LEVEL_FOLDBACK", "enum": "0xFF03", "target": "Data",
"queryable": true, "query_level": null, "length_min": 0, "length_max": 512 }

  ]

}
```


18 Appendix D Glossary

Beacon Mode: The operational state of an Offboarded device. In Beacon Mode, a device cannot process authenticated network traffic and only transmits unauthenticated, periodic presence beacons (Security-Mode 0xFF) to alert Managers that it requires onboarding.

CHANGE_COUNT: A 16-bit wrap-around counter used to guarantee eventual consistency without requiring reliable transport.

Consumer: An endpoint or device that receives control data from the Sig-Net network and processes, displays, or translates it into a physical format. A Consumer takes data off the network.

Discard Load: The processing burden placed on a Node by receiving multicast packets for Universes that share its multicast group, but to which the Node itself is not assigned. The Node must parse the CoAP URI of these extraneous packets to determine they are not relevant before discarding them.

Gateway (DMX Gateway): A specific type of Sig-Net Device designed to bridge the Sig-Net network to legacy DMX512/RDM networks. It translates between network payloads and physical serial data.

Hardware-Secure Storage: Key storage resistant to software-level extraction.

HMAC (Hash-based Message Authentication Code): A cryptographic algorithm used to guarantee that a packet was generated by an authorised device (possessing the correct derived key role) and has not been tampered with in transit.

HKDF (HMAC-based Extract-and-Expand Key Derivation Function): A simple, standard cryptographic key derivation function defined in RFC 5869.

K0 (Root Key): A 256-bit symmetric cryptographic key that acts as the single root of trust for a Sig-Net network. It must be distributed to all devices out-of-band and is used to mathematically derive the role-specific keys.

Kc (Citizen Key): A 256-bit key derived from K0 using HKDF-Expand. Used by all onboarded Sig-Net Devices (Managers, Senders, and Nodes) to cryptographically sign their On-Boot Notification packets, and by Nodes to authenticate their routine discovery replies and status reports.

Ks (Sender Key): A 256-bit key derived from K0 using HKDF-Expand. Used to authenticate level data and timecode packets.

Km_global (Global Manager Key): A 256-bit key derived from K0 using HKDF-Expand. Used to authenticate global management multicasts such as TID_POLL.

Km_local (Node-Specific Manager Key): A 256-bit key derived from K0 using HKDF-Expand, bound to a specific Node's TUID. Used to authenticate targeted configuration commands.

Lost Mode: A management fallback state triggered when a fully Onboarded Node stops receiving valid TID_POLL requests from an active Manager. In Lost Mode, the Node continues to process any active Sender data normally, but begins transmitting authenticated "lost" notifications to the <mult_node_lost> URI to signal its orphaned status to the network.

Manager: A Sig-Net Device. See Section 6.2

Multicast Folding™: A network routing strategy where a large, technically unlimited number of universes are mathematically mapped ("folded") into a smaller, fixed pool of multicast IP addresses using a modulo operation. This prevents IGMP table exhaustion on network switches while keeping Discard Load manageable on large systems.

Native Configuration: Commands and parameters natively defined by the Sig-Net protocol using TIDs, as distinct from encapsulated RDM payloads.

Node: See Section 6.2.

Offboarded: A Sig-Net Device that has not been assigned a Root Key (K0) and has not been explicitly configured for Open Mode. An offboarded device operates exclusively in Beacon Mode; it cannot participate in standard Sig-Net communication and shall only transmit unauthenticated presence beacons to alert Managers that it requires onboarding.

Open Mode: An explicit, user-selected operational state where a device operates without cryptographic keys or HMAC authentication. It is intended strictly for low-risk environments. To prevent downgrade attacks, Open Mode is isolated from Secure Mode devices via CoAP Option 2076.

PBKDF2 (Password-Based Key Derivation Function 2): A standard cryptographic key derivation function (defined in RFC 8018) used by Sig-Net to stretch a human-readable passphrase into a cryptographically secure 256-bit Root Key (K0), providing robust defence against offline dictionary attacks.

RDM-Responder: A non-networked device (e.g. a standard DMX moving light) sitting downstream of a Gateway, communicating via standard E1.20 serial RDM.

Sender: See Section 6.2.

Sequence Number (Sig-Net-Seq-Num): A 32-bit unsigned integer that increments by 1 for every packet transmitted within a specific session. Along with the Session ID, it guarantees packet freshness and prevents individual packet replay attacks.

Session ID (Sig-Net-Session-ID): An increasing 32-bit unsigned integer stored in non-volatile memory. It increments upon device initialisation or when the Sequence Number overflows. It serves as the primary defence against whole-session replay attacks.

Sig-Net Device: Any physical IP-connected piece of hardware equipped with a Sig-Net network interface. A Sig-Net Device may implement the functional classes of Manager, Sender, Node, or any combination thereof.

SoemCode: A 32-bit unique product identifier self-assigned by the manufacturer. The upper 16 bits consist of the manufacturer's ESTA Manufacturer ID, and the lower 16 bits are a manufacturer-defined product variant ID. It is reported during network polling, allowing Managers to identify product types during node-discovery without relying on external databases.

Supplier: An endpoint or device that generates control data and transmits it onto the Sig-Net network.

TUID (Transport Unique Identifier): A 48-bit identifier for a Sig-Net Device.

TID (Type Identifier): A 2-byte unsigned integer forming the first field of a Sig-Net TLV, defining the structure of the data payload.

TLV (Type-Length-Value): A data encoding scheme used by Sig-Net to efficiently pack disparate data types (e.g. DMX levels, Priority, and RDM commands) into a single UDP payload.

UID (Unique Identifier): A 48-bit identifier consisting of a 16-bit ESTA Manufacturer ID and a 32-bit device ID, used for addressing during RDM transactions, as defined in [E1.20].

19 Appendix E CRA and ENISA Compliance Reference (Informative)

This appendix is provided as a navigation aid for manufacturers and security reviewers compiling Technical Files for regulatory certification. It maps the security requirements of the EU Cyber Resilience Act (CRA) and the corresponding implementation guidance from the European Union Agency for Cybersecurity (ENISA) against the relevant sections of this specification. It does not constitute a compliance assessment or guarantee. Regulatory compliance remains the sole responsibility of the manufacturer as stated in Section 1.3.

19.1 CRA Annex 1, Part 1 Mapping

CRA Annex I, Part I	Requirement Summary	Sig-Net Reference
Point 2(a)	No known exploitable vulnerabilities	Section 1.4 — Vulnerability Disclosure
Point 2(b)	Secure by default configuration	Section 7.2 — K0 onboarding mandate; Section 7.2.3 — Passphrase Mapping & Strength Requirements;
Point 2(c)	Protection against unauthorised access	Section 4.5 — Authentication; Section 4.7 — Authorisation; Section 7.3 — Key Derivation
Point 2(d)	Protection of confidentiality	Section 4.4 — Encryption rationale; Section 5.6 — Eavesdropping; Section 7.6 — Key storage
Point 2(e)	Protection of integrity	Section 4.4 — Encryption rationale; Section 4.6 — Integrity; Section 8.5 — HMAC Calculation; Section 8.6 — Replay Prevention. Note: Sig-Net addresses integrity via HMAC. The confidentiality aspect of this point is addressed through the manufacturer's risk assessment as described in Section 4.4.
Point 2(f)	Protection of stored and transmitted data	Section 7.6 — Key storage; Section 14.1.2 — Key storage hardware
Point 2(g)	Minimisation of attack surface	Section 4.7 — Implicit authorisation; Section 5.7 — Blast radius containment; Section 7.3 — Per-device Km_local
Point 2(h)	Resilience against denial of service	Section 8.6 Step 8 — DoS rate limiting; Section 10.2.2 — Beacon Mode (multicast isolation); Section 10.2.2 Manager Operation (poll snooping)
Point 2(i)	Limitation of impact of incidents	Section 5.7 — Compromised device blast radius; Section 7.3.1 — Key Lifecycle and Erasure (Discard K0 after onboarding)
Point 2(j)	Recording and monitoring capability	Section 11.8.1 – TID_DG_SECURITY_EVENT (provides a standardised network- reporting mechanism for security anomalies, supporting manufacturer compliance).
Point 2(k)	Secure update mechanism	Section 4.9.1 — Firmware updates (manufacturer responsibility); Section 4.9.2 — Key rotation

19.2 ENISA Security by Design and Default Playbook Mapping

The following table maps the architectural features of Sig-Net to the specific implementation playbooks published by ENISA to assist manufacturers in proving "Security by Design and Default."

ENISA Playbook	Playbook Objective	Sig-Net Implementation Reference
4.2 Least Privilege	Limit blast radius and prevent lateral movement.	Section 10.3.3: The Global Manager Key (Km_global) is stripped of configuration privileges. All configuration is restricted to the specific Km_local key of the target node.
4.3 Strong Identity & Auth	Prevent impersonation.	Section 7.3: All roles (Manager, Sender, Node) utilise mathematically distinct keys derived via HKDF, backed by stateless HMAC-SHA256 signatures.
4.4 Attack Surface Minimisation	Remove unnecessary interfaces.	Section 8.6: The responder state machine drops unsupported cryptographic modes, malformed CoAP codes, and unpatched universes at the routing layer before processing.
4.6 Open Design	Avoid security through obscurity.	Sig-Net utilises open, peer-reviewed cryptographic standards (PBKDF2, HKDF, HMAC, SHA-256) rather than proprietary obfuscation.
4.10 Logging & Monitoring	Provide visibility to detect misuse.	Section 11.8.1: Nodes are mandated to track and report HMAC failures, replay anomalies, and DoS rate-limiting events via TID_DG_SECURITY_EVENT.
4.16 Restrictive Initial Access	Eliminate shared/default credentials.	Section 7.2: Explicitly prohibits shipping with default Root Keys. Manual hexadecimal entry is banned to prevent trivial/weak key reuse.
4.18 Unique Device Secrets	Ship unique cryptographic identities.	Section 7.3: Device management keys (Km_local) are mathematically bound to the Node's unique hardware identifier (TUID).
4.19 Mandatory Security Onboarding	Block operation until setup is complete.	Section 7.2.2: Devices lacking a Root Key are completely locked out of authenticated traffic and restricted to an isolated "Beacon

ENISA Playbook	Playbook Objective	Sig-Net Implementation Reference
		Mode" until physical onboarding occurs.
4.22 Secure Recovery Lifecycle	Provide secure reset mechanisms.	Section 7.7: Defines authenticated network wipe commands (TID_RT_OFFBOARD) with strict boot-time safety lockouts, alongside out-of-band physical wipe requirements.
5.0 Machine Readable Attestation	Automate security validation.	Section 11.6: Nodes dynamically report their Protocol Version, Firmware Version, and Role Capabilities via Root TIDs, directly supporting the automated generation of Machine-Readable Security Manifests (MRSM) by conformance tools.

20 Appendix F Architectural Rationale (Informative)

This appendix provides the engineering and cryptographic justification for several key design decisions within the Sig-Net protocol, answering common implementation questions regarding the choice of underlying technologies.

20.1 Why CoAP?

The Constrained Application Protocol (RFC 7252) provides an ultra-lightweight, standardised UDP header that is natively understood by modern IT infrastructure. While MQTT was evaluated—and in some respects represents a superior conceptual match for Sig-Net's Publish/Subscribe state model—the ESTA FENCE project has selected CoAP as its foundation. Standardising on CoAP aligns Sig-Net with the industry's future trajectory, allowing manufacturers to reuse underlying network libraries and minimise firmware bloat.

20.2 Why Cleartext Routing

[OSCORE] classifies the CoAP Uri-Path as an encrypted inner option. While a proxy-routing URI can technically be exposed in the outer header, doing so duplicates data and wastes CPU cycles. Sig-Net explicitly relies on the cleartext URI (e.g. /sig-net/v1/<scope>/level/101) as the definitive hardware-layer routing filter for its Multicast Folding™ architecture. Because multiple universes share a single multicast group, a Node receives significant traffic destined for other devices. By keeping the URI in cleartext and appending the HMAC as a trailing CoAP Option, network switches and multi-port Gateways can parse routing targets at line-rate, dropping irrelevant multicast universes (the Discard Load) without wasting critical CPU cycles invoking a cryptographic decryption engine.

20.3 Why Out-of-Band Keying?

Because Sig-Net V1 prioritises real-time execution speed by using plaintext payloads, a new Root Key (K0) cannot safely be transmitted over the network without encryption. Out-of-band Onboarding (e.g. USB,

NFC, or local UI) guarantees that physical proximity is the root of trust, eliminating remote, network-based key interception vectors during initial setup.

20.4 Why a Custom HMAC structure instead of OSCORE?

Standard secure CoAP (OSCORE, RFC 8613) is a point-to-point (unicast) protocol, rendering it fundamentally incompatible with the decision to use multicast.

While [Group OSCORE] extends support to multicast, it achieves group-level source authentication by mandating the use of asymmetric digital signatures (e.g. Ed25519/ECDSA). Verifying asymmetric signatures at a sustained 44Hz per universe on mid-range embedded microcontrollers introduces unacceptable latency.

Instead, Sig-Net uses a custom trailing symmetric HMAC. This provides the required stateless integrity and implicit group authentication while remaining highly performant on constrained silicon. Furthermore, Sig-Net requires cleartext URIs for line-rate hardware routing (allowing DMX Gateways to drop unpatched universes without invoking cryptography), which standard [OSCORE] explicitly obscures.

20.5 Why a Custom TLV structure instead of CBOR?

The standard data serialisation format for CoAP environments is Concise Binary Object Representation (CBOR, RFC 8949). While CBOR is highly flexible and self-describing, parsing dynamic maps and arrays introduces unnecessary computational overhead and memory fragmentation risks on constrained microcontrollers.

Entertainment control data—such as a fixed 512-byte DMX universe, an encapsulated RDM packet, or a 5-byte timecode frame—is highly deterministic. A simple, flat Type-Length-Value (TLV) structure allows receivers to parse payloads using basic pointer arithmetic, entirely avoiding the CPU overhead and library footprint of a full CBOR decoder. This custom TLV approach maximises bandwidth efficiency and execution speed, matching the extreme real-time processing requirements of the live events industry.

20.6 Why CoAP NON POST only?

Sig-Net is designed for extreme real-time performance and multi-manager environments. Relying on CoAP Confirmable (CON) messages introduces state-tracking overhead, retry-loop fragility, and potentially ACK storms on large networks. By exclusively using Non-Confirmable (NON) POST messages, Sig-Net relies on the deterministic CHANGE_COUNT mechanism (Section 10.2.7) to guarantee eventual consistency across the network, completely decoupling state synchronisation from transport reliability.

20.7 Why multicast RDM responses?

Standard E1.20 RDM is historically a Request/Response protocol. However, the latest E1.20-2025 standard introduces asynchronous "Push Notifications" (Qued Message mandate) precisely because the old polling model cannot scale on dense IP networks. By mandating that all nodes proactively multicast their TID_RDM_RESPONSE packets to <mult_node_send>, Sig-Net natively implements this new RDM push model into its Pub/Sub architecture.

This allows a Main Console to track a patch change made by a Riggers Remote, achieving real-time, stateless synchronisation across all Managers and completely eliminating the need for a fragile, centralised RDMnet Broker.

20.8 Future Extensibility

The protocol is structurally designed to accommodate future security requirements via the Sig-Net-Security-Mode (Option 2076) and the v1 URI namespace. Should future silicon advancements or legislative mandates require payload encryption, in-band key exchange (e.g. DTLS handshakes), or new

CoAP verbs, a version update (e.g. Security-Mode 0x80 or /sig-net/v2/) can introduce these features without breaking the core TLV routing or HMAC architecture.

21 Appendix G Document Revisions (Informative)

V0.1 (9/2/2026)

- Initial concept draft.

V0.7 (7/3/2026)

- Release to three companies for preliminary peer review.

V0.8 (13/3/2026)

- Changed TI to TID for RDM PID analogy.
- Changed all section cross references to Word style for pdf hot-links.
- Implemented PR suggestion to change default universe to multicast mapping.
- Added note about edge case where universe could be mapped to different multicast on different nodes. Strong do not do this recommendation.
- Added to 10.1 a rule to define how we handle an unknown TID.
- Reorganised the actual multicast address defaults in Appendix A to put the streaming and administrative on different subnets.
- Defined broadcast and Vendorcast tuid.
- Defined broadcast endpoint as all data endpoints.
- Reworked Native Configuration Commands and RDM to use unicast with a mandated multicast update mechanism.
- Resolved numerous errors where Manager and Sender use was confused for historical reasons.
- Defined concept of “Discard Load”. See glossary.
- Added concept of Sig-Net-Sender-ID which is a concatenation of TUID+Endpoint. This is used as a CID replacement for slot-priority and also to resolve the backup console reply attack scenario.
- Mandate unilateral RDM Get Response on a change (single TLV in payload)
- Include xxx for both RDM and Sig-Net device firmware upload.
- Added support for proprietary protocol wrapping.
- Fixed two security issues relating to Session ID across boots (replay attack).
- Added and then deleted an attempt at OTW keying and key rotation.
- Added Node power on behaviour (poll reply)
- Added mandated fields to TID list.
- Changed port direction terminology to supplier/consumer.
- Implemented version number and protocol number support TIDs.
- Implemented TID for Supported TIDs.
- Implemented Diagnostics TIDs.
- Released to additional companies for peer review.

V0.9 (16/3/2026)

- Clarified that Change Count is only incremented for TID changes, not PID.
- Added TID_SYNC
- Added all the network setting TIDs
- Add rollback timer for network changes.
- Added concept of Lost Mode and Beacon Mode.
- Move Beacon to its own multicast to allow filtering for DoS.
- Reorganised section 10.2 to make difference between Lost Mode and Beacon Mode more obvious.
- Changed the equation for the default Multicast Folding patch. (Provides zero Discard Load for shows < 100 universe).
- Added node reply mandate to ensure MTU limit for fragmentation not broken.
- Corrected node receiver parsing state machine to include check for version.
- Resolved storage ambiguity in 13.1.2
- Fixed broadcast tuid key flaw.
- Added limits for session tracking.
- Add password base K0 option for ‘PC based systems’.
- Changed RDM response to multicast irrelevant of whether the command was unicast or multicast.
- Clarified how per-universe priority is handled.
- Allowed but discouraged unicast TID_LEVEL.
- Made Multicast Folding configuration optional.
- Moved responsibility for Entropy check to the manager.
- Renamed Controller to Sender.
- Clarified Root Endpoint is mandated, Data Endpoint is not.
- Removed references to Secure Boot.
- Added diagram to show how key types are used.
- Fixed TID enum collision.

- Added URI destination to all TID definitions.

V0.10 (21/3/2026)

- Acknowledgements updated.
- Terms updated.
- Section 7.2.2 added. This defines password stretching to avoid manual entry of k0 and allow 8-char password.
- Section 1.3. Clarify that it is also a user / integrator responsibility.
- Section 4.5/4.7. Slightly informal text improved.
- Section 6.3 redrafted to clarify the human aspect.
- Section 6.5 clarify this is expected to be the same as the RDM ID.
- Section 7.4 clarify the k0 generation intent.
- Added section 8.6 stp 9 to clarify NV burn out issue.
- Section 7.6 updated to describe the concept of 'stateless' and avoid k0 storage.
- Section 8.3 Session-ID definition updated to encompass software device.
- Major rework on TID Get. Poll argument ALL changed to QUERY_LEVEL Section 11.9. Plus Length = 0 is now globally a TID Get.
- Added Wireshark example.
- Changed 8.6 stp 6: We're moving to keep session ID in ram on the Node. This fixes our NV wear and 'reboot to fix' issue with a trivial increase in threat surface.

V0.11 (21/3/2026)

- Fixed TID length = 0 option on all Queryable TIDs
- Fixed error caused by Node Sender ID stored in ram. See First Packet Bootstrapping.
- Softened conditions of use.
- Changed name to Sig-Net (changed all uri syntax).
- Added logo.
- New Section 12 added to address versioning. Section 8.6-4 updated to point to Section 12.
- The passcode updates from v0.10 had not fully been implemented in other sections. Additionally, the simple hash was not adequate. Updated 7.3 to add a specific key life cycle section. Updated 7.6 to match. Updated 7.2.2 for higher security.
- Passcode vs k0 entry is now mandated based on hardware. i.e. humans are not allowed to enter 64-char code. Numerous sections updated including 7.2.2
- TID_POLL payload changed to add manager tuid and manager SoemCode. Purpose is to make it simpler for managers to discard their own comms when backing off in a multi-manager environment.
- Added the concept of Manufacturer specific TIDs (Section 10.1.1). Note this is in addition to manufacturer specific payload.
- 10.4.3 updated to make it clear that packing query TIDs is allowed and expected.
- Section 8.5-3. This is a wire breaker. Sig-Net_Sender_ID was shown a 1 byte. Corrected to 8 bytes.
- TID_SYNC definition had trailing slash in URI. Fixed.
- Thread modelling led to these changes:
    - Section 8.6 DoS mitigation has been changed to mandate rate limiting. This conclusion was reach from the threat modelling process.
    - Section 10.2.2 reworked to add a poll jitter. This is to avoid colliding polls in multi-manager environment.
    - Assorted trivial cleaning.
    - Added mandates for beacon mode to mitigate DoS vector.
    - Section 7.2.3 Passphase entry made more onerous in order to achieve 50 bits entropy implicitly required for compliance. Also, SALT added to HMAC.
    - Section 10.2.1 Beacon rules enhanced.
    - Traceability is required for CRA audit. Nodes were fine but Senders and Managers previously had no 'announce' onto the network. Updated 10.2.5 / 7.1 to require all devices to do an announce of: TID_POLL_REPLY, TID_RT_FIRMWARE_VERSION, TID_RT_PROTOCOL_VERSION. Also rebranded Node Key (Kc) as Citizen Key (Kc).
    - Appendix D updated to match Threat Analysis.
- Introduced <node_prosessing_max> to give a deterministic timeout on lost TID/PID responses.
- Increased <sync_lost_timeout>.

V0.12 (25/3/2026)

- Fig1 added. Key Hierarchy.
- Section 7.2.3 passphrase length error corrected.
- Section 7.2.3 Added informative notes regarding the passphrase to K0 algorithm using a fixed public SALT. Making the point that whilst a variable SALT (e.g. Venue name) would be preferred, it destroys the UI gain of a 10-digit pass code. Also, that a fixed public SALT is still better than a null SALT as it guards against existing rainbow table use.
- Section 7.2.3 Added recommendation that Manager 'offer' strong passphrases. This is to improve the [Shannon] score relative to human created passcodes which are intrinsically weaker.
- Kn to Kc replacement completed.
- TUID broadcast and Vendorcast deleted from document. It was causing a security vulnerability (Km_global attack vector).

- Section 7.2.3 updated for passcode entry. Max length defined. Manager mandated checks, node and sender not mandated. Reference code recommended. Also rework text to make the entry options clearer.
- Section 8.6. Improves DoS mitigation.
- General. Removed association of Session ID with Boot Counting.
- Added TID_RT_ROLE_CAPABILITY.
- Credits update.
- Peer review feedback that HMAC vs encryption processing power argument is flawed. Redrafted Section 4.2 to clarify the rationale is about multi-universe scaling (particularly on platforms without cyber hardware assist).
- Section 7.7. Added info note on epoch ending for clarity.
- Section 7.6 updated to address storage realities.
- Section 7.7.2 / 7.5 updated to specifically allow secure out-of-band.
- Section 8.8 updated to deal with a session exhaustion issue.
- Section 8.3 reworded for clarity.
- Section 8.2/6.8.4 updated to clarify that non-tuid indices are decimal in URI.
- Section 11.3 updated. RDM TIDs are allowed to root endpoint (config / firmware)
- Section 9.2.5. Mandate TID_EP_MULT_COMMAND user initiated by Manager.
- General terminology clean up for Parameter Queries
- Section 10.4.2 Batching applies to SET (clarification)
- Updated Wireshark example (was showing Endpoint = 0 == illegal)
- Section 10.8.1 Timecode stop and stutter addressed.
- Contradiction is TID_DG_SECURITY_EVENT mandate fixed. (Shall throughout).
- Section 11.1.1 URI was incorrect, fixed.
- Section 10.3.1 The poll URI not listed, fixed.
- Section 11.6.1 wording improved.
- Section 11.6.5/11.7.2 the issue of a “clear command” resolved.
- Section 11.6.7 Clarified Identify does not persist.
- Section 11. Major clean up on length and URI (particularly Endpoints)
- Section 11.6.10 TID_RT_REBOOT added.
- Section 11.8.1 clarified TLV packing expectation.
- Section 11.9 evolved into a summary.
- Added Section 12.4 to deal with legacy senders in future versions of Sig-Net
- Figure 2 improved.
- TID_EP_DIRECTION changed to include RDM enable / disable.
- TID_EP_FAILOVER added
- TID_RT_MODEL_NAME added
- TID_EP_DMX_TIMING added
- [sACN] Normative corrected.
- Added Appendix E.
- Section 4.4 reworked.
- Mandated lower case for K0 display ‘B’ vs ‘8’ issue.
- Section 8.5 (1-1) improved with [CoAP] section references.
- [Shannon] added.
- [CSPRNG] added.
- Section 8.6. Numerous pleasingly pedantic improvements.
- Section 10.4.5. Fixed Rollback verification issue.
- Section 10.6.3. Active data changed to Dynamic data.

V0.14 (2/4/2026)

- Inadvertent deletion of <mult_manager_send> reinstated.
- Section 9.2.2 updated for above.
- Section 10.3.1 updated for above.
- Section 10.3.2 updated for above.
- TID_EP_REFRESH_CAPABILITY added. Section 10.6.3 adjusted to implement new TID.
- Added new Section 18.2 To provide a summary cross reference for the new ENISA document.
- Section 11.6.7 Updated identify and added mute.
- <ip_rollback_timer> increased to 60s. 30s was deemed too low to allow for (eg) spanning tree cycle times.
- Section 10.2.4 updated to clarify that an individual TLV must also not exceed the MTU limit.
- Section 11.8.1 clarified that attacker IP is indicative only.

V0.15 (3/4/2026)

- Added Section 2.2 informative references.

- Updated Section 8.3 to make Option 2236 variable length. This is for future compatibility of a potentially encrypted payload.
- Updated preface.
- Updates Section 4.4
- Section 11.7.4 Changed name and expanded functions.
- Section 11.3.1-5 Corrected to remove root endpoint.
- Section 10.5.3 Added to exclude certain PID classes from RDM payload on Virtual Endpoints. This plugs a security hole where a virtual endpoint can adjust the root via legacy protocols.
- Section 6.8.2 Introduced concept of a Virtual Endpoint.
- Normative updated.
- TID_RDM_FLOW_CONTROL added.
- Section 6.6 updated to define ephemeral TUIDs.

V0.16 (9/4/2026)

- Section 10.7.2. Dealt with the 'Sync problem' by linking TID_SYNC to Sig-Net-Sender-ID.
- Wire breaking change. Scope now added to URI to avoid multi-tenanted networks tripping over each other.
- Added normative [URI].
- TID_RT_MODEL_NAME payload change.
- Numerous trivial fixes.
- Section 10.2.3 Dealt with potential endpoint broadcast storm.
- Section 10.2.7 Resolved IP change consistency tracking.
- Section 10.4.5 Mandated IP change confirmation from old IP.
- Section 10.4.2 Added notes about data endpoint hardware limitations.
- Section 11.7.8 Added stop DMX option.
- Section 11.7.5 Added Fallback mode.
- Section 11.8.1/2 Updated endpoint to zero only.
- Section 11.8.2 Added rate limiting to resolve DoS attack concern.
- Section 14.1.2 Rolled in dealing with legacy product storage.
- Section 7.6 and 14.1.2 Expanded to ban export of derived keys and address derived key storage in legacy products.
- Section 8.2 Corrected ASCII value error.
- Section 10.2.1 Corrected Beacon rate error.
- Section 10.2.1 Beacon logging limited to managers.
- Section 10.1 Incorrect Appendix reference.
- All Appendix cross references changed to section number references.
- Section 10.5.2 Added clarification that gateways do not need to cache downstream RDM data and that they can and should rely on the [RDM]-2025 Queue mandate.
- Section 11.3.4 Breaking Change: x of y packets added to payload.
- Section 11.6.1 typo fixed.
- Section 8.3 Made it clearer that CoAP requires the exact option order to be maintained.
- Sections 11.7.8 / 11.8.3 / 11.7.4 Typos fixed.
- New Appendix C with json definitions of TIDs.

V0.16a (15/4/2026)

- Removed references to unpublished ANSI standards

V0.17 (14/4/2026)

- Section 10.2.6 Updated the mandated tids in heartbeat to match announce
- TID_EP_STATUS bit fields updated.
- Prohibition on E1.37-7 was a typo – removed.
- Modified Sections 7.2, 7.5, 4.9.2 to remove the OTW mandate.
- Added <mult_preview>
- Section 11.2.3 TID_PREVIEW added.
- Section 11.6.12. Added warning about loss of contact.
- Replaced "Provisioning" with "Onboarding" ready for OTW.
- Mandated a Sender respond to Poll (OTW requirement).
- Section 6.2. Defined Visualiser role. (OTW requirement).
- Section 11.6.9. Added Visualiser role flag.
- Section 8.3. Use of term thread caused confusion – replaced.
- Section 11.6.10 added TID_RT_OTW_CAPABILITY.
- Section 10.2.1 updated for revised mandated TLVs.
- Section 10.2.5 updated for revised mandated TLVs.
- Section 10.2.6 updated for revised mandated TLVs.

V0.18 (20/4/2026)

- Section 11.7.7 Updated bit fields.
- Section 11.2.1/2 Mandate for Visualisers added.
- Section 11.8.3 Visualiser mandate changed to no.
- Section 10.6/11.7.6. Short packet priority clarified.
- Section 7.2.3 Removed ambiguity.
- Section 16. Definitions added.
- Section 10.2.7 / 10.4.2. Updated to clarify that status changes as well as config changes should generate a 'publish' and a CHANGE_COUNT increment.
- Section 7.2.2 updated to reference 10.2.1 to remove ambiguity.
- Section 7.7.1/7.2.3 updated for clarity.
- Section 6.6. Added informative on how TUID is displayed.

V0.19 (3/5/2026)

- Section 6.6. Corrected TUID display type to upper case hex.
- Section 8.6.9. Redraft throttling per Discord discussion.
- Section 7.2. Added reference to SNOW.
- Section 7.2.3. was contradicting 7.6. Updated 7.2.3 to clarify that a Manager does not drop K0.
- Section 7.6. Visualiser role included.
- Section 7.7.1. Fixed the warm reboot security concern.
- Section 8.3. Visualiser role included.
- Section 6.6. updated to replace ephemeral TUID with Dynamic TUID and clarify they can be stored. Also added informative notes on collision and seeding.
- Section 6.2. Introduced the Equal Manager and Guest Manager concepts.
- Section 7.5. Added the key export for restricted guest mode.
- Section 10.4.3. Clarified query mechanics for Guest Mode.
- Added Section 13.3. to give examples of multi-manager mode operation.
- Added Section 7.2.5. to introduce Open Mode.
- Section 8.3. Added 0x01 = Open Mode.
- Section 8.6. Defined Open Mode operation.
- Section 11.6.8. Added bit field for Open Mode.
- Section 18. Added Open Mode definition.
- Section 18. Deleted Unprovisioned.
- Section 18. Updated Offboarded.
- Section 8.4. Added Scope to table.
- Section 6.2. Changed logical roles to 4.
- Section 8.2. Fixed ASCII UTF-8 issue.
- Changed TID_PATCH to TID_UNIVERSE
- Appendix A corrected for missing endpoint in uri.

V0.20 (4/5/2026)

- Section 11.6.9. Open Mode supported bit added.
- Section 7.2.5. Prohibit of Open Mode only devices.
- Added Section 11.10 TID summary.
- Errant TID_PATCH fixed.
- Section 11. Changed TID_POLL_REPLY to target all endpoint.
- Section 10.2.3. Updated to clarify that all poll responses, independent of endpoint, start with a poll response. This was previously ambiguous and has been clarified to ensure the CHANGE_COUNT mechanism is not compromised.
- TID_EP_CAPABILITY and TID_EP_REFRESH_CAPABILITY moved to the QUERY_CONFIG category.
- Section 11.2.5. Updated to modern frame rates.
- Section 11.8.2. Corrected target to Root to match URI (typo).
- Section 11.6.4. Referenced RDM for version.
- Section 15, 14.1.3, 11.6.6, 9.2.3 updated for changes to Multicast Folding.

V1.0 (9/5/2026)

- Section 8.3. Typo
- Section 13.3. Clarified Km_local status in example.
- Section 8.6.1. Updated c & d for typo.
- Section 12.3/4. Updated to resolve logical error.
- Section 8.3. Updated for better resolution of Session exhaustion.
- Section 10.6.3. Integrated TID_PRIORITY and added informative re sACN and Art-Net expectations.

- Section 8.3. Typo in hex number.
- Appendix C. Updated for new TIDs
- Section 14.1.3. Corrected example.
- Section 7.5.1. Informative note to Guest Access.
- Section 8.3 & 10.2.1. Reduced HMAC length to zero on mode 0x01 and 0xff. (Beacon Mode Wire Break).
- Section 8.3. Session and Sequence exemption for Open Mode and Beacon Mode.
- Section 10.2.2. Open Mode clarification.
- Section 10.6.2. Renamed to encompass open mode.
- Section 11.7.11. TID_EP_PROTOCOL added
- Section 10.6.4. Defined stream loss on non-Sig-Net protocols.
- Section 10.5.4 added. Gateway responsibilities. Queue polling, ACK_OVERFLOW handling, ACK_TIMER handling.
- Section 10.5.2. Updated for state sync and background get anti-overload mandate.
- Section 10.2.2. Open mode clarification.
- Section 10.6.2. Open mode clarification.

Copyright © Singularity (UK) Ltd 2026. All rights reserved.
Sig-Net™, Multicast Folding™, SNACtest™ and SNOW™ are trademarks of Singularity (UK) Ltd.